
TRÍ TUỆ NHÂN TẠO

Machine Learning

Phần 1 — Basic

Biên soạn:

ĐẶNG TẤN QUÍ

SĐT: 0377 122 210

2026

Mục lục

Lời nói đầu	3
1 Khái niệm nền (đọc trước)	4
1.1 Machine Learning là gì?	4
1.2 Các thành phần cốt lõi: dữ liệu, đặc trưng, mô hình	4
1.3 Train / validation / test và overfitting	4
1.4 Các kiểu học máy	5
1.5 Chu trình giải bài toán ML (nhìn từ góc “thực hành”)	7
2 Home (Trang chủ) — Tổng quan học máy	9
2.1 Học máy hoạt động như thế nào?	9
2.2 Một số thuật toán học máy phổ biến	10
2.3 Vì sao học máy quan trọng?	10
2.4 Ứng dụng của học máy	11
2.5 Điều kiện tiên quyết	11
2.6 FAQ — Câu hỏi thường gặp	11
3 Introduction — Giới thiệu về học máy	12
3.1 Học máy hoạt động như thế nào?	13
3.2 Vì sao cần học máy?	14
3.3 Lược sử học máy	14
3.4 Use cases của học máy	14
3.5 Ưu điểm và nhược điểm của học máy	15
3.5.1 Ưu điểm	15
3.5.2 Nhược điểm	15
3.6 Thách thức và vấn đề đạo đức	15
3.7 ML algorithms vs Traditional programming	15
3.8 ML vs Deep Learning; ML vs Generative AI	16
3.9 Tương lai của học máy	16
3.10 Cách học Machine Learning (5 bước)	17
4 Getting Started — Bắt đầu với học máy	17
4.1 Điều kiện tiên quyết để bắt đầu	17
4.1.1 Ngôn ngữ lập trình: Python hoặc R	17
4.1.2 Thư viện và packages	18
4.1.3 Toán và thống kê	18
5 Basic Concepts — Các khái niệm cơ bản	18
5.1 Các thành phần cốt lõi (tóm tắt)	18
5.2 Overfitting (Quá khớp)	18
5.3 Underfitting (Thiếu khớp)	19
5.4 Khi nào nên “làm cho máy học”?	19
5.5 Định nghĩa học máy của Tom Mitchell và mô hình T-P-E	20
5.5.1 Task (T)	20
5.5.2 Experience (E)	20
5.5.3 Performance (P)	21

6	Ecosystem — Hệ sinh thái Python cho học máy	21
6.1	Python Machine Learning Ecosystem	21
6.2	Vì sao chọn Python cho Machine Learning?	21
6.3	Điểm mạnh và điểm yếu của Python	22
6.4	Cài đặt Python	22
6.4.1	Cài Python riêng lẻ	22
6.4.2	Cài bằng Anaconda	23
6.5	IDE (Integrated Development Environment)	23
6.5.1	Jupyter Notebook	23
6.6	Python libraries và packages	24
7	Python Libraries — Các thư viện Python cho học máy	27
7.1	Danh sách thư viện ML phổ biến	27
7.2	NumPy	27
7.3	Pandas	28
7.4	SciPy	29
7.5	Scikit-learn	30
7.6	PyTorch	31
7.7	TensorFlow	31
7.8	Keras	32
7.9	Matplotlib	32
7.10	Seaborn	33
7.11	OpenCV	33
7.12	NLTK	33
7.13	spaCy	34
8	Applications — Ứng dụng của học máy	34
8.1	Danh mục các lĩnh vực ứng dụng	34
8.2	Image and Speech Recognition	35
8.3	Natural Language Processing (NLP)	35
8.4	Finance Sector	36
8.4.1	Fraud Detection	36
8.4.2	Algorithmic Trading	36
8.5	E-commerce and Retail	36
8.5.1	Recommendation Systems	37
8.5.2	Demand Forecasting	37
8.5.3	Customer Segmentation	37
8.6	Automotive Sector	37
8.7	Computer Vision	37
8.8	Manufacturing and Industries	38
8.9	Healthcare Sector	38
9	Life Cycle — Vòng đời dự án học máy	39
9.1	Machine Learning Life Cycle là gì?	39
9.2	Các pha chính trong vòng đời	39
9.3	Problem Definition	39
9.4	Data Preparation	40
9.4.1	Data Collection	40
9.4.2	Data Preprocessing	40
9.4.3	Phân tích dữ liệu (Analyzing Data)	41

9.4.4	Tạo feature mới và chọn feature (Feature Engineering and Selection) . . .	41
9.5	Model Development	41
9.6	Model Deployment	42
9.7	Monitoring and Maintenance	42
10	Required Skills — Các kỹ năng cần có cho học máy	43
10.1	Danh sách kỹ năng chính	43
10.2	Programming Skills	44
10.3	Statistics and Mathematics	44
10.3.1	Statistics	44
10.3.2	Mathematics	45
10.3.3	Mathematical Notation	45
10.4	Probability Theory	45
10.5	Optimization Problem	45
10.6	Data Structures	46
10.7	Data Preprocessing	46
10.8	Data Visualization	46
10.9	Machine Learning Algorithms	46
10.10	Neural Networks & Deep Learning	46
10.11	Natural Language Processing	47
10.12	Problem-solving Skills	47
10.13	Communication Skills	47
10.14	Business Acumen	47
11	Implementation — Triển khai học máy trong thực tế	48
11.1	Các bước triển khai học máy	49
11.1.1	Data Collection and Preparation	49
11.1.2	Data Exploration and Visualization	49
11.1.3	Feature Selection and Engineering	49
11.1.4	Model Selection and Training	49
11.1.5	Model Evaluation	49
11.1.6	Model Tuning	50
11.1.7	Deployment and Monitoring	50
11.2	Chọn ngôn ngữ và IDE để phát triển ML	50
11.2.1	Language Choice	50
11.2.2	IDEs	51
12	Challenges & Common Issues — Thách thức và vấn đề thường gặp	51
12.1	Overfitting	51
12.2	Underfitting	51
12.3	Data Quality Issues	52
12.4	Imbalanced Datasets	52
12.5	Model Interpretability	52
12.6	Generalization	52
12.7	Scalability	53
12.8	Ethical Considerations	53

13 Limitations — Giới hạn của học máy	53
13.1 Dependence on Data Quality	54
13.2 Lack of Transparency	54
13.3 Limited Applicability	54
13.4 High Computational Costs	54
13.5 Data Privacy Concerns	54
13.6 Ethical Considerations	54
13.7 Dependence on Experts	55
13.8 Lack of Creativity and Intuition	55
13.9 Limited Interpretability	55
14 Real-Life Examples — Ví dụ học máy trong đời sống	55
14.1 Danh sách ví dụ	55
14.2 Virtual Assistants and Chatbots	56
14.3 Fraud Detection in Banking and Finance	56
14.4 Healthcare Diagnosis and Treatment	57
14.5 Autonomous Vehicles	57
14.6 Recommendation Systems	57
14.7 Target Advertising	58
14.8 Image Recognition	58
15 Data Structure — Cấu trúc dữ liệu trong học máy	58
15.1 Data Structure là gì?	58
15.2 Các cấu trúc dữ liệu thường dùng trong ML	59
15.2.1 Arrays	59
15.2.2 Lists	59
15.2.3 Dictionaries	60
15.2.4 Linked Lists	60
15.2.5 Stack and Queue	60
15.2.6 Trees	60
15.2.7 Graphs	61
15.2.8 Hash Maps	61
15.3 Cấu trúc dữ liệu được dùng trong ML như thế nào?	61
15.3.1 Storing and Accessing Data	61
15.3.2 Pre-processing Data	61
15.3.3 Creating Feature Vectors	62
15.3.4 Building Decision Trees	62
15.3.5 Building Graphs	62
16 Mathematics — Toán học nền tảng cho học máy	62
16.1 Đại số tuyến tính (Linear Algebra)	62
16.2 Giải tích (Calculus)	63
16.3 Lý thuyết xác suất (Probability Theory)	64
16.4 Thống kê (Statistics)	65
17 Artificial Intelligence — Trí tuệ nhân tạo và học máy	65
17.1 Trí tuệ nhân tạo (AI) là gì?	66
17.2 Các nhánh của AI	66
17.2.1 Robotics	67
17.2.2 Computer Vision	67

17.2.3 Expert Systems	67
17.2.4 Fuzzy Logic	67
17.2.5 Neural Networks	67
17.2.6 Natural Language Processing (NLP)	67
17.3 AI và ML: tổng quan so sánh	68
17.4 Bảng: AI và Machine Learning	68
18 Neural Networks — Mạng nơ-ron và học máy	69
18.1 Học máy là gì?	69
18.2 Mạng nơ-ron là gì?	69
18.3 Machine Learning và Neural Networks	70
19 Deep Learning — Học sâu và học máy	70
19.1 Học máy là gì?	71
19.2 Học sâu (Deep Learning) là gì?	71
19.3 Khác biệt giữa Machine Learning và Deep Learning	72
19.4 So sánh sâu hơn	72
19.5 AI không nhất thiết qua ML	73
20 Getting Datasets — Lấy bộ dữ liệu cho học máy	73
20.1 Dataset là gì?	74
20.2 Các loại dataset	74
20.3 Lấy dataset cho Machine Learning	74
20.4 Dataset công khai / mã nguồn mở phổ biến	74
20.4.1 Kaggle Datasets	75
20.4.2 AWS Datasets	75
20.4.3 Google Dataset Search Engine	75
20.4.4 UCI Machine Learning Repository	75
20.4.5 Microsoft Dataset	75
20.4.6 Scikit-learn Dataset	76
20.4.7 HuggingFace Datasets Hub	76
20.4.8 Government Datasets	76
20.5 Data Scraping	77
20.6 Data Purchase	77
20.7 Data Collection	77
20.8 Chiến lược có dataset chất lượng cao	77
21 Categorical Data — Dữ liệu phân loại trong học máy	78
21.1 Dữ liệu phân loại là gì?	78
21.2 Kỹ thuật xử lý dữ liệu phân loại	78
21.3 One-Hot Encoding	78
21.4 Label Encoding	79
21.5 Frequency Encoding	80
21.6 Target Encoding	81
21.7 Binary Encoding	82
22 Data Loading — Nạp dữ liệu vào dự án học máy	83
22.1 Lưu ý khi nạp dữ liệu CSV	83
22.1.1 File Header (hàng tiêu đề)	83
22.1.2 Comments (dòng chú thích)	84

22.1.3	Delimiter (ký tự phân tách)	84
22.1.4	Quotes (dấu ngoặc kép)	84
22.2	Các cách nạp file CSV trong Python	85
22.2.1	Dùng module CSV (built-in)	85
22.2.2	Dùng thư viện Pandas	85
22.2.3	Dùng thư viện NumPy	85
22.2.4	Dùng thư viện SciPy	85
22.2.5	Dùng thư viện scikit-learn	86
23	Data Understanding — Hiểu dữ liệu trước khi huấn luyện	86
23.1	Các bước trong giai đoạn hiểu dữ liệu	87
23.2	Vì sao phải hiểu dữ liệu trước khi đưa vào dự án ML?	87
23.3	Hiểu dữ liệu bằng thống kê	87
23.3.1	Xem dữ liệu thô (Looking at Raw Data)	87
23.3.2	Kích thước dữ liệu (Dimensions)	88
23.3.3	Kiểu dữ liệu từng thuộc tính	88
23.3.4	Tóm tắt thống kê (describe)	89
23.3.5	Phân phối lớp (Class Distribution)	89
23.3.6	Tương quan giữa các thuộc tính	90
23.3.7	Độ lệch phân phối (Skew)	90
24	Data Preparation — Chuẩn bị dữ liệu cho học máy	91
24.1	Data Preparation là gì?	91
24.2	Tầm quan trọng	92
24.3	Các bước trong quy trình chuẩn bị dữ liệu	92
24.4	Data Collection và tích hợp	93
24.5	Data Cleaning	93
24.6	Data Transformation	93
24.6.1	Scaling (Min-Max)	94
24.6.2	Normalization (L1 và L2)	94
24.6.3	Standardization	95
24.6.4	Binarization	95
24.6.5	Encoding (Label Encoding)	96
24.6.6	Log Transformation	97
24.7	Data Reduction	97
24.8	Data Splitting	97
24.9	Ví dụ Python: breast cancer dataset	97
24.10	Data Preparation và Feature Engineering	98
25	Models — Các loại mô hình và phương pháp học máy	98
25.1	Supervised Learning (tổng quan)	100
25.1.1	Classification	100
25.1.2	Regression	100
25.2	Unsupervised Learning (tổng quan)	101
25.2.1	Clustering	101
25.2.2	Association	101
25.2.3	Dimensionality Reduction	102
25.2.4	Anomaly Detection	102
25.3	Semi-supervised Learning (tổng quan)	102
25.4	Reinforcement Learning (tổng quan)	102

26 Supervised vs Unsupervised Learning — So sánh hai hướng học	103
26.1 Bảng khác biệt chính	104
26.2 Chọn Supervised hay Unsupervised?	105
26.3 Semi-supervised Learning (lối thoát trung gian)	105
27 Supervised Learning — Học máy có giám sát	105
27.1 Supervised Machine Learning là gì?	105
27.2 Cách Supervised Learning hoạt động	105
27.3 Hai loại bài toán Supervised	106
27.3.1 Classification (phân loại)	106
27.3.2 Regression (hồi quy)	106
27.4 Thuật toán Supervised Learning	106
27.4.1 Linear Regression	106
27.4.2 K-Nearest Neighbors (kNN)	106
27.4.3 Decision Trees	108
27.4.4 Naive Bayes	109
27.4.5 Logistic Regression	109
27.4.6 Support Vector Machines (SVM)	110
27.4.7 Random Forest	111
27.4.8 Gradient Boosting	112
27.5 Ưu điểm Supervised Learning	112
27.6 Nhược điểm	112
27.7 Ứng dụng	113
28 Unsupervised Learning — Học máy không giám sát	113
28.1 Unsupervised Machine Learning là gì?	113
28.2 Cách Unsupervised Learning hoạt động	114
28.3 Ba nhóm phương pháp	114
28.3.1 Clustering	114
28.3.2 Association Rule Mining	115
28.3.3 Dimensionality Reduction	115
28.4 Thuật toán Unsupervised Learning	115
28.5 Ưu điểm	115
28.6 Nhược điểm	116
28.7 Ứng dụng	116
28.8 Anomaly Detection	116
29 Semi-Supervised Learning — Học máy bán giám sát	116
29.1 Semi-Supervised Learning là gì?	117
29.2 So với Supervised Learning	118
29.3 So với Unsupervised Learning	118
29.4 Khi nào chọn Semi-Supervised?	118
29.5 Cách Semi-Supervised Learning hoạt động	118
29.6 Các giả định (assumptions)	119
29.6.1 Giả định mượt mà (Smoothness Assumption)	119
29.6.2 Giả định cụm (Cluster Assumption)	119
29.6.3 Giả định mật độ thấp (Low Density Separation)	119
29.6.4 Giả định manifold (Manifold Assumption)	119
29.7 Kỹ thuật Semi-Supervised	119

29.8 Thách thức	120
29.9 Ứng dụng	120
30 Reinforcement Learning — Học tăng cường	120
30.1 Reinforcement Learning là gì?	120
30.2 Cách Reinforcement Learning hoạt động	121
30.3 Bốn thành phần chính (ngoài agent và môi trường)	121
30.4 Markov Decision Processes (MDP)	122
30.5 Các bước trong quy trình RL	122
30.6 Hai loại Reinforcement	123
30.7 Loại thuật toán RL	123
30.8 Ưu điểm	123
30.9 Nhược điểm	123
30.10 Ứng dụng	124
30.10.1 Robotics	124
30.10.2 Natural Language Processing	124
30.11 Reinforcement Learning vs Supervised Learning	124

Lời nói đầu

Nguồn

Tài liệu biên soạn và dịch từ Tutorialspoint Machine Learning: https://www.tutorialspoint.com/machine_learning/.

Đặng Tấn Quý

1 Khái niệm nền (đọc trước)

1.1 Machine Learning là gì?

Machine Learning (ML)

Học máy là một nhánh của **Trí tuệ nhân tạo (AI)** tập trung vào việc xây dựng thuật toán/mô hình có thể **học từ dữ liệu** để dự đoán hoặc ra quyết định mà không cần viết quy tắc tường minh cho mọi trường hợp.

ML giúp giải bài toán nào?

ML đặc biệt hữu ích khi:

- Quy tắc quá phức tạp/khó mô tả bằng tay (nhận dạng ảnh/giọng nói, NLP).
- Môi trường thay đổi theo thời gian (hành vi người dùng, mạng, rủi ro gian lận).
- Cần ra quyết định dựa trên dữ liệu với quy mô lớn (tự động hóa).

1.2 Các thành phần cốt lõi: dữ liệu, đặc trưng, mô hình

Dữ liệu (data)

Dữ liệu là “nguyên liệu” của ML. Dữ liệu có thể là có cấu trúc (bảng), bán cấu trúc, hoặc phi cấu trúc (văn bản, ảnh, âm thanh). Chất lượng và số lượng dữ liệu ảnh hưởng mạnh đến chất lượng mô hình.

Đặc trưng (feature)

Feature là các biến/thuộc tính mô tả đầu vào. Chọn feature phù hợp (feature selection) và tạo feature mới (feature engineering) thường quyết định phần lớn hiệu quả mô hình.

Mô hình (model)

Mô hình là biểu diễn toán học học được mối liên hệ giữa feature và đầu ra (dự đoán/quyết định). Mô hình được học từ tập train và đánh giá trên tập validation/test để kiểm tra khả năng tổng quát.

1.3 Train / validation / test và overfitting

Huấn luyện (training)

Training là quá trình cho thuật toán xem nhiều ví dụ dữ liệu và điều chỉnh tham số nội bộ để giảm sai số dự đoán.

Kiểm thử (testing)

Testing là đánh giá mô hình trên dữ liệu **chưa từng thấy** nhằm ước lượng khả năng tổng quát hóa.

Overfitting và Underfitting

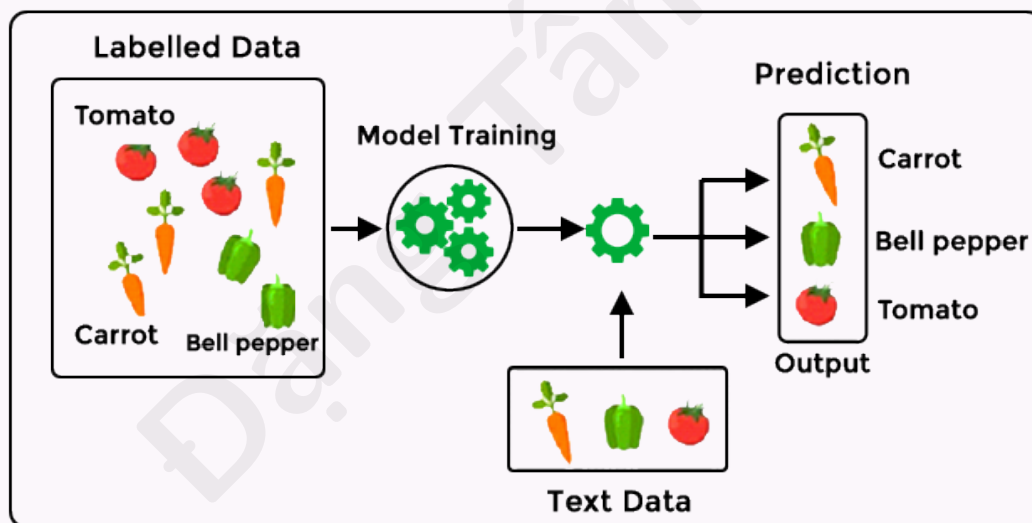
- **Overfitting**: mô hình quá phức tạp, bám sát train, kém trên dữ liệu mới.
- **Underfitting**: mô hình quá đơn giản, không học được cấu trúc, kém cả train và test.

1.4 Các kiểu học máy

Học có giám sát (Supervised)

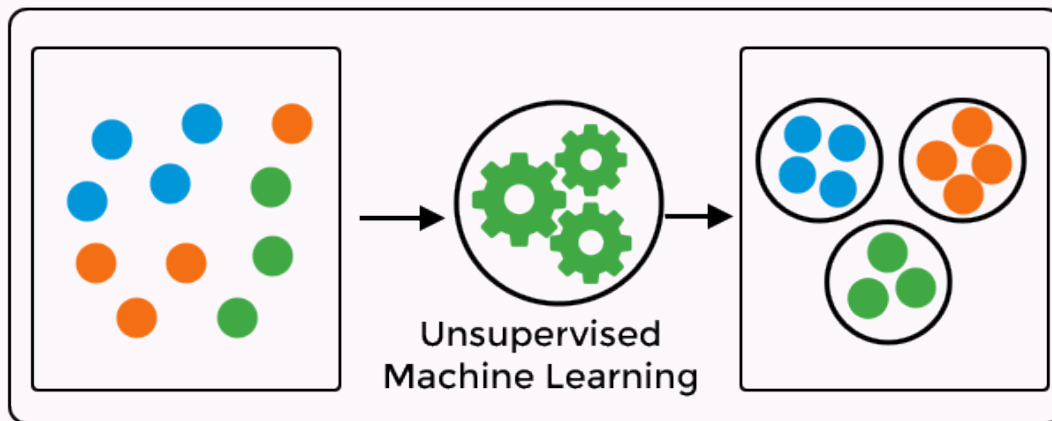
Dữ liệu có **nhãn** (đầu ra đúng). Hai nhóm bài toán phổ biến: **hồi quy** (đầu ra số) và **phân loại** (đầu ra lớp).

Minh họa supervised

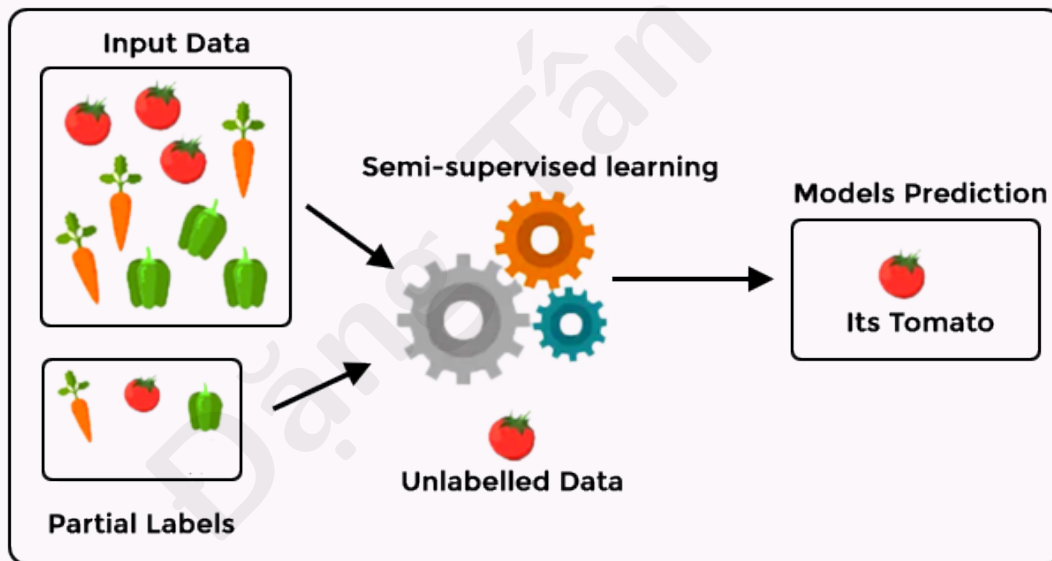


Học không giám sát (Unsupervised)

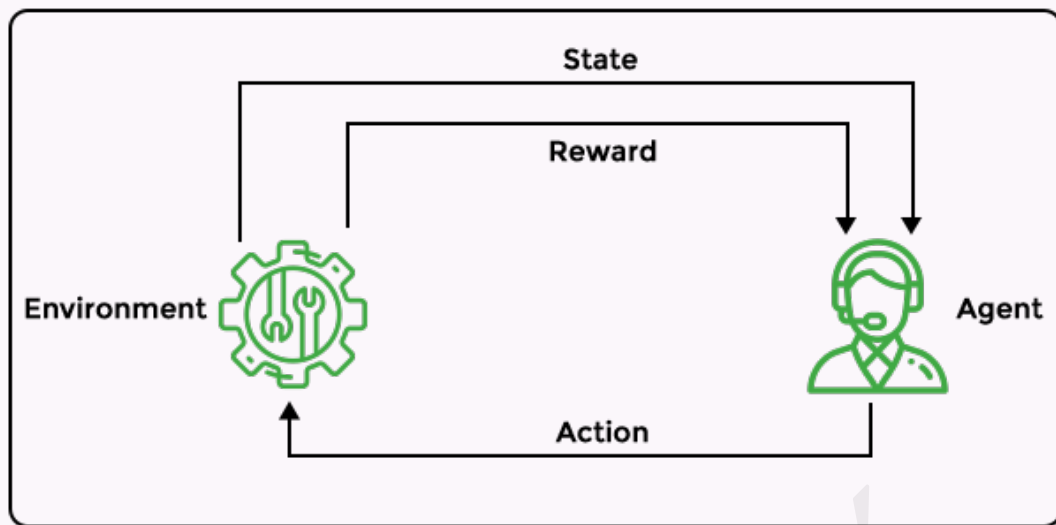
Dữ liệu **không có nhãn**. Mục tiêu thường là tìm cấu trúc/nhóm trong dữ liệu: phân cụm, phát hiện bất thường, giảm chiều, ...

Minh hoạ unsupervised**Bán giám sát (Semi-supervised)**

Kết hợp một phần nhỏ dữ liệu có nhãn và phần lớn dữ liệu không nhãn (thường do gán nhãn tốn kém).

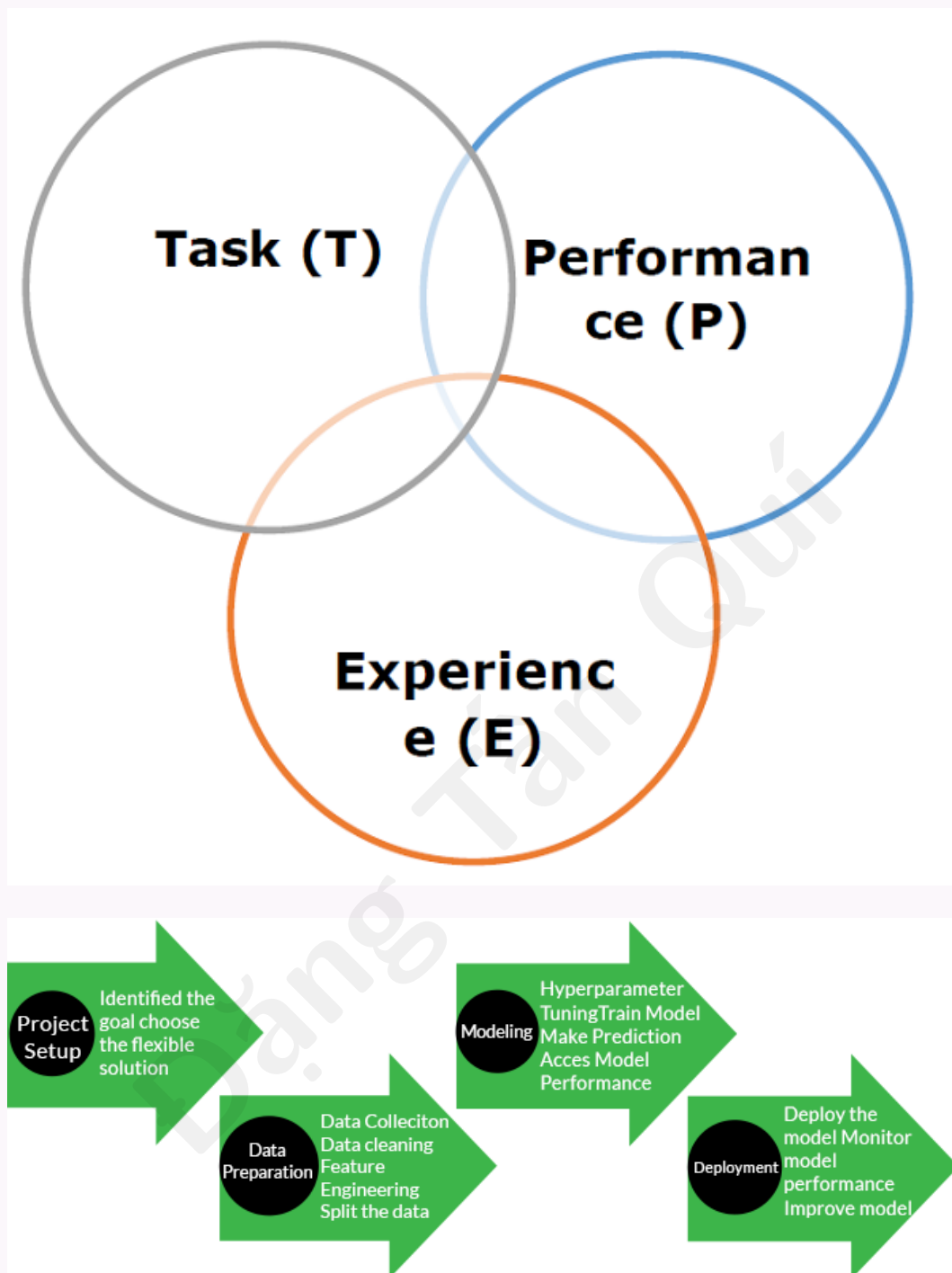
Minh hoạ semi-supervised**Tăng cường (Reinforcement)**

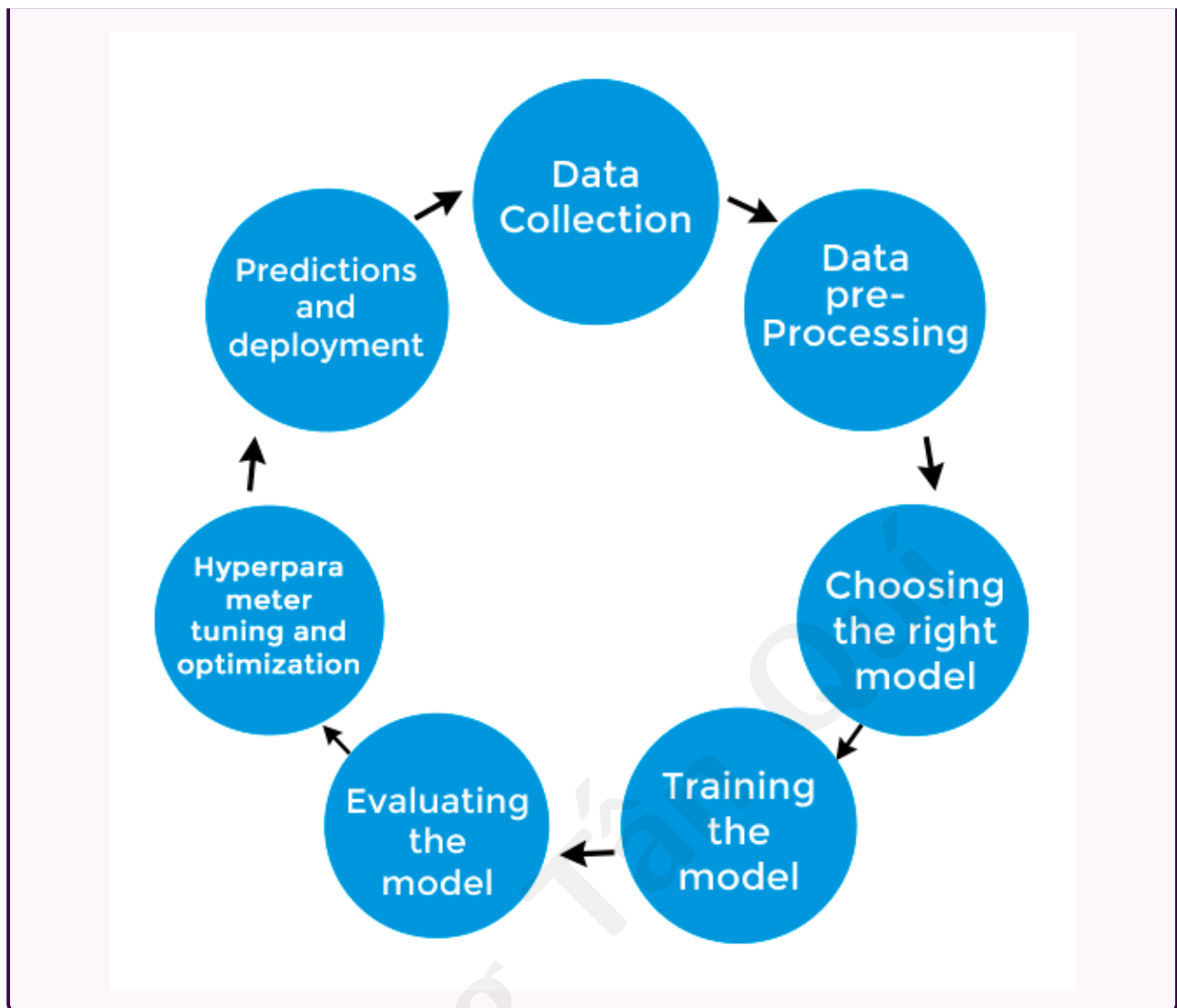
Tác tử (agent) tương tác với môi trường, nhận thưởng/phạt, học chính sách hành động để tối đa hóa phần thưởng dài hạn.

Minh hoạ reinforcement**1.5 Chu trình giải bài toán ML (nhìn từ góc “thực hành”)****Chuỗi bước điển hình**

1. Xác định bài toán và tiêu chí thành công.
2. Thu thập và tiền xử lý dữ liệu (làm sạch, thiếu dữ liệu, nhiễu).
3. Khám phá dữ liệu và trực quan hóa để hiểu phân phối/pattern.
4. Chọn/thiết kế feature.
5. Chọn mô hình, huấn luyện, đánh giá.
6. Tinh chỉnh siêu tham số, kiểm tra lại.
7. Triển khai, giám sát, cập nhật theo thời gian.

Hình minh hoạ khái niệm nền





2 Home (Trang chủ) — Tổng quan học máy

2.1 Học máy hoạt động như thế nào?

Quy trình tổng quát

Một quy trình học máy thường bao gồm: thiết lập dự án, chuẩn bị dữ liệu, xây mô hình và triển khai.

Các giai đoạn (sequential process)

1. **Thu thập dữ liệu (Data Collection)**: thu dữ liệu từ nhiều nguồn (database, file văn bản, ảnh, âm thanh, web scraping, ...), tổ chức về định dạng phù hợp (CSV hoặc database) và đảm bảo hữu ích cho bài toán.
2. **Tiền xử lý dữ liệu (Data Pre-processing)**: xóa trùng, sửa lỗi, xử lý thiếu dữ liệu (loại bỏ/điền), chuẩn hoá/định dạng.
3. **Chọn mô hình phù hợp (Choosing the Right Model)**: sau khi dữ liệu sẵn

sàng, chọn mô hình (linear regression, decision trees, neural networks, ...) tùy dữ liệu, bài toán, quy mô, độ phức tạp và tài nguyên tính toán.

4. **Huấn luyện mô hình (Training the Model)**: huấn luyện để mô hình dự đoán tốt hơn.
5. **Đánh giá mô hình (Evaluating the model)**: kiểm thử trên dữ liệu mới mà mô hình chưa thấy khi huấn luyện.
6. **Tối ưu và điều chỉnh siêu tham số (Hyperparameter Tuning and Optimization)**: thử các tổ hợp siêu tham số và cross-validation để mô hình hoạt động tốt trên nhiều tập dữ liệu.
7. **Dự đoán và triển khai (Predictions and Deployment)**: dùng mô hình đã tối ưu để dự đoán dữ liệu mới; triển khai vào môi trường thật để xử lý dữ liệu thực tế.

2.2 Một số thuật toán học máy phổ biến

Danh sách thuật toán

- **Neural Networks**: mô phỏng kiểu “nhiều nút liên kết”, dùng cho NLP, nhận dạng ảnh/giọng nói, tạo ảnh.
- **Linear Regression**: dự đoán số từ dữ liệu quá khứ; ví dụ ước lượng giá nhà.
- **Logistic Regression**: dự đoán “có/không”; ví dụ lọc spam, QC.
- **Clustering**: nhóm dữ liệu tương tự mà không có hướng dẫn.
- **Decision Trees**: cây quyết định để phân loại/dự đoán; dễ hiểu và kiểm tra.
- **Random Forests**: kết hợp nhiều cây quyết định để tăng chất lượng.

2.3 Vì sao học máy quan trọng?

Ý nghĩa tổng quát

Học máy quan trọng trong tự động hoá, trích xuất insight từ dữ liệu và hỗ trợ ra quyết định.

Một số lý do ML

- **Xử lý dữ liệu**: phân tích dữ liệu lớn từ mạng xã hội, cảm biến, ... để tìm pattern/insight.
- **Insight dựa trên dữ liệu**: tìm xu hướng/kết nối mà con người có thể bỏ sót.
- **Tự động hoá**: tự động hoá tác vụ lặp, giảm lỗi, tiết kiệm thời gian.
- **Cá nhân hoá**: phân tích sở thích người dùng để gợi ý nội dung/sản phẩm.
- **Dự báo**: dự báo kết quả tương lai (sales forecast, quản trị rủi ro, lập kế hoạch).

- **Nhận dạng mẫu:** ảnh, giọng nói, NLP.
- **Tài chính / bán lẻ / an ninh:** chấm điểm tín dụng, phát hiện gian lận, tối ưu chuỗi cung ứng, ...
- **Cải tiến liên tục:** mô hình được cập nhật với dữ liệu mới để thích nghi tốt hơn.

2.4 Ứng dụng của học máy

Một số ứng dụng thường gặp

- **Nhận dạng giọng nói:** chuyển giọng nói thành văn bản; trợ lý ảo.
- **Chăm sóc khách hàng:** chatbot, agent ảo, trả lời FAQ, đề xuất sản phẩm.
- **Thị giác máy tính:** phân tích ảnh/video; y tế; xe tự lái.
- **Hệ gợi ý:** đề xuất sản phẩm/phim/nội dung theo hành vi.
- **RPA:** tự động hoá thao tác lặp.
- **Giao dịch tự động:** AI-driven trading.
- **Phát hiện gian lận:** phát hiện giao dịch đáng ngờ trong tài chính.

2.5 Điều kiện tiên quyết

Prerequisites to Learn Machine LearningML

Nên quen với khái niệm dữ liệu/thông tin, dữ liệu có cấu trúc/phi cấu trúc/bán cấu trúc, xử lý dữ liệu, nền tảng AI; nắm dữ liệu có nhãn/không nhãn, trích chọn đặc trưng và ứng dụng trong ML.

2.6 FAQ — Câu hỏi thường gặp

Một số câu hỏi tiêu biểu

- ML là gì?
- Vì sao ML quan trọng?
- Các loại ML?
- Ứng dụng phổ biến?
- Thành phần cơ bản của hệ thống ML?
- Ngôn ngữ lập trình hay dùng?
- Supervised vs unsupervised khác gì?
- Thuật toán phổ biến?

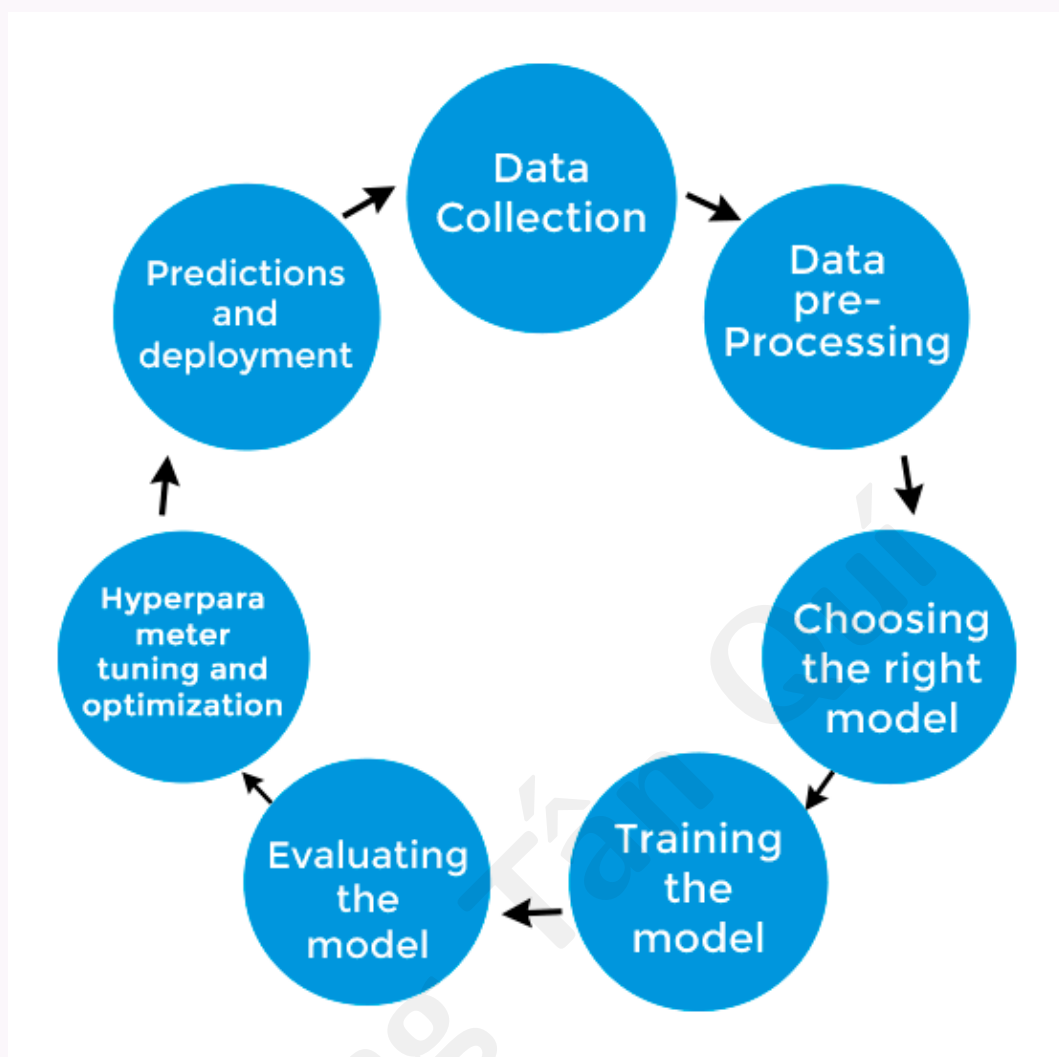
- Đánh giá mô hình thế nào?
- Thách thức phổ biến?
- Bắt đầu học ra sao?
- Các vấn đề đạo đức?
- ML vs AI?
- ML áp dụng cho loại dữ liệu nào?
- Thu thập/chuẩn bị dữ liệu ra sao?

3 Introduction — Giới thiệu về học máy

Bối cảnh: thời đại dữ liệu

Chúng ta đang sống trong “thời đại dữ liệu”: năng lực tính toán và lưu trữ tăng, dữ liệu tăng mỗi ngày. Thách thức lớn là **làm cho dữ liệu có ý nghĩa** (make sense of data). Doanh nghiệp/tổ chức xây hệ thống thông minh dựa trên Data Science, Data Mining và Machine Learning.

Hình minh hoạ quy trình ML



3.1 Học máy hoạt động như thế nào?

3 thành phần chính

Cơ chế “máy học từ mô hình” được chia thành 3 phần:

- **Decision Process:** dựa trên dữ liệu vào và nhãn đầu ra, mô hình hình thành một logic về pattern nhận diện được.
- **Cost Function:** đo lỗi giữa giá trị kỳ vọng và giá trị dự đoán; dùng để đánh giá hiệu năng.
- **Optimization Process:** tối thiểu hoá cost bằng cách điều chỉnh trọng số trong giai đoạn training; lặp đánh giá–tối ưu cho đến khi lỗi giảm.

3.2 Vì sao cần học máy?

Quyết định dựa trên dữ liệu ở quy mô lớn

Con người rất thông minh, còn AI vẫn chưa vượt con người ở nhiều mặt. Vậy tại sao vẫn cần “làm cho máy học”? Một lý do phù hợp: **ra quyết định dựa trên dữ liệu với hiệu quả và quy mô**.

Tự động hoá và quyết định dựa trên dữ liệu

Gần đây, tổ chức đầu tư mạnh vào AI/ML/DL để khai thác thông tin then chốt từ dữ liệu cho các nhiệm vụ thực tế. Các quyết định dựa trên dữ liệu có thể dùng thay cho việc lập trình logic ở các bài toán vốn khó lập trình bằng tay. Ta vẫn cần trí tuệ con người, nhưng cũng cần giải quyết bài toán thực tế hiệu quả ở quy mô lớn.

3.3 Lược sử học máy

Các mốc

- 1959: Arthur Samuel tạo chương trình ước tính xác suất thắng trong trò checkers.
- 1960–1970: giai đoạn neural networks phát triển.
- Sau đó ML tiến triển nhờ phương pháp thống kê như Bayesian networks, decision tree learning.
- 2010s: “cách mạng Deep Learning” với NLP, CNN, speech recognition.
- Hiện nay ML trở thành công nghệ phổ biến trong nhiều lĩnh vực (y tế, tài chính, giao thông, ...).

3.4 Use cases của học máy

Một số use case

- **Recommendation system:** engine gợi ý theo sở thích/engagement trước đó.
- **Voice assistants:** nhận dạng tiếng nói + xử lý ngôn ngữ + tổng hợp giọng.
- **Fraud detection:** phát hiện hoạt động bất thường (thường ở tài chính).
- **Health care:** chẩn đoán, cải thiện độ chính xác ảnh y khoa, cá nhân hoá điều trị.
- **RPA:** tự động hoá tác vụ lặp.
- **Drive-less cars:** xe tự lái; ví dụ Tesla.
- **Computer vision:** hiểu ảnh/video; nhận diện khuôn mặt.

3.5 Ưu điểm và nhược điểm của học máy

3.5.1 Ưu điểm

Advantages of Machine Learning

- **Tự động hoá:** tác vụ lặp có thể tự động hoá; ví dụ chatbot giảm thời gian chờ.
- **Cải thiện trải nghiệm và ra quyết định:** phân tích dữ liệu lớn để ra quyết định; cá nhân hoá sản phẩm/dịch vụ.
- **Ứng dụng rộng:** áp dụng trong nhiều lĩnh vực.
- **Cải tiến liên tục:** mô hình có thể học tiếp khi được retrain bằng dữ liệu mới.

3.5.2 Nhược điểm

Disadvantages of Machine Learning

- **Thu thập dữ liệu khó:** cần dữ liệu liên quan, không thiên lệch, chất lượng tốt.
- **Kết quả có thể không chính xác:** độ tin cậy phụ thuộc dữ liệu và thuật toán.
- **Dễ sai nếu dữ liệu/thuật toán sai:** ví dụ dữ liệu nhỏ làm mô hình học pattern kém, dự đoán lệch.
- **Cần bảo trì:** mô hình phải được giám sát và cập nhật liên tục.

3.6 Thách thức và vấn đề đạo đức

Challenges in Machine Learning

- **Data privacy:** dữ liệu có thể nhạy cảm; cần ẩn danh, bảo vệ dữ liệu, cân bằng sử dụng và quyền riêng tư.
- **Impact on jobs:** tự động hoá thay đổi việc làm; đồng thời tạo nghề mới (data scientist, ML engineer, ...).
- **Bias & discrimination:** tránh dùng thuộc tính nhạy cảm sai cách (race, gender, ...).
- **Ethical consideration:** minh bạch, trách nhiệm giải trình, tác động xã hội.

3.7 ML algorithms vs Traditional programming

So sánh theo tiêu chí

Tiêu chí	Thuật toán học máy	Lập trình truyền thống
Cách tiếp cận	Máy học bằng cách huấn luyện trên dữ liệu lớn.	Lập trình viên viết quy tắc tường minh (code) để máy làm theo.
Dữ liệu	Phụ thuộc mạnh vào dữ liệu; dữ liệu quyết định chất lượng mô hình.	Phụ thuộc ít hơn; đầu ra chủ yếu phụ thuộc logic được mã hoá.
Độ phức tạp bài toán	Hợp bài toán phức tạp (image segmentation, NLP) cần nhận diện pattern.	Hợp bài toán có đầu ra/logic rõ ràng.
Tính linh hoạt	Linh hoạt, thích nghi khi retrain bằng dữ liệu mới.	Ít linh hoạt; thay đổi phải sửa code thủ công.
Kết quả	Khó dự đoán hoàn toàn vì phụ thuộc dữ liệu, mô hình, ...	Có thể dự đoán chính xác nếu logic đúng và đầy đủ.

3.8 ML vs Deep Learning; ML vs Generative AI**ML vs Deep Learning**

Deep learning là một nhánh của ML, khác nhau ở cách thuật toán học: deep learning dùng cấu trúc “giống não người” (mạng sâu). Deep learning thường hiệu quả hơn ở bài toán phức tạp (ví dụ xe tự lái).

ML vs Generative AI

Machine Learning chủ yếu dùng cho dự báo và ra quyết định; Generative AI tập trung tạo nội dung mới (ảnh/video/văn bản) theo pattern đã học.

3.9 Tương lai của học máy**Một số xu hướng**

Nguồn đề cập các xu hướng như:

- AutoML và synthetic data giúp ML dễ tiếp cận hơn.
- Quantum computing được kỳ vọng tăng tốc xử lý dữ liệu.
- Multi-modal AI: phân tích nhiều dạng input (text/speech/image/sensor).
- Edge computing, robotics, ...

3.10 Cách học Machine Learning (5 bước)

Lộ trình 5 bước

1. Học nền tảng (dữ liệu, thống kê, thuật toán, Python).
2. Chọn framework (TensorFlow, PyTorch, scikit-learn, ...).
3. Luyện với dữ liệu thật (Kaggle, UCI, ...).
4. Tự làm dự án (từ đơn giản đến phức tạp).
5. Tham gia cộng đồng (forum/meetup) để học hỏi và nhận phản hồi.

4 Getting Started — Bắt đầu với học máy

Mục tiêu

Phần này mô tả học máy là gì, các loại học máy, một số thuật toán tiêu biểu, use case, và điều kiện tiên quyết để bắt đầu.

4.1 Điều kiện tiên quyết để bắt đầu

Học ML dễ hơn khi chuẩn bị đúng

Để bắt đầu học ML, bạn nên nắm một số nền tảng về khoa học máy tính, đồng thời quen với lập trình, thư viện và toán-thống kê.

4.1.1 Ngôn ngữ lập trình: Python hoặc R

Gợi ý

Có nhiều ngôn ngữ dùng được (C++, Java, Python, R, Julia, ...), nhưng Python rất phổ biến trong ML và Data Science.

Các chủ đề Python cơ bản nên nắm

- Variables, basic data types.
- Data structures (list, set, dictionaries).
- Loops and conditionals.
- Functions.
- String formatting.
- Classes and objects.

4.1.2 Thư viện và packages

Thư viện gợi ý

- NumPy, Pandas (tiền xử lý).
- scikit-learn.
- Matplotlib.

4.1.3 Toán và thống kê

Mức toán cần thiết

- Đại số tuyến tính.
- Giải tích cơ bản.
- Xác suất-thống kê.

5 Basic Concepts — Các khái niệm cơ bản

Mục tiêu

Bài này giới thiệu các khái niệm cơ bản trong học máy: dữ liệu, đặc trưng, mô hình, huấn luyện/kiểm thử, overfitting/underfitting, và khi nào nên dừng học máy.

5.1 Các thành phần cốt lõi (tóm tắt)

5.2 Overfitting (Quá khớp)

Overfitting

Overfitting xảy ra khi mô hình quá phức tạp và khớp dữ liệu huấn luyện “quá sát”. Hệ quả là mô hình hoạt động kém trên dữ liệu mới vì đã “chuyên biệt” cho tập train.

Cách giảm overfitting

- Dùng tập validation để đánh giá trong quá trình phát triển.
- Dùng **regularization** để làm mô hình đơn giản hơn.

5.3 Underfitting (Thiếu khớp)

Underfitting

Underfitting xảy ra khi mô hình quá đơn giản và không thể nắm bắt pattern/mối quan hệ trong dữ liệu. Điều này dẫn tới hiệu năng kém trên cả train và test.

Cách giảm underfitting

- Tăng độ phức tạp mô hình.
- Thu thập thêm dữ liệu.
- Giảm regularization.
- Feature engineering.

Cân bằng giữa underfitting và overfitting

Giảm underfitting là một bài toán cân bằng giữa độ phức tạp mô hình và lượng dữ liệu hiện có. Tăng độ phức tạp có thể giúp bớt underfitting, nhưng nếu dữ liệu không đủ để “đỡ” độ phức tạp đó, mô hình có thể chuyển sang overfitting. Vì vậy cần theo dõi hiệu năng và điều chỉnh độ phức tạp khi cần.

5.4 Khi nào nên “làm cho máy học”?

Vì sao phải để máy học thay vì viết quy tắc?

Ngoài lý do “cần ML”, còn có câu hỏi: trong kịch bản nào ta nên làm cho máy học? Có nhiều tình huống cần ra quyết định dựa trên dữ liệu với hiệu quả và quy mô lớn.

Một số tình huống

- **Thiếu chuyên gia con người:** ví dụ điều hướng ở vùng chưa biết hoặc nhiệm vụ trong không gian.
- **Tình huống động:** hành vi thay đổi theo thời gian; ví dụ kết nối mạng, hạ tầng tổ chức.
- **Khó chuyển chuyên môn thành bài toán tính toán:** con người có trực giác/chuyên môn nhưng không viết được quy tắc; ví dụ nhận dạng giọng nói, tác vụ nhận thức.

5.5 Định nghĩa học máy của Tom Mitchell và mô hình T–P–E

Định nghĩa (Tom Mitchell)

Một chương trình máy tính được gọi là “học” từ kinh nghiệm E đối với một lớp nhiệm vụ T và thước đo hiệu năng P , nếu hiệu năng của nó khi thực hiện nhiệm vụ trong T , được đo bằng P , được cải thiện nhờ kinh nghiệm E .

Ba thành phần của bài toán học

Định nghĩa trên nhấn mạnh ba thành phần (cũng là thành phần chính của thuật toán học):

- **Task (T)**: nhiệm vụ.
- **Performance (P)**: thước đo hiệu năng.
- **Experience (E)**: kinh nghiệm (dữ liệu/quá trình học).

Cách diễn giải ngắn

ML là lĩnh vực AI gồm các thuật toán học mà:

- Cải thiện hiệu năng P .
- Khi thực hiện nhiệm vụ T .
- Theo thời gian nhờ kinh nghiệm E .

5.5.1 Task (T)

Task theo góc nhìn ML

Theo góc nhìn “bài toán thực”, T có thể là dự đoán giá nhà hoặc chọn chiến lược marketing tốt. Theo góc nhìn ML, T thường là những nhiệm vụ mà cách lập trình truyền thống khó giải vì phải xử lý trên nhiều điểm dữ liệu. Ví dụ về nhiệm vụ ML: classification, regression, structured annotation, clustering, transcription, ...

5.5.2 Experience (E)

Kinh nghiệm là gì?

Kinh nghiệm là kiến thức thu được từ các điểm dữ liệu cung cấp cho thuật toán. Khi được cung cấp dataset, mô hình chạy lặp và học các pattern nội tại; phần học được gọi là experience E . Theo tương tự với con người: con người học từ tình huống, quan hệ, ... Supervised/unsupervised/reinforcement là các cách khác nhau để “tạo kinh nghiệm”. Kinh nghiệm của mô hình sẽ được dùng để giải nhiệm vụ T .

5.5.3 Performance (P)

Thước đo hiệu năng

Thước đo hiệu năng cho biết thuật toán có đang làm đúng kỳ vọng không. P là metric định lượng đánh giá mô hình thực hiện T bằng kinh nghiệm E tốt đến đâu. Một số metric: accuracy score, F1 score, confusion matrix, precision, recall, sensitivity, ...

6 Ecosystem — Hệ sinh thái Python cho học máy

Vì sao nói về “hệ sinh thái”?

Hệ sinh thái học máy là tập hợp công cụ/công nghệ dùng để phát triển ứng dụng ML. Python có nhiều thư viện và công cụ tạo thành các “thành phần” của hệ sinh thái này, khiến Python trở thành ngôn ngữ quan trọng cho ML và Data Science.

6.1 Python Machine Learning Ecosystem

Machine learning ecosystem

Hệ sinh thái ML là tập hợp các công cụ và công nghệ được dùng để phát triển ứng dụng ML. Với Python, ba thành phần chính:

- **Ngôn ngữ lập trình:** Python
- **IDE:** môi trường phát triển
- **Thư viện Python:** các package phục vụ ML/Data Science

6.2 Vì sao chọn Python cho Machine Learning?

Lý do chọn Python

Stack Overflow Developer Survey 2023: Python là ngôn ngữ phổ biến thứ ba và phổ biến nhất cho ML/Data Science. Các đặc điểm khiến Python được ưa chuộng:

Các điểm nổi bật

- **Nhiều package mạnh:** NumPy, SciPy, pandas, scikit-learn, ...
- **Prototype nhanh:** thuận lợi thử nghiệm thuật toán mới.
- **Hỗ trợ cộng tác:** (nhu cầu collaboration trong data science).
- **Một ngôn ngữ cho nhiều khâu:** trích xuất dữ liệu, thao tác dữ liệu, phân tích, feature extraction, modeling, evaluation, deployment, cập nhật.

6.3 Điểm mạnh và điểm yếu của Python

Strengths

- Dễ học và dễ hiểu (cú pháp đơn giản).
- Đa mục đích (hỗ trợ structured/OOP/functional).
- Số lượng module lớn (dễ mở rộng).
- Cộng đồng mã nguồn mở lớn (dễ sửa lỗi, thích nghi).
- Có khả năng mở rộng (cấu trúc tốt cho chương trình lớn).

Weakness

Tốc độ thực thi chậm hơn ngôn ngữ biên dịch vì Python là ngôn ngữ thông dịch.

6.4 Cài đặt Python

Hai cách cài

- Cài Python riêng lẻ.
- Dùng bản phân phối đóng gói sẵn như **Anaconda**.

6.4.1 Cài Python riêng lẻ

Unix/Linux (tóm tắt)

- Vào www.python.org/downloads/.
- Tải source code dạng nén cho Unix/Linux.
- Giải nén; có thể chỉnh Modules/Setup nếu cần.
- Chạy: `./configure, make, make install`.

Windows (tóm tắt)

- Vào www.python.org/downloads/.
- Tải Windows installer `python-XYZ.msi`.
- Chạy installer, chấp nhận cấu hình mặc định và hoàn tất.

MacOS (tóm tắt)

- Gợi ý dùng Homebrew.
- Cài Homebrew (lệnh), sau đó `brew update` và `brew install python3`.

6.4.2 Cài bằng Anaconda

Các bước

1. Tải gói cài từ www.anaconda.com/distribution/ theo OS.
2. Chọn phiên bản Python và bản 64-bit/32-bit.
3. Chạy file cài đặt.
4. Kiểm tra: mở command prompt và gõ `python`.

6.5 IDE (Integrated Development Environment)

IDE là gì?

IDE là công cụ phần mềm kết hợp các công cụ phát triển vào một giao diện đồ họa. Một số IDE phổ biến trong ML/Data Science: Jupyter Notebook, PyCharm, VS Code, Spyder, Sublime Text, Atom, Thonny, Google Colab Notebook.

6.5.1 Jupyter Notebook

Vì sao Jupyter hữu ích?

Jupyter notebooks cung cấp môi trường tương tác để phát triển ứng dụng Data Science:

- Trình bày phân tích theo từng bước (code, ảnh, text, output).
- Giúp ghi lại “thought process”.
- Có thể lưu kết quả như một phần của notebook.
- Dễ chia sẻ với người khác.

Cài và chạy

- Nếu dùng Anaconda: thường đã có sẵn.
- Chạy: `jupyter notebook` để mở server (ví dụ localhost:8888).
- Nếu dùng Python chuẩn: cài `pip3 install jupyter`.

Các loại cell trong Jupyter

- **Code cells:** viết code, gửi tới kernel.
- **Markdown cells:** ghi chú text/ảnh/LaTeX/HTML.
- **Raw cells:** hiển thị nguyên văn, không qua cơ chế convert.

6.6 Python libraries và packages

Thư viện giúp làm nhanh và đúng

Hệ sinh thái Python có rất nhiều thư viện giúp xây dựng mô hình ML nhanh hơn: dữ liệu, trực quan hóa, mô hình, deep learning, ...

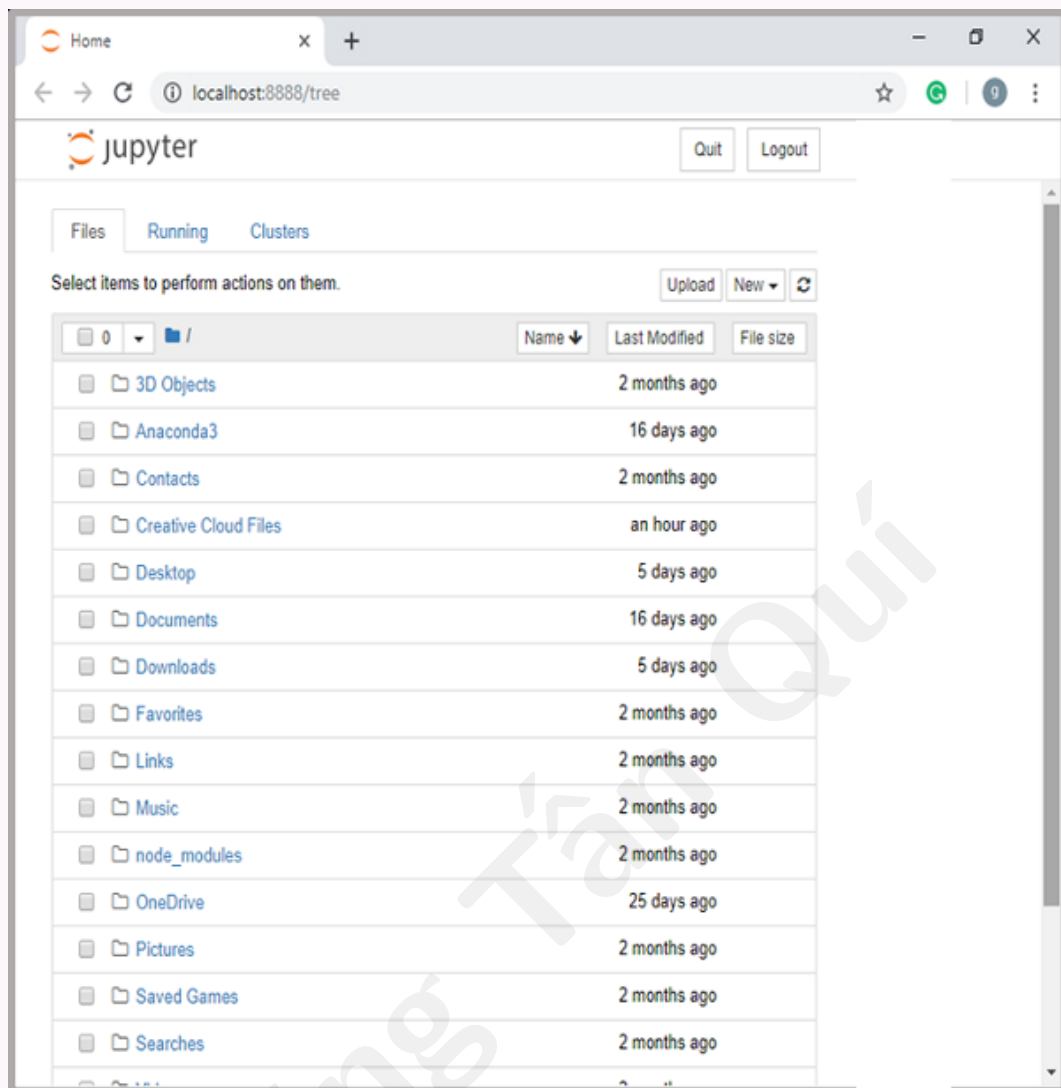
Một số thư viện (kèm lệnh cài)

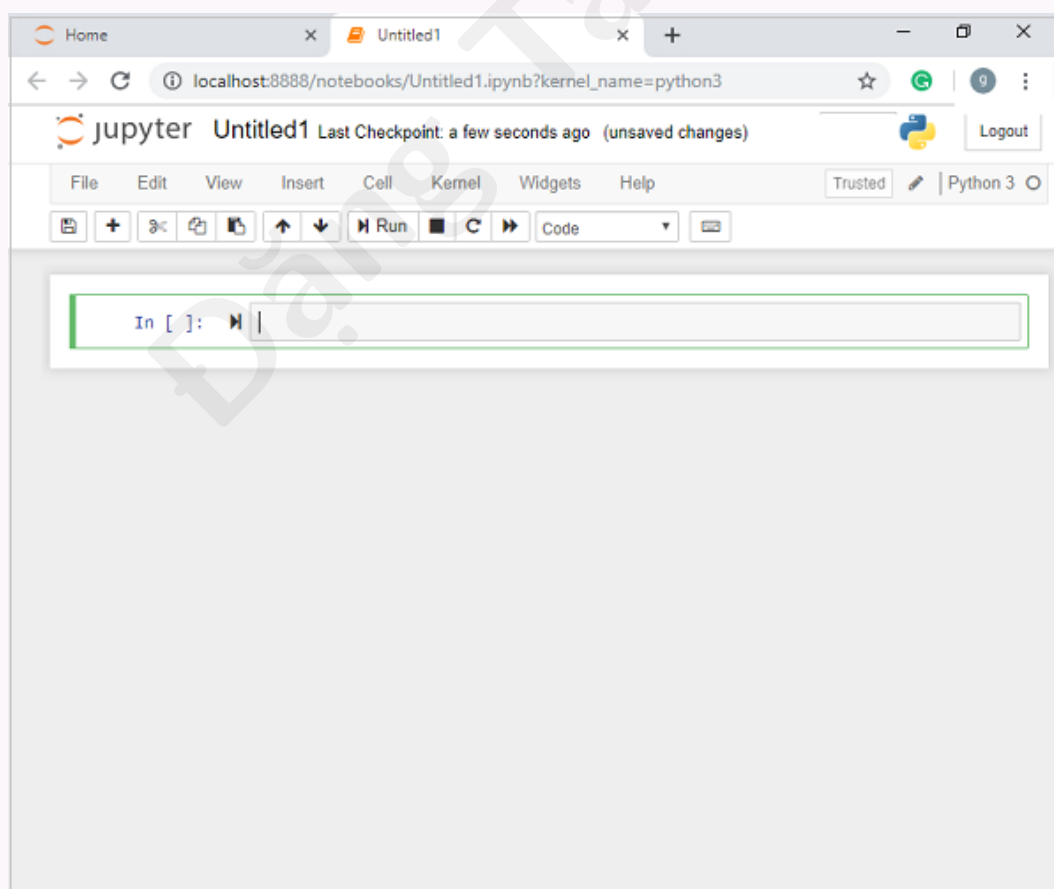
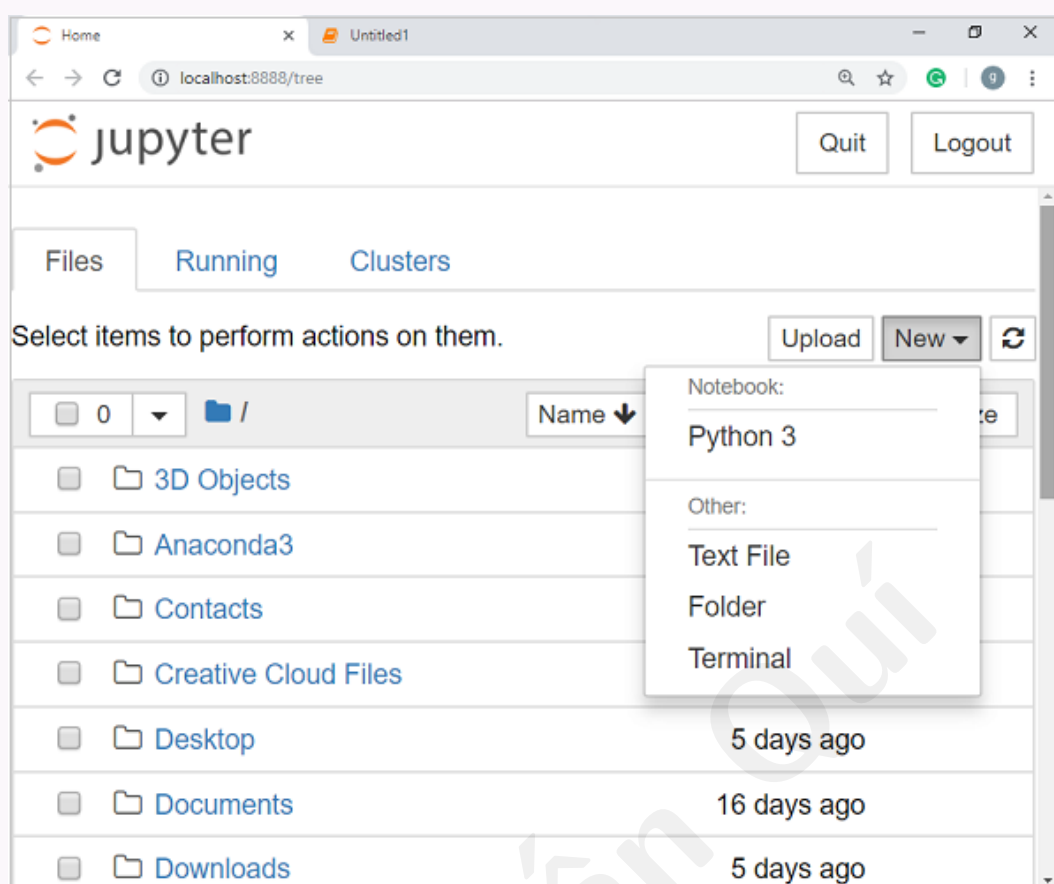
- NumPy: `pip3 install numpy`
- Pandas: `pip3 install pandas`
- Scikit-learn: `pip3 install scikit-learn`
- TensorFlow: `pip install tensorflow`
- PyTorch: `pip install torch`
- Keras: `pip3 install keras`
- OpenCV: `pip3 install opencv-python`

Ghi chú

Ngoài các thư viện trên còn có nhiều tool/framework khác như XGBoost, LightGBM, spaCy, NLTK. Một số thư viện có thể cần thêm dependency hoặc yêu cầu hệ thống; nên xem tài liệu chính thức khi cài.

Hình minh hoạ hệ sinh thái Python





7 Python Libraries — Các thư viện Python cho học máy

Vì sao dùng thư viện?

Thư viện Python là tập các đoạn code và hàm có thể tái sử dụng cho một tác vụ cụ thể. Chúng giúp giảm công sức lập trình khi tác vụ lặp và phức tạp. Học máy là lĩnh vực liên ngành: thuật toán kết hợp lập trình và toán. Thay vì tự cài đặt toàn bộ thuật toán kèm công thức thống kê/toán, ta dùng thư viện để làm nhanh và đúng hơn.

7.1 Danh sách thư viện ML phổ biến

Các thư viện được liệt kê

- NumPy.
- Pandas.
- SciPy.
- Scikit-learn.
- PyTorch.
- TensorFlow.
- Keras.
- Matplotlib.
- Seaborn.
- OpenCV.
- NLTK.
- spaCy.
- XGBoost.
- LightGBM.
- Gensim.

7.2 NumPy

NumPy

NumPy là package xử lý mảng/ma trận (array/matrix) cho tính toán khoa học; thao tác đại số tuyến tính, biến đổi Fourier và các phép toán khác. NumPy cung cấp đối tượng mảng đa chiều hiệu năng cao và công cụ để thao tác ma trận, là thành phần quan trọng của hệ sinh thái ML Python.

Các phép toán quan trọng

- Phép toán toán học và logic trên array.
- Biến đổi Fourier.
- Các phép toán liên quan đại số tuyến tính.

So sánh trực giác

Có thể xem NumPy như “thay thế MATLAB” khi đi cùng SciPy và Matplotlib.

Cài đặt và import

- Với Anaconda: thường đã có sẵn.
- Với Python chuẩn: `pip3 install numpy`.
- Import: `import numpy as np`.

Ví dụ: tạo array 1 chiều

```
import numpy as np
data = np.array([1,2,3,4,5])
print(data)
print(len(data))
print(type(data))
print(data.shape)
```

Output:

```
[1 2 3 4 5]
5
<class 'numpy.ndarray'>
(5,)
```

7.3 Pandas

Pandas

Pandas là thư viện mạnh để thao tác và phân tích dữ liệu. Trong pipeline ML, pandas thường dùng ở bước tiền xử lý/chuẩn bị dữ liệu. Pandas dựa trên hai cấu trúc dữ liệu chính: **Series** (1 chiều) và **DataFrame** (2 chiều).

5 bước xử lý dữ liệu

Load, Prepare, Manipulate, Model, Analyze.

Các cấu trúc dữ liệu trong Pandas

- **Series**: mảng 1 chiều có nhãn trục (homogeneous).
- **DataFrame**: cấu trúc 2 chiều dạng bảng, có thể chứa dữ liệu khác kiểu (heterogeneous).
- **Panel**: cấu trúc 3 chiều (như container của DataFrame) — **đã lỗi thời/deprecated** trong pandas; hiện dùng MultiIndex hoặc xarray.

Ví dụ: Series từ ndarray

```
import pandas as pd
import numpy as np
data = np.array(['g', 'a', 'u', 'r', 'a', 'v'])
s = pd.Series(data)
print(s)
```

Output:

```
0    g
1    a
2    u
3    r
4    a
5    v
dtype: object
```

7.4 SciPy

SciPy

SciPy là thư viện mã nguồn mở cho tính toán khoa học trên dữ liệu lớn. Nó bao gồm các module cho tối ưu và các phép toán như tích phân, đại số tuyến tính, xử lý tín hiệu. SciPy xây trên NumPy nhưng mở rộng để làm các tác vụ phức tạp hơn.

Cài đặt/nhập

- Với Anaconda: thường đã có sẵn.
- Với Python chuẩn: `pip3 install scipy`.
- Ví dụ import: `from scipy import linalg`.

Ví dụ: nghịch đảo ma trận

```
import numpy as np
import scipy
from scipy import linalg
A = np.array([[1, 2], [3, 4]])
print(linalg.inv(A))
```

Output:

```
[[-2.   1. ]  
 [ 1.5 -0.5]]
```

7.5 Scikit-learn

Scikit-learn

Scikit-learn là thư viện ML mã nguồn mở xây trên NumPy và SciPy, hỗ trợ supervised và unsupervised learning. Nó cung cấp công cụ cho tiền xử lý dữ liệu, chọn feature, chọn mô hình, đánh giá mô hình, ...

Đặc điểm nổi bật

- Xây trên NumPy, SciPy và Matplotlib.
- Mã nguồn mở (BSD license).
- Dễ tiếp cận và tái sử dụng.
- Phủ nhiều thuật toán: classification, clustering, regression, giảm chiều, model selection, ...

Cài đặt/nhập

- Với Python chuẩn: `pip3 install scikit-learn`.
- Ví dụ import dataset: `from sklearn.datasets import load_breast_cancer`.

Ví dụ: load Breast Cancer dataset

```
from sklearn.datasets import load_breast_cancer  
data = load_breast_cancer()  
print(data.target[[10, 50, 85]])  
print(list(data.target_names))
```

Output:

```
[0 1 0]  
['malignant', 'benign']
```

7.6 PyTorch

PyTorch

PyTorch là thư viện deep learning mã nguồn mở dựa trên Torch, thường dùng phát triển mạng nơ-ron sâu. Nó dựa trên Python trực quan và có thể định nghĩa computational graph động; phù hợp cho nghiên cứu và phát triển cần linh hoạt.

Cài đặt/nhập

- Cài (ví dụ CPU Windows, Python 3.8+): `pip3 install torch torchvision torchaudio`.
- Link hướng dẫn cài: <https://pytorch.org/get-started/locally/>.
- Import: `import torch`.

Ví dụ: NumPy array → torch tensor

```
import numpy as np
import torch
x = np.ones([3,4])
y = torch.from_numpy(x)
print(y)
```

Output:

```
tensor([[1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.]], dtype=torch.float64)
```

7.7 TensorFlow

TensorFlow

TensorFlow là thư viện mã nguồn mở của Google cho ML/DL, hỗ trợ xây computational graph và chạy hiệu quả trên nhiều phần cứng. Thường dùng cho NLP, nhận dạng ảnh, nhận dạng chữ viết tay.

Cài đặt/nhập

- Cài: `pip3 install tensorflow`.
- Link hướng dẫn: <https://www.tensorflow.org/install/pip>.
- Import: `import tensorflow as tf`.

Ví dụ: tạo tensor

```
import tensorflow as tf
data = tf.constant([[2,1],[4,6]])
```

```
print(data)
```

Output:

```
tf.Tensor(  
[[2 1]  
 [4 6]], shape=(2, 2), dtype=int32)
```

7.8 Keras

Keras

Keras là thư viện mạng nơ-ron mức cao để tạo mô hình deep learning. Nó chạy trên TensorFlow/CNTK/Theano và có API trực quan, phù hợp người mới và nghiên cứu vì dễ prototype nhanh.

Cài đặt/nhập

- Cài: `pip3 install keras`.
- Import: `import keras`.

Ví dụ: dùng CIFAR-10 dataset

```
import keras  
(x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()  
print(x_train.shape)  
print(x_test.shape)  
print(y_train.shape)  
print(y_test.shape)
```

Output:

```
(50000, 32, 32, 3)  
(10000, 32, 32, 3)  
(50000, 1)  
(10000, 1)
```

7.9 Matplotlib

Matplotlib

Matplotlib là thư viện vẽ phổ biến để trực quan hóa dữ liệu: graph, plot, histogram, bar chart, ... Nó hỗ trợ phân tích, khám phá và trình bày dữ liệu.

Cài đặt/nhập

- Cài: `pip3 install matplotlib`.
- Import: `import matplotlib.pyplot as plt`.

Ví dụ: vẽ đường thẳng

```
import matplotlib.pyplot as plt
plt.plot([1,2,3],[1,2,3])
plt.show()
```

7.10 Seaborn

Seaborn

Seaborn là thư viện mã nguồn mở xây trên Matplotlib và tích hợp tốt với pandas. Nó giúp tạo biểu đồ thống kê đẹp và nhiều thông tin, phù hợp phân tích business/marketing.

Cài đặt/nhập

- Cài: `pip3 install seaborn`.
- Import: `import seaborn as sns`.

7.11 OpenCV

OpenCV

OpenCV (Open Source Computer Vision Library) là thư viện Python cho thị giác máy tính và xử lý ảnh. Thư viện này giúp nhận diện pattern trong ảnh, trích xuất feature, và có thể tích hợp với NumPy để xử lý cấu trúc array.

7.12 NLTK

NLTK

NLTK (Natural Language ToolKit) là môi trường Python thường dùng phát triển NLP. Nó có các interface như WordNet và thư viện cho classification, tokenization, parsing, semantic reasoning.

7.13 spaCy

spaCy

spaCy là thư viện Python mã nguồn mở, hỗ trợ tác vụ NLP nâng cao theo cách nhanh và hiệu quả. Tokenization và POS tagging là hai tác vụ thư viện làm tốt.

Các tool/framework khác

XGBoost, LightGBM, Gensim là các công cụ khác trong Python cho ML. Việc học hệ sinh thái thư viện giúp bạn hiểu cách build/train/deploy mô hình.

8 Applications — Ứng dụng của học máy

Tổng quan

Học máy đã trở thành công nghệ phổ biến và ảnh hưởng đến nhiều khía cạnh cuộc sống: kinh doanh, y tế, giải trí, ... Học máy giúp ra quyết định và tìm các phương án giải bài toán, từ đó nâng hiệu quả công việc ở nhiều lĩnh vực.

Một số ứng dụng thành công

Chatbot, dịch ngôn ngữ, nhận diện khuôn mặt, hệ gợi ý, xe tự hành, phát hiện đối tượng, phân tích ảnh y tế, ...

8.1 Danh mục các lĩnh vực ứng dụng

Các lĩnh vực

- Image and Speech Recognition
- Natural Language Processing
- Finance Sector
- E-commerce and Retail
- Automotive Sector
- Computer Vision
- Manufacturing and Industries
- Healthcare Sector

8.2 Image and Speech Recognition

Nhận dạng ảnh và giọng nói

Nhận dạng ảnh và giọng nói là hai mảng mà học máy đã cải thiện đáng kể. Thuật toán ML được dùng trong các ứng dụng như:

- Nhận diện khuôn mặt,
- Phát hiện đối tượng (object detection),
- Nhận dạng giọng nói (speech recognition),

Nhằm nhận diện và phân loại ảnh/giọng nói một cách chính xác.

8.3 Natural Language Processing (NLP)

NLP là gì?

Xử lý ngôn ngữ tự nhiên (NLP) là lĩnh vực của khoa học máy tính nghiên cứu tương tác giữa máy và con người bằng ngôn ngữ tự nhiên. NLP dùng thuật toán học máy để nhận diện từ loại, cảm xúc và nhiều khía cạnh khác của văn bản; nó phân tích, hiểu và sinh ngôn ngữ người.

NLP xuất hiện ở đâu?

NLP hiện diện trong nhiều ứng dụng trên Internet như phần mềm dịch, công cụ tìm kiếm, chatbot, phần mềm sửa ngữ pháp, trợ lý giọng nói, ...

Một số ứng dụng NLP

- Phân tích cảm xúc (sentiment analysis)
- Tổng hợp giọng nói (speech synthesis)
- Nhận dạng giọng nói (speech recognition)
- Phân loại văn bản (text classification)
- Chatbots
- Dịch ngôn ngữ (language translation)
- Tạo caption (caption generation)
- Tóm tắt văn bản (document summarization)
- Trả lời câu hỏi (question answering)
- Tự động hoàn thành trong công cụ tìm kiếm (autocomplete in search engines)

8.4 Finance Sector

ML trong tài chính

Vai trò của học máy trong tài chính là duy trì giao dịch an toàn. Trong giao dịch, dữ liệu được chuyển thành thông tin phục vụ ra quyết định.

8.4.1 Fraud Detection

Phát hiện gian lận

Học máy được dùng rộng rãi để phát hiện gian lận. Phát hiện gian lận là quá trình dùng mô hình ML để theo dõi giao dịch và hiểu pattern trong dataset nhằm nhận diện hoạt động gian lận/đáng ngờ.

Lợi ích

Thuật toán ML có thể phân tích lượng dữ liệu giao dịch rất lớn để phát hiện pattern và anomaly, giúp ngăn thất thoát tài chính và bảo vệ khách hàng.

8.4.2 Algorithmic Trading

Giao dịch thuật toán

Thuật toán học máy được dùng để tìm các pattern phức tạp trong dataset lớn nhằm phát hiện tín hiệu giao dịch mà con người khó nhận ra.

Các ứng dụng tài chính khác

- Phân tích và dự báo thị trường chứng khoán,
- Đánh giá và quản trị rủi ro tín dụng,
- Phân tích an ninh và tối ưu danh mục,
- Định giá và quản trị tài sản.

8.5 E-commerce and Retail

ML trong thương mại điện tử/bán lẻ

Học máy giúp cải thiện kinh doanh qua hệ gợi ý và quảng cáo nhắm mục tiêu, tăng trải nghiệm người dùng. Học máy cũng giúp marketing dễ hơn bằng cách tự động hoá tác vụ lặp.

8.5.1 Recommendation Systems

Hệ gợi ý

Hệ gợi ý cung cấp đề xuất cá nhân hoá dựa trên hành vi, sở thích và tương tác trước đó của người dùng. Thuật toán ML phân tích dữ liệu người dùng và tạo đề xuất cho sản phẩm, dịch vụ và nội dung.

8.5.2 Demand Forecasting

Dự báo nhu cầu

Công ty dùng ML để hiểu nhu cầu tương lai của sản phẩm/dịch vụ dựa trên xu hướng thị trường, hành vi khách hàng và dữ liệu bán hàng lịch sử.

8.5.3 Customer Segmentation

Phân khúc khách hàng

ML có thể phân nhóm khách hàng thành các nhóm có đặc điểm tương tự. Mục đích là hiểu hành vi và nhắm mục tiêu trải nghiệm cá nhân hoá.

8.6 Automotive Sector

Xe tự hành

Một đổi mới lớn là xe tự hành (driverless vehicles) có thể cảm nhận môi trường, tự lái vượt chướng ngại mà không cần hỗ trợ người. Xe tự hành dùng ML để phân tích liên tục xung quanh và dự đoán các khả năng xảy ra.

8.7 Computer Vision

Computer vision trong ML

Computer vision là ứng dụng của học máy dùng thuật toán và mạng nơ-ron để dạy máy rút trích thông tin có ý nghĩa từ ảnh/video số. Computer vision được dùng trong nhận diện khuôn mặt, chẩn đoán bệnh từ MRI, và xe tự hành.

Một số tác vụ

- Phát hiện và nhận dạng đối tượng (object detection and recognition)
- Phân loại và nhận dạng ảnh (image classification and recognition)
- Nhận dạng khuôn mặt (facial recognition)
- Xe tự hành (autonomous vehicles)

- Phân tách đối tượng (object segmentation)
- Xây dựng lại ảnh (image reconstruction)

8.8 Manufacturing and Industries

Bảo trì dự đoán (predictive maintenance)

ML được dùng để theo dõi điều kiện vận hành của máy móc. Predictive maintenance giúp phát hiện lỗi/khuyết tật của thiết bị đang hoạt động để tránh dừng máy bất ngờ. Phát hiện bất thường cũng giúp lập kế hoạch bảo trì định kỳ.

Predictive maintenance là gì?

Predictive maintenance là quá trình dùng thuật toán ML để dự đoán khi nào cần bảo trì một máy (ví dụ thiết bị trong nhà máy). Phân tích dữ liệu từ cảm biến và nguồn khác giúp phát hiện pattern báo hiệu khả năng hỏng, để bảo trì trước khi máy “gãy”.

8.9 Healthcare Sector

ML trong y tế

ML có nhiều ứng dụng trong y tế. Ví dụ: phân tích ảnh y khoa để phát hiện bệnh (như ung thư), hoặc dự đoán kết quả điều trị dựa trên tiền sử bệnh và yếu tố khác.

Một số ứng dụng

1. **Phân tích ảnh y khoa và chẩn đoán (Medical Imaging and Diagnostics):** phân tích pattern trong ảnh để chỉ ra sự hiện diện của bệnh.
2. **Phát hiện thuốc (Drug Discovery):** phân tích dataset lớn để dự đoán hoạt tính sinh học và tìm thuốc tiềm năng bằng phân tích cấu trúc hoá học.
3. **Chẩn đoán bệnh (Disease Diagnosis):** nhận diện một số loại bệnh (ví dụ: ung thư vú, suy tim, Alzheimer, viêm phổi) bằng thuật toán ML.

Kết luận

Đây chỉ là vài ví dụ. Khi ML tiếp tục phát triển, ta kỳ vọng ML được dùng ở nhiều lĩnh vực hơn, nâng hiệu quả, độ chính xác và sự tiện lợi.

9 Life Cycle — Vòng đời dự án học máy

Vòng đời ML là quy trình lặp

Vòng đời học máy là một quy trình lặp để xây dựng dự án ML end-to-end. Việc xây mô hình là quá trình liên tục, nhất là khi dữ liệu tăng theo thời gian. ML tập trung cải thiện hiệu năng hệ thống bằng cách huấn luyện với dữ liệu thực tế. Muốn dự án ML thành công, cần theo các bước/pha được định nghĩa rõ ràng.

9.1 Machine Learning Life Cycle là gì?

Định nghĩa

Vòng đời ML là quy trình lặp đi từ một bài toán kinh doanh (business problem) đến một lời giải ML. Nó là “kim chỉ nam” để phát triển dự án ML và cung cấp hướng dẫn/best practices ở từng pha khi xây solution.

Đặc trưng của vòng đời

Quy trình gồm nhiều pha từ xác định bài toán đến triển khai và giám sát. Trong khi phát triển dự án, ta thường quay lại các bước trước nhiều lần.

9.2 Các pha chính trong vòng đời

Danh sách pha

- Xác định bài toán (Problem Definition)
- Chuẩn bị dữ liệu (Data Preparation)
- Xây mô hình (Model Development)
- Triển khai mô hình (Model Deployment)
- Giám sát và bảo trì (Monitoring and Maintenance)

9.3 Problem Definition

Bước đầu tiên: xác định bài toán

Bước đầu tiên là xác định bài toán cần giải. Đây là bước quan trọng giúp bắt đầu xây một solution ML. Việc xác định bài toán giúp hiểu: đầu ra mong muốn là gì, phạm vi nhiệm vụ, và mục tiêu.

Yêu cầu của problem definition

Vì đặt nền móng cho mô hình ML, mô tả bài toán phải **rõ ràng** và **ngắn gọn**. Bước này gồm: hiểu bài toán kinh doanh, định nghĩa problem statement, và xác định success criteria cho mô hình.

9.4 Data Preparation

Data preparation là gì?

Chuẩn bị dữ liệu là quá trình “làm dữ liệu sẵn sàng cho phân tích” bằng: **data exploration**, **feature engineering** và **feature selection**. Trong đó:

- Phân tích dữ liệu (Data exploration): trực quan hóa và hiểu dữ liệu.
- Tạo feature mới (Feature engineering): tạo feature mới từ feature sẵn có.
- Chọn feature (Feature selection): chọn feature liên quan nhất để train mô hình.

Các bước trong data preparation

Data preparation thường gồm: thu thập dữ liệu, tiền xử lý, feature engineering & selection; và thường bao gồm cả EDA.

9.4.1 Data Collection

Thu thập dữ liệu

Sau khi phân tích problem statement, bước tiếp theo là thu thập dữ liệu từ nhiều nguồn làm “nguyên liệu thô” cho mô hình. Một số tiêu chí khi thu thập:

- **Liên quan và hữu ích (Relevant and usefulness)**: dữ liệu liên quan problem statement và hữu ích để train hiệu quả.
- **Chất lượng và số lượng (Quality and Quantity)**: chất lượng và số lượng ảnh hưởng trực tiếp đến hiệu năng.
- **Đa dạng (Variety)**: dữ liệu đa dạng để mô hình học nhiều kịch bản và nhận ra pattern.

Ví dụ nơi thu thập: khảo sát, database sẵn có, Kaggle. Dữ liệu có thể là primary (thu riêng cho bài toán) hoặc secondary (dữ liệu đã tồn tại).

9.4.2 Data Preprocessing

Tiền xử lý dữ liệu

Dữ liệu thu thập thường không có cấu trúc và “lộn xộn”, ảnh hưởng xấu đến kết quả. Vì vậy cần tiền xử lý để tăng độ chính xác và hiệu năng. Các vấn đề cần xử lý: missing values, duplicate data, invalid data và noise.

Tiền xử lý dữ liệu (Data wrangling)

Bước này là data preprocessing hoặc data wrangling, nhằm làm dữ liệu “dễ tiêu thụ” và hữu ích cho phân tích.

9.4.3 Phân tích dữ liệu (Analyzing Data)

Phân tích dữ liệu

Sau khi dữ liệu được sắp xếp, cần hiểu dữ liệu: trực quan hóa và tóm tắt thống kê để lấy insight. Các công cụ như Power BI, Tableau giúp trực quan hóa để hiểu pattern/trend, từ đó hỗ trợ quyết định trong feature engineering và model selection.

9.4.4 Tạo feature mới và chọn feature (Feature Engineering and Selection)

Feature Engineering

Feature là đại lượng đo được (measurable quantity) được quan sát khi huấn luyện. Feature engineering là quá trình tạo feature mới hoặc cải thiện feature hiện có để mô hình hiểu pattern/trend chính xác hơn.

Feature Selection

Feature selection là chọn các feature ổn định và liên quan nhất với problem statement. Feature engineering/selection giúp giảm kích thước dữ liệu, quan trọng khi dữ liệu tăng trưởng.

9.5 Model Development

Xây mô hình từ dữ liệu đã chuẩn bị

Trong bước model development, ta xây mô hình ML từ dữ liệu đã chuẩn bị. Quy trình gồm: chọn thuật toán phù hợp, huấn luyện, điều chỉnh siêu tham số, và đánh giá bằng cross-validation.

Ba bước chính

1. **Model Selection:** chọn mô hình theo đặc tính dữ liệu, độ phức tạp bài toán, đầu ra mong muốn, và mức phù hợp với problem definition.
2. **Model Training:** cho thuật toán học pattern/mối quan hệ trong feature từ dataset đã tiền xử lý.
3. **Model Evaluation:** đánh giá bằng metric (accuracy/precision/recall/F1); nếu chưa đạt thì tuning siêu tham số và lặp.

Lặp lại nếu chưa đạt

Nếu mô hình vẫn không đạt, quay lại bước chọn mô hình rồi train/evaluate tiếp để cải thiện.

9.6 Model Deployment

Triển khai mô hình

Triển khai là đưa mô hình vào production: tích hợp với hệ thống để phục vụ người dùng/quản trị/mục đích khác, và kiểm thử trong kịch bản thực. Hai yếu tố cần kiểm tra:

- **Portable:** khả năng chuyển phần mềm từ máy này sang máy khác.
- **Scalable:** mở rộng mà không phải thiết kế lại để giữ hiệu năng.

9.7 Monitoring and Maintenance

Giám sát để biết mô hình đang “xuống cấp”

Giám sát là đo metric và phát hiện vấn đề của mô hình. Khi phát hiện vấn đề, mô hình có thể cần train lại bằng dữ liệu mới hoặc thay đổi kiến trúc.

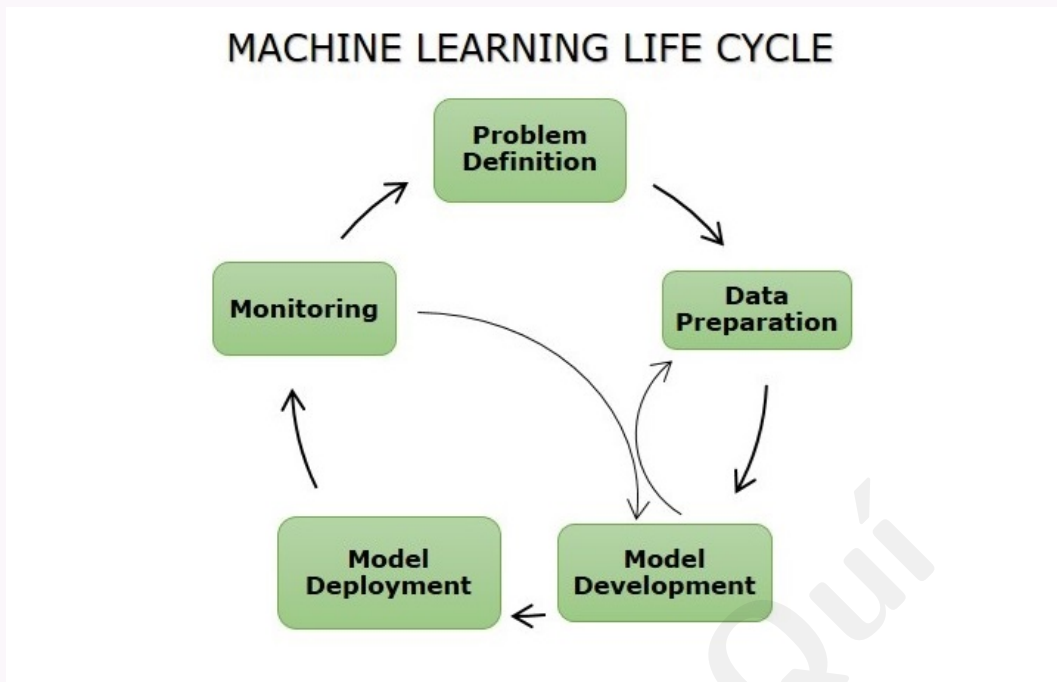
Khi vấn đề trở thành bài toán mới

Đôi khi vấn đề không giải được chỉ bằng retrain, và chính vấn đề đó trở thành problem statement mới. Khi đó vòng đời được “làm lại” từ khâu phân tích bài toán để phát triển mô hình cải tiến.

Kết luận

Vòng đời ML là quy trình lặp; có thể cần quay lại bước trước để cải thiện hiệu năng hoặc đáp ứng yêu cầu mới. Theo vòng đời giúp đảm bảo mô hình hiệu quả, chính xác và đáp ứng yêu cầu kinh doanh.

Hình minh hoạ vòng đời dự án ML



10 Required Skills — Các kỹ năng cần có cho học máy

Tổng quan

Học máy là lĩnh vực phát triển nhanh và để thành công thường cần kết hợp **kỹ năng kỹ thuật** và **kỹ năng mềm**. Khuyến nghị: nếu muốn theo học máy như một hướng nghề nghiệp, hãy học đủ các kỹ năng giúp tăng năng lực tổng thể.

10.1 Danh sách kỹ năng chính

Key skills

- Kỹ năng lập trình (Programming Skills)
- Thống kê và Toán (Statistics and Mathematics)
- Cấu trúc dữ liệu (Data Structures)
- Tiền xử lý dữ liệu (Data Preprocessing)
- Trực quan hoá dữ liệu (Data Visualization)
- Thuật toán học máy (Machine Learning Algorithms)
- Mạng nơ-ron và Học sâu (Neural Networks & Deep Learning)
- Xử lý ngôn ngữ tự nhiên (Natural Language Processing)
- Kỹ năng giải quyết vấn đề (Problem-solving Skills)

- Kỹ năng giao tiếp (Communication Skills)
- Hiểu kinh doanh (Business Acumen)

10.2 Programming Skills

Vì sao cần lập trình?

Học máy cần nền tảng lập trình vững, đặc biệt ở các ngôn ngữ như Python, R và Java. Thành thạo lập trình giúp bạn xây, kiểm thử và triển khai mô hình ML.

Python và R

- Python rất phổ biến vì có nhiều thư viện như NumPy, Matplotlib, scikit-learn, Seaborn, Keras, TensorFlow, ... giúp đơn giản hoá quy trình.
- Các nội dung Python cơ bản: basic data types; dictionaries/lists/sets; loops/conditionals; functions; list comprehensions.
- R cũng phổ biến trong ML; có thể làm các tác vụ ML nặng “dễ hơn”; ngoài ngôn ngữ, cần nắm các package đi kèm.

10.3 Statistics and Mathematics

Toán–thống kê là nền để hiểu mô hình

Hiểu tốt toán và thống kê giúp bạn hiểu/áp dụng mô hình thống kê, thuật toán và phương pháp để phân tích và diễn giải dữ liệu.

10.3.1 Statistics

Thống kê dùng để làm gì?

Thống kê giúp suy luận từ dữ liệu và rút kết luận. Công thức thống kê giúp diễn giải dữ liệu để ra quyết định dựa trên dữ liệu. Chia thống kê thành:

- **Thống kê mô tả (Descriptive statistics):** đơn giản hoá/tổ chức dữ liệu bằng mean, range, variance, standard deviation.
- **Thống kê suy luận (Inferential statistics):** dùng mẫu nhỏ để kết luận về tập lớn, qua hypothesis testing, null/alternative testing, ...

10.3.2 Mathematics

Các mảng toán nên biết

- **Đại số (Algebra)**: biến, hằng, hàm; phương trình tuyến tính; logarit.
- **Đại số tuyến tính (Linear algebra)**: vectơ, ma trận, trị riêng (eigenvalues).
- **Giải tích (Calculus)**: đạo hàm, tích phân, gradient descent.

Toán liên quan ML như thế nào?

Ví dụ: công thức hồi quy tuyến tính $y = ax + b$ là biểu thức tuyến tính trong đại số.

10.3.3 Mathematical Notation

Đọc được ký hiệu quan trọng hơn “làm tay phức tạp”

Mức toán cần biết có thể chỉ ở mức beginner, nhưng điều quan trọng là bạn đọc và hiểu ký hiệu toán trong phương trình. Nếu đọc và hiểu được ký hiệu, bạn sẵn sàng học ML; nếu không, nên ôn lại toán.

10.4 Probability Theory

Xác suất là tiền đề quan trọng

Vì ML liên quan dự đoán, xác suất là nền tảng quan trọng. Gợi ý nắm: biến ngẫu nhiên, mật độ/phân phối xác suất, ...

Ví dụ kiểm tra kiến thức: phân loại theo xác suất có điều kiện

$$p(c_i|x, y) = \frac{p(x, y|c_i) p(c_i)}{p(x, y)}.$$

Quy tắc phân loại Bayes:

- Nếu $P(c_1|x, y) > P(c_2|x, y)$ thì lớp là c_1 .
- Nếu $P(c_1|x, y) < P(c_2|x, y)$ thì lớp là c_2 .

10.5 Optimization Problem

Tối ưu hoá trong ML

Ví dụ một bài toán tối ưu (mang tính kiểm tra khả năng đọc ký hiệu). Trong ML, tối ưu hoá thường xuất hiện khi huấn luyện mô hình (tối thiểu hoá loss / tối đa hoá mục tiêu).

10.6 Data Structures

Vì sao cần cấu trúc dữ liệu?

Hiểu cấu trúc dữ liệu giúp giải bài toán thực và xây sản phẩm phần mềm. Trong ML, cấu trúc dữ liệu giúp xử lý và hiểu bài toán phức tạp. Một số cấu trúc: arrays, stack, queue, binary trees, maps, ...

10.7 Data Preprocessing

Tiền xử lý dữ liệu

Chuẩn bị dữ liệu cho ML cần kiến thức về làm sạch, biến đổi và chuẩn hoá dữ liệu. Điều này gồm: phát hiện và sửa lỗi, missing values, và bất nhất trong dữ liệu.

10.8 Data Visualization

Trực quan hoá dữ liệu

Trực quan hoá là tạo biểu diễn đồ hoạ để hiểu và diễn giải dataset phức tạp. Data scientist cần tạo visualizations hiệu quả để truyền đạt insight. Công cụ: Tableau, Power BI, ...

Hiểu các loại biểu đồ

Nhiều trường hợp bạn cần hiểu các loại biểu đồ để hiểu phân phối dữ liệu và diễn giải output của thuật toán.

10.9 Machine Learning Algorithms

Kiến thức thuật toán

Liệt kê: regression, decision trees, random forests, k-NN, SVM, neural networks. Hiểu điểm mạnh/yếu của thuật toán quan trọng để xây mô hình hiệu quả và biết khi nào áp dụng.

10.10 Neural Networks & Deep Learning

NN và DL

Neural networks được thiết kế để máy hoạt động “tương tự não người”, gồm các nút/neurons liên kết để học từ dữ liệu. Deep learning là nhánh của ML huấn luyện mạng sâu để phân tích dataset phức tạp; cần hiểu CNN, RNN và các chủ đề liên quan.

10.11 Natural Language Processing

NLP

NLP là nhánh AI tập trung vào tương tác giữa máy và người bằng ngôn ngữ tự nhiên. Kỹ thuật như sentiment analysis, text classification, named entity recognition.

10.12 Problem-solving Skills

Kỹ năng giải quyết vấn đề

Cần khả năng xác định vấn đề, tạo giả thuyết và phát triển lời giải; cần tư duy sáng tạo và logic cho bài toán phức tạp.

10.13 Communication Skills

Kỹ năng giao tiếp

Data scientist cần giải thích khái niệm kỹ thuật cho stakeholder không kỹ thuật, truyền đạt kết quả phân tích và ý nghĩa một cách rõ ràng, súc tích.

10.14 Business Acumen

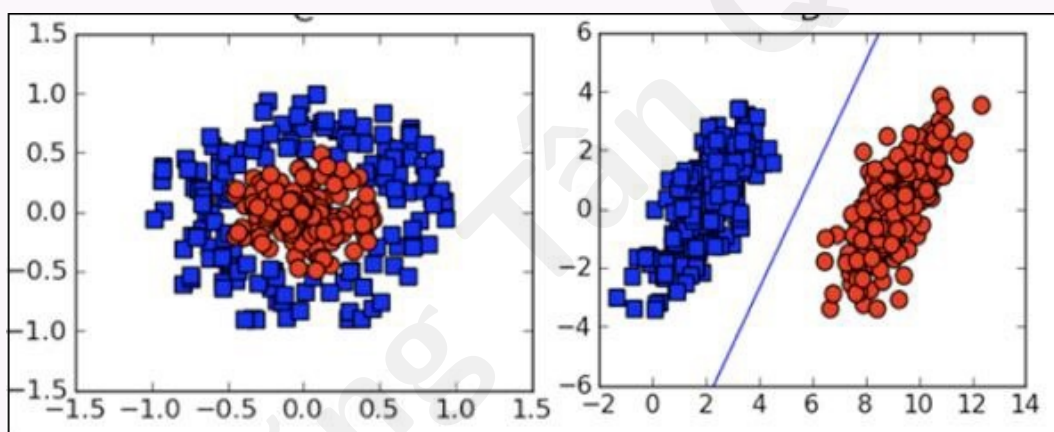
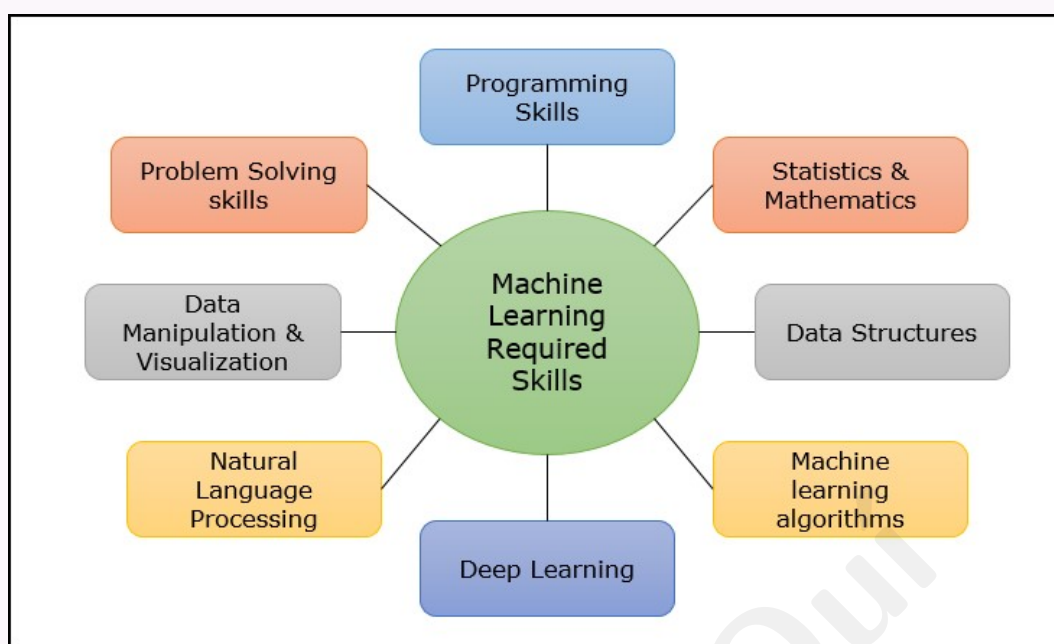
Hiểu bối cảnh kinh doanh

Vì ML thường dùng để giải bài toán kinh doanh, hiểu bối cảnh và biết cách áp dụng ML vào vấn đề kinh doanh là quan trọng.

Kết luận

Học máy cần nhiều kỹ năng (kỹ thuật, toán, kỹ năng mềm) và cần kết hợp chúng để xây dựng và triển khai mô hình hiệu quả.

Hình minh hoạ kỹ năng nền tảng



11 Implementation — Triển khai học máy trong thực tế

Mục tiêu

Bài này mô tả các bước triển khai học máy (implementation) trong thực tế: từ dữ liệu, trực quan hoá, feature engineering đến chọn mô hình, đánh giá, tuning và triển khai/giám sát.

11.1 Các bước triển khai học máy

11.1.1 Data Collection and Preparation

Thu thập và chuẩn bị dữ liệu

Bước đầu tiên là thu thập dữ liệu để huấn luyện và kiểm thử mô hình. Dữ liệu cần **liên quan** bài toán. Sau khi thu thập, cần tiền xử lý và làm sạch để loại bỏ dữ liệu bị thiếu, nhiễu, trùng lặp, ...

11.1.2 Data Exploration and Visualization

Khám phá và trực quan hoá dữ liệu

Bước tiếp theo là khám phá và trực quan hoá để hiểu cấu trúc và nhận diện pattern/trend. Các công cụ như Matplotlib và Seaborn để vẽ histogram, scatter plots, heatmaps.

11.1.3 Feature Selection and Engineering

Chọn feature và tạo feature

Chọn các feature liên quan bài toán hoặc tạo feature mới từ feature sẵn có để tăng độ chính xác mô hình.

11.1.4 Model Selection and Training

Chọn mô hình và huấn luyện

Sau khi dữ liệu chuẩn bị và feature được chọn/tạo, bước tiếp theo là chọn thuật toán phù hợp để huấn luyện. Chia train/test và fit mô hình trên train. Có thể dùng nhiều thuật toán như linear regression, logistic regression, decision trees, random forests, SVM, neural networks, ...

11.1.5 Model Evaluation

Đánh giá mô hình

Sau khi huấn luyện, cần đánh giá hiệu năng. Metric như accuracy, precision, recall, F1-score; và có thể dùng cross-validation để kiểm thử khả năng tổng quát.

11.1.6 Model Tuning

Điều chỉnh siêu tham số

Có thể cải thiện hiệu năng bằng điều chỉnh siêu tham số (các tham số không học từ dữ liệu mà do người dùng đặt). Tìm bộ tối ưu bằng grid search và random search.

11.1.7 Deployment and Monitoring

Triển khai và giám sát

Sau khi huấn luyện và tuning, triển khai mô hình vào production. Triển khai gồm tích hợp mô hình vào quy trình/hệ thống. Sau đó cần giám sát thường xuyên để đảm bảo mô hình tiếp tục hoạt động tốt và phát hiện vấn đề cần xử lý.

Kết luận

Mỗi bước cần công cụ/kỹ thuật khác nhau. Triển khai thành công cần kết hợp kỹ năng kỹ thuật và hiểu bối cảnh kinh doanh.

11.2 Chọn ngôn ngữ và IDE để phát triển ML

Chọn công cụ theo năng lực và bối cảnh

Để phát triển ứng dụng ML, bạn cần quyết định nền tảng, IDE và ngôn ngữ. Nhiều lựa chọn có thể đáp ứng nhu cầu vì đều có hiện thực các thuật toán AI/ML phổ biến.

Nếu tự phát triển thuật toán

Gợi ý cân nhắc:

- Ngôn ngữ bạn thành thạo.
- IDE bạn quen dùng.
- Nền tảng triển khai (nhiều nền tảng miễn phí; một số có phí license theo mức sử dụng).

11.2.1 Language Choice

Ngôn ngữ hỗ trợ ML

Python, R, Matlab, Octave, Julia, C++, C...

11.2.2 IDEs

IDE hỗ trợ ML

R Studio, PyCharm, iPython/Jupyter Notebook, Julia, Spyder, Anaconda, Rodeo, Google Colab... Lưu ý: mỗi IDE có ưu/nhược, nên thử trước khi chọn một IDE chính.

12 Challenges & Common Issues — Thách thức và vấn đề thường gặp

Tổng quan

Học máy phát triển nhanh và có nhiều ứng dụng hứa hẹn. Tuy nhiên, để khai thác hết tiềm năng, có một số thách thức và vấn đề cần được giải quyết.

12.1 Overfitting

Overfitting

Overfitting xảy ra khi mô hình được huấn luyện trên một tập dữ liệu hạn chế và trở nên quá phức tạp, dẫn đến hiệu năng kém khi kiểm thử trên dữ liệu mới.

Cách xử lý

- Kiểm tra chéo (Cross-validation)
- Điều chỉnh (Regularization)
- Dừng sớm (Early stopping)

12.2 Underfitting

Underfitting

Underfitting xảy ra khi mô hình quá đơn giản và không thể nắm bắt pattern trong dữ liệu.

Cách xử lý

- Dùng mô hình phức tạp hơn
- Thêm nhiều feature cho dữ liệu

12.3 Data Quality Issues

Chất lượng dữ liệu quyết định chất lượng mô hình

Mô hình ML “tốt đến đâu” phụ thuộc dữ liệu huấn luyện. Dữ liệu chất lượng kém có thể tạo ra mô hình không chính xác.

Một số vấn đề chất lượng dữ liệu

- Giá trị thiếu (Missing values)
- Giá trị sai (Incorrect values)
- Giá trị ngoại lai (Outliers)

12.4 Imbalanced Datasets

Imbalanced datasets

Mất cân bằng dữ liệu xảy ra khi một lớp phổ biến hơn đáng kể so với lớp khác. Điều này có thể tạo mô hình bị lệch: rất đúng với lớp đa số nhưng kém với lớp thiểu số.

12.5 Model Interpretability

Mô hình khó giải thích

Mô hình ML có thể rất phức tạp, khiến khó hiểu mô hình đi đến dự đoán thế nào. Điều này gây khó khăn khi giải thích cho stakeholder hoặc cơ quan quản lý.

Một số kỹ thuật hỗ trợ

- Quan trọng của feature (Feature importance)
- Biểu đồ phụ thuộc riêng phần (Partial dependence plots)

12.6 Generalization

Generalization

Mô hình ML được huấn luyện trên một dataset cụ thể, và có thể hoạt động kém trên dữ liệu mới nằm ngoài phân phối của tập huấn luyện.

Cách xử lý

Cross-validation và regularization.

12.7 Scalability

Scalability

Mô hình ML có thể tốn kém tính toán và khó mở rộng cho dataset lớn.

Một số cách xử lý

- Tính toán phân tán (Distributed computing)
- Xử lý song song (Parallel processing)
- Lấy mẫu (Sampling)

12.8 Ethical Considerations

Vấn đề đạo đức

Mô hình ML có thể gây lo ngại đạo đức khi được dùng để ra quyết định ảnh hưởng đến con người. Bias, privacy và transparency.

Một số kỹ thuật hỗ trợ

- Chỉ số công bằng (Fairness metrics)
- AI giải thích được (Explainable AI)

Kết luận

Giải quyết các vấn đề trên cần kết hợp chuyên môn kỹ thuật và hiểu biết kinh doanh, đồng thời hiểu các cân nhắc đạo đức. Khi xử lý tốt, ML có thể tạo mô hình chính xác và đáng tin cậy, cung cấp insight và tạo giá trị cho doanh nghiệp.

13 Limitations — Giới hạn của học máy

ML mạnh, nhưng không phải lúc nào cũng phù hợp

Học máy là công nghệ mạnh và đã thay đổi cách ta phân tích dữ liệu. Tuy nhiên, như mọi công nghệ, ML có những giới hạn cần hiểu rõ để dùng đúng.

13.1 Dependence on Data Quality

Phụ thuộc vào chất lượng dữ liệu

Mô hình ML “tốt đến đâu” phụ thuộc vào dữ liệu huấn luyện. Nếu dữ liệu thiếu, thiên lệch hoặc chất lượng kém, mô hình có thể hoạt động không tốt.

13.2 Lack of Transparency

Thiếu minh bạch

Mô hình ML có thể rất phức tạp, khiến khó hiểu mô hình đi đến dự đoán như thế nào. Thiếu minh bạch làm khó việc giải thích kết quả với stakeholder.

13.3 Limited Applicability

Khả năng áp dụng có giới hạn

Mô hình ML được thiết kế để tìm pattern trong dữ liệu, nên không phù hợp với mọi loại dữ liệu hay mọi loại bài toán.

13.4 High Computational Costs

Chi phí tính toán cao

Mô hình ML có thể tốn tài nguyên tính toán, cần nhiều năng lực xử lý và lưu trữ.

13.5 Data Privacy Concerns

Lo ngại về quyền riêng tư dữ liệu

Mô hình ML đôi khi thu thập và dùng dữ liệu cá nhân, dẫn tới lo ngại về quyền riêng tư và an toàn dữ liệu.

13.6 Ethical Considerations

Vấn đề đạo đức

Mô hình ML có thể duy trì/lan truyền thiên lệch hoặc phân biệt đối xử với một số nhóm, tạo ra vấn đề đạo đức.

13.7 Dependence on Experts

Phụ thuộc vào chuyên gia

Phát triển và triển khai mô hình ML cần chuyên môn về data science, thống kê và lập trình. Vì vậy, với tổ chức không có sẵn kỹ năng này, việc triển khai ML có thể khó khăn.

13.8 Lack of Creativity and Intuition

Thiếu sáng tạo và trực giác

Thuật toán ML giỏi tìm pattern trong dữ liệu nhưng thiếu sáng tạo và trực giác. Vì vậy, ML có thể không giải tốt bài toán cần tư duy sáng tạo hoặc trực giác.

13.9 Limited Interpretability

Khả năng diễn giải hạn chế

Một số mô hình (ví dụ deep neural networks) khó diễn giải. Điều này làm khó hiểu vì sao mô hình đưa ra dự đoán.

14 Real-Life Examples — Ví dụ học máy trong đời sống

ML không xa lạ như ta nghĩ

Học máy đã biến đổi nhiều ngành nhờ tự động hoá, dự đoán kết quả và phát hiện pattern trong dữ liệu lớn. Ví dụ thực tế như trợ lý ảo và chatbot (Google Assistant, Siri, Alexa), hệ gợi ý, Tesla autopilot, IBM Watson for Oncology, ...

ML trong đời sống hằng ngày

Nhiều người nghĩ ML gắn với robot tương lai và rất phức tạp, nhưng thực tế mỗi ngày chúng ta dùng ML (có thể không để ý) như Google Maps, email, Alexa, ...

14.1 Danh sách ví dụ

Các ví dụ tiêu biểu

- Trợ lý ảo và Chatbot (Virtual Assistants and Chatbots)
- Phát hiện gian lận trong ngân hàng và tài chính (Fraud Detection in Banking and Finance)
- Chẩn đoán và điều trị y tế (Healthcare Diagnosis and Treatment)
- Xe tự hành (Autonomous Vehicles)

- Hệ gợi ý (Recommendation Systems)
- Quảng cáo nhắm mục tiêu (Target Advertising)
- Nhận dạng ảnh (Image Recognition)

14.2 Virtual Assistants and Chatbots

NLP và trải nghiệm hội thoại

Xử lý ngôn ngữ tự nhiên (NLP) là một mảng của ML tập trung hiểu và sinh ngôn ngữ người. NLP được dùng trong trợ lý ảo và chatbot như Siri, Alexa, Google Assistant để tạo trải nghiệm cá nhân hoá và hội thoại. Thuật toán ML phân tích pattern ngôn ngữ và phản hồi truy vấn theo cách tự nhiên và chính xác.

Virtual assistants

Trợ lý ảo tương tác qua lệnh giọng nói, có thể thay công việc trợ lý cá nhân như gọi điện, lên lịch hẹn, hoặc đọc email. Các trợ lý phổ biến: Alexa, Apple Siri, Google Assistant.

Chatbots

Chatbot là chương trình ML được thiết kế để trò chuyện với người dùng. Ứng dụng nhằm thay thế một phần công việc chăm sóc khách hàng: cung cấp thông tin, trả lời FAQ và hỗ trợ cơ bản.

14.3 Fraud Detection in Banking and Finance

ML cho an toàn và bảo mật

ML không chỉ để “làm dễ” mà còn để tăng an toàn: ví dụ phát hiện gian lận. Thuật toán được huấn luyện trên dataset có hành vi gian lận/không mong muốn để học pattern và phát hiện khi chúng xảy ra trong tương lai.

Ví dụ thực tế

Thuật toán có thể phân tích giao dịch và nhận ra pattern chỉ ra gian lận. Ví dụ: công ty thẻ tín dụng phát hiện giao dịch nghi gian lận và báo cho khách hàng theo thời gian thực. Ngân hàng dùng ML để phát hiện rửa tiền, hành vi bất thường trong tài khoản, và phân tích rủi ro tín dụng. PayPal là một ví dụ dùng ML để cải thiện giao dịch hợp lệ trên nền tảng.

14.4 Healthcare Diagnosis and Treatment

ML hỗ trợ y tế cá nhân hoá

Ứng dụng ML trong y tế rất đa dạng: cá nhân hoá điều trị, theo dõi bệnh nhân, chẩn đoán từ ảnh y khoa. Kết hợp ML và y học nhằm tăng hiệu quả và tính cá nhân hoá.

Ví dụ

Thuật toán ML có thể phân tích dữ liệu y tế như X-ray, MRI, dữ liệu gene để hỗ trợ chẩn đoán. ML cũng có thể gợi ý điều trị hiệu quả dựa trên tiền sử và gene của bệnh nhân. Ví dụ: IBM Watson for Oncology dùng ML để phân tích hồ sơ y tế và đề xuất điều trị ung thư cá nhân hoá.

14.5 Autonomous Vehicles

Xe tự hành

Xe tự hành dùng ML để thay thế một phần người lái. Xe được thiết kế để đến đích, tránh vật cản và phản ứng điều kiện giao thông. Thuật toán ML phân tích dữ liệu từ cảm biến/camera để nhận ra vật cản và quyết định cách phản ứng.

Ví dụ

Xe tự hành có thể giúp giảm tai nạn và tăng hiệu quả. Một số công ty: Tesla, Waymo, Uber đang dùng ML để phát triển xe tự lái. Tesla Vision dùng camera/cảm biến và neural net processing để hiểu môi trường; AutoPilot là hệ thống hỗ trợ lái nâng cao.

14.6 Recommendation Systems

Cá nhân hoá ở quy mô lớn

Các nền tảng e-commerce/giải trí như Amazon và Netflix dùng hệ gợi ý (thuật toán ML) để đề xuất cá nhân hoá dựa trên lịch sử duyệt/xem. Điều này tăng hài lòng khách hàng và tăng doanh thu.

Các ví dụ hệ gợi ý

- Netflix: dùng lịch sử xem, tìm kiếm, rating để gợi ý phim/chương trình.
- Amazon: dùng sản phẩm đã xem, mua, thêm vào giỏ để gợi ý.
- Spotify: gợi ý bài hát/playlist theo lịch sử nghe, tìm kiếm, liked songs.
- YouTube: gợi ý video theo lịch sử xem, tìm kiếm, liked videos và nhiều yếu tố khác.
- LinkedIn: gợi ý việc làm/kết nối theo hồ sơ, kỹ năng, vị trí, ngành, ...

14.7 Target Advertising

Quảng cáo nhắm mục tiêu

Quảng cáo nhắm mục tiêu dùng ML để khai thác insight từ dữ liệu nhằm “may đo” quảng cáo theo sở thích, hành vi và nhân khẩu học của cá nhân/nhóm.

14.8 Image Recognition

Nhận dạng ảnh

Nhận dạng ảnh là ứng dụng computer vision, thường cần nhiều tác vụ CV như image classification, object detection và image identification. Ứng dụng: nhận diện khuôn mặt, tìm kiếm hình ảnh (visual search), chẩn đoán y khoa, nhận dạng người, ...

Kết luận

Ngoài các ví dụ trên, ML còn dùng trong quản lý năng lượng, phân tích mạng xã hội, predictive maintenance, ... Học máy có tiềm năng “cách mạng” nhiều ngành và cải thiện cuộc sống.

15 Data Structure — Cấu trúc dữ liệu trong học máy

Vì sao cấu trúc dữ liệu quan trọng trong ML?

Cấu trúc dữ liệu đóng vai trò quan trọng trong học máy vì giúp tổ chức, thao tác và phân tích dữ liệu. Dữ liệu là nền tảng của mô hình ML, và cấu trúc dữ liệu được dùng có thể ảnh hưởng đáng kể đến hiệu năng và độ chính xác.

Mục tiêu của bài

Cấu trúc dữ liệu giúp xây và hiểu các bài toán phức tạp trong ML. Chọn cấu trúc dữ liệu cẩn thận có thể giúp tăng hiệu năng và tối ưu mô hình. Bài này giới thiệu một số cấu trúc dữ liệu hay dùng và cách chúng được dùng trong ML.

15.1 Data Structure là gì?

Định nghĩa

Cấu trúc dữ liệu là cách tổ chức và lưu trữ dữ liệu để sử dụng hiệu quả. Chúng gồm các cấu trúc như array, linked list, stack, ... được thiết kế để hỗ trợ các thao tác cụ thể. Trong ML, cấu trúc dữ liệu đặc biệt quan trọng ở tiền xử lý, cài đặt thuật toán và tối ưu hoá.

15.2 Các cấu trúc dữ liệu thường dùng trong ML

Chọn đúng cấu trúc giúp làm nhanh và tiết kiệm bộ nhớ

Cấu trúc dữ liệu phù hợp giúp xử lý nhanh hơn, truy cập dữ liệu dễ hơn và lưu trữ hiệu quả hơn.

15.2.1 Arrays

Arrays

Array là cấu trúc cơ bản để lưu và thao tác dữ liệu trong ML. Phần tử truy cập qua chỉ số (index). Array cho truy xuất nhanh vì dữ liệu nằm liên tiếp trong bộ nhớ.

Vì sao dùng array trong ML?

Vì có thể thực hiện phép toán vector hoá (vectorized operations) trên array, nên biểu diễn input data dạng array là lựa chọn tốt.

Một số tác vụ dùng array

- Dữ liệu thô thường biểu diễn dưới dạng array.
- Chuyển pandas DataFrame sang list (lý do: pandas series yêu cầu cùng kiểu; Python list có thể chứa nhiều kiểu).
- Tiền xử lý: normalization, scaling, reshaping.
- Word embedding: tạo ma trận đa chiều.

Hạn chế của array

Array dễ dùng và index nhanh, nhưng kích thước cố định, có thể là hạn chế khi làm với dataset lớn.

15.2.2 Lists

Lists

List là tập các phần tử có thể khác kiểu (heterogeneous), truy cập bằng iterator. Trong ML, list thường dùng để lưu cấu trúc dữ liệu phức tạp như nested lists, dictionaries, tuples. List linh hoạt và hỗ trợ kích thước thay đổi, nhưng thường chậm hơn array do cần iteration.

15.2.3 Dictionaries

Dictionaries

Dictionary là tập các cặp key-value, truy cập theo key. Thường dùng để lưu metadata hoặc label gắn với dữ liệu; hữu ích để tạo lookup table. Dictionary truy cập nhanh nhưng có thể tốn bộ nhớ khi dataset lớn.

15.2.4 Linked Lists

Linked lists

Linked list là tập các node, mỗi node chứa dữ liệu và tham chiếu tới node tiếp theo. Trong ML, có thể dùng để lưu và thao tác dữ liệu tuần tự như time-series. Linked list chèn/xoá hiệu quả nhưng truy cập dữ liệu thường chậm hơn array/list.

Nhận xét

Linked list hữu ích cho dữ liệu động (thêm/bớt nhiều), nhưng ít phổ biến hơn array vì array truy xuất dữ liệu hiệu quả hơn.

15.2.5 Stack and Queue

Stack (LIFO)

Stack theo LIFO (Last In First Out). Cách tiếp cận “stacking classifier” có thể dùng để giải multi-class bằng cách chia thành nhiều bài toán nhị phân, rồi “stack” các output để đưa vào meta-classifier.

Queue (FIFO)

Queue theo FIFO (First In First Out), giống người xếp hàng. Queue dùng trong multi-threading để tối ưu và điều phối luồng dữ liệu giữa các thread; thường dùng để xử lý dữ liệu lớn và feed batch cho training để train liên tục và hiệu quả.

15.2.6 Trees

Trees

Tree là cấu trúc phân cấp, thường dùng trong thuật toán ra quyết định như decision trees và random forests. Tree cho thuật toán tìm kiếm/sắp xếp hiệu quả nhưng có thể phức tạp khi cài đặt và có thể bị overfitting.

Binary trees

Binary trees như một dạng tree, cũng hay dùng trong decision trees và random forests, với các đặc tính tương tự.

15.2.7 Graphs

Graphs

Graph gồm nodes và edges, dùng để biểu diễn quan hệ phức tạp giữa các điểm dữ liệu. Adjacency matrices và linked lists có thể dùng để tạo và thao tác graph. Graph có thuật toán mạnh cho clustering, classification và prediction, nhưng có thể phức tạp và gặp vấn đề scalability.

Ứng dụng graph

Graph dùng nhiều trong recommendation systems, link prediction và phân tích mạng xã hội.

15.2.8 Hash Maps

Hash maps

Hash maps thường dùng trong ML nhờ khả năng lưu và truy xuất key-value. Chúng thường dùng để lưu metadata/label; hữu ích để tạo lookup tables; nhưng có thể tốn bộ nhớ khi dataset lớn.

Cấu trúc chuyên biệt

Ngoài các cấu trúc trên, nhiều thư viện/framework ML còn cung cấp cấu trúc chuyên biệt cho use case cụ thể, như matrices và tensors cho deep learning. Khi chọn cấu trúc dữ liệu, nên cân nhắc: kích thước dữ liệu, tốc độ xử lý và bộ nhớ.

15.3 Cấu trúc dữ liệu được dùng trong ML như thế nào?

15.3.1 Storing and Accessing Data

Lưu và truy cập dữ liệu

Thuật toán ML cần nhiều dữ liệu cho train/test. Arrays, lists, dictionaries giúp lưu và truy cập hiệu quả. Ví dụ: array lưu tập số; dictionary lưu metadata/label gắn với dữ liệu.

15.3.2 Pre-processing Data

Tiền xử lý dữ liệu

Trước khi train mô hình, cần làm sạch, biến đổi và chuẩn hoá. Lists và arrays có thể dùng để lưu và thao tác dữ liệu trong tiền xử lý. Ví dụ: list lọc missing values; array chuẩn hoá dữ liệu.

15.3.3 Creating Feature Vectors

Tạo vector đặc trưng

Vector đặc trưng là thành phần quan trọng vì biểu diễn feature dùng để dự đoán. Arrays và matrices thường dùng để tạo feature vectors. Ví dụ: array lưu pixel ảnh; matrix lưu phân phối tần suất từ trong tài liệu văn bản.

15.3.4 Building Decision Trees

Xây cây quyết định

Cây quyết định dùng cấu trúc tree để ra quyết định dựa trên tập feature. Cây quyết định hữu ích cho classification và regression. Cây được tạo bằng cách chia dữ liệu đệ quy theo feature “thông tin nhất”; cấu trúc tree giúp dễ duyệt quá trình ra quyết định và dự đoán.

15.3.5 Building Graphs

Xây dựng graph

Graph dùng để biểu diễn quan hệ phức tạp giữa các điểm dữ liệu. Adjacency matrices và linked lists dùng để tạo/thao tác graph. Graph dùng cho clustering, classification và prediction.

16 Mathematics — Toán học nền tảng cho học máy

Vai trò của toán trong ML

Học máy là lĩnh vực liên ngành gồm khoa học máy tính, thống kê và toán học. Toán đóng vai trò then chốt trong việc phát triển và hiểu các thuật toán học máy. Bài này trình bày các khái niệm toán cần thiết: đại số tuyến tính, giải tích, xác suất và thống kê.

16.1 Đại số tuyến tính (Linear Algebra)

Đại số tuyến tính

Đại số tuyến tính là nhánh toán học nghiên cứu phương trình tuyến tính và cách biểu diễn chúng trong không gian vector. Trong học máy, đại số tuyến tính dùng để biểu diễn và thao tác dữ liệu: vector và ma trận biểu diễn điểm dữ liệu, đặc trưng và trọng số trong mô hình.

Vector và ma trận

Vector là danh sách có thứ tự các số; ma trận là mảng hình chữ nhật các số. Ví dụ: một vector có thể biểu diễn một điểm dữ liệu; một ma trận có thể biểu diễn cả tập dữ liệu. Các phép toán tuyến tính (nhân ma trận, nghịch đảo, ...) dùng để biến đổi và phân tích dữ liệu.

Các khái niệm đại số tuyến tính quan trọng trong ML

- **Vector và ma trận:** biểu diễn dataset, feature, giá trị mục tiêu, trọng số, ...
- **Phép toán ma trận:** cộng, trừ, nhân, chuyển vị — xuất hiện trong hầu hết thuật toán ML.
- **Giá trị riêng và vector riêng:** hữu ích cho giảm chiều, ví dụ PCA.
- **Chiều (projection):** siêu phẳng và chiếu lên mặt phẳng — cần để hiểu SVM.
- **Phân tích ma trận:** phân tích ma trận, SVD — trích thông tin quan trọng từ dữ liệu.
- **Tensor:** trong deep learning biểu diễn dữ liệu đa chiều; tensor có thể là vô hướng, vector hoặc ma trận.
- **Gradient:** tìm giá trị tối ưu của tham số mô hình.
- **Ma trận Jacobian:** phân tích quan hệ giữa biến đầu vào và đầu ra của mô hình.
- **Trực giao (orthogonality):** khái niệm cốt lõi trong PCA, SVM.

16.2 Giải tích (Calculus)**Giải tích**

Giải tích là nhánh toán học nghiên cứu tốc độ thay đổi và tích lũy. Trong ML, giải tích dùng để tối ưu mô hình bằng cách tìm cực tiểu hoặc cực đại của hàm số — điển hình là thuật toán **gradient descent**.

Gradient descent (tóm tắt)

Gradient descent là thuật toán tối ưu lặp: cập nhật trọng số mô hình theo gradient của hàm mất mát (loss). Gradient là vector các đạo hàm riêng của loss theo từng trọng số. Lặp lại cập nhật theo hướng gradient âm nhằm **giảm** loss.

Các khái niệm giải tích quan trọng trong ML

- **Hàm số:** cốt lõi của ML — mô hình học một hàm ánh xạ đầu vào $\rightarrow$ đầu ra khi huấn luyện; cần nắm hàm liên tục/rời rạc.
- **Đạo hàm, gradient, độ dốc:** hiểu cách thuật toán tối ưu (gradient descent) hoạt động.

- **Đạo hàm riêng:** tìm cực đại/cực tiểu; thường dùng trong tối ưu.
- **Quy tắc chuỗi (chain rule):** tính đạo hàm loss có nhiều biến — ứng dụng chính trong mạng nơ-ron.
- **Phương pháp tối ưu:** tìm tham số làm cost function nhỏ nhất; gradient descent là phương pháp phổ biến nhất.

16.3 Lý thuyết xác suất (Probability Theory)

Xác suất

Xác suất nghiên cứu sự không chắc chắn và tính ngẫu nhiên. Trong ML, xác suất mô hình hoá và phân tích dữ liệu không chắc chắn hoặc biến thiên; phân phối xác suất (Gaussian, Poisson, ...) mô tả xác suất điểm dữ liệu hoặc sự kiện.

Suy luận Bayes

Suy luận Bayes là kỹ thuật mô hình hoá xác suất phổ biến trong ML, dựa trên định lý Bayes: xác suất giả thuyết cho dữ liệu quan sát tỷ lệ với xác suất dữ liệu cho giả thuyết nhân với xác suất tiên nghiệm của giả thuyết. Cập nhật tiên nghiệm theo dữ liệu quan sát cho phép dự đoán hoặc phân loại theo xác suất.

Các khái niệm xác suất quan trọng trong ML

- **Xác suất đơn giản:** nền tảng; mọi bài toán phân loại đều dùng xác suất; hàm SoftMax trong ANN dùng xác suất đơn giản.
- **Xác suất có điều kiện:** ví dụ bộ phân loại Naive Bayes.
- **Biến ngẫu nhiên:** gán giá trị khởi tạo tham số mô hình — coi là bước đầu của quá trình huấn luyện.
- **Phân phối xác suất:** dùng khi xây hàm mất mát cho phân loại.
- **Phân phối liên tục và rời rạc:** mô hình hoá các kiểu dữ liệu khác nhau trong ML.
- **Hàm phân phối:** mô hình hoá phân phối sai số trong hồi quy tuyến tính và mô hình thống kê khác.
- **Ước lượng hợp lý cực đại (MLE):** nền của một số phương pháp ML và deep learning cho phân loại.

16.4 Thống kê (Statistics)

Thống kê

Thống kê nghiên cứu thu thập, phân tích, diễn giải và trình bày dữ liệu. Trong ML, thống kê dùng để đánh giá/so sánh mô hình, ước lượng tham số và kiểm định giả thuyết.

Cross-validation

Cross-validation là kỹ thuật thống kê đánh giá hiệu năng mô hình trên dữ liệu mới, chưa thấy. Tập dữ liệu được chia thành nhiều phần; mô hình được huấn luyện và đánh giá trên từng phần, từ đó ước lượng hiệu năng trên dữ liệu mới và so sánh các mô hình.

Các khái niệm thống kê quan trọng trong ML

- **Mean, median, mode:** hiểu phân phối dữ liệu và phát hiện outlier.
- **Độ lệch chuẩn, phương sai:** đo độ biến thiên của dataset và phát hiện outlier.
- **Phân vị (percentiles):** tóm tắt phân phối và nhận diện outlier.
- **Phân phối dữ liệu:** cách các điểm dữ liệu trải trên tập dữ liệu.
- **Độ lệch (skewness) và độ nhọn (kurtosis):** đo hình dạng phân phối xác suất.
- **Bias và variance:** mô tả nguồn sai số trong dự đoán; overfitting/underfitting.
- **Kiểm định giả thuyết:** giả thuyết tạm thời có thể kiểm tra bằng dữ liệu.
- **Hồi quy tuyến tính:** thuật toán hồi quy supervised phổ biến nhất.
- **Hồi quy logistic:** thuật toán supervised quan trọng trong ML.
- **PCA:** chủ yếu dùng cho giảm chiều trong ML.

17 Artificial Intelligence — Trí tuệ nhân tạo và học máy

AI và ML: hai từ hay bị nhầm

“Artificial Intelligence” và “Machine Learning” là hai cụm từ rất phổ biến trong công nghệ. Chúng thường được dùng thay cho nhau nhưng **không** trùng nghĩa. AI và ML có quan hệ với nhau nhưng khác về định nghĩa, ứng dụng và hàm ý. Bài này so sánh sự khác biệt và mối liên hệ giữa hai khái niệm.

17.1 Trí tuệ nhân tạo (AI) là gì?

Định nghĩa AI

Trí tuệ nhân tạo là lĩnh vực rộng, phát triển các máy thông minh có thể thực hiện tác vụ thường cần trí tuệ con người: nhận thức, suy luận, học và ra quyết định. Nói đơn giản: AI là khả năng của máy thực hiện tác vụ thường cần sự can thiệp hoặc trí tuệ của con người.

Hai loại AI

- **AI hẹp (narrow / weak AI)**: thiết kế cho tác vụ cụ thể (nhận dạng giọng nói, nhận dạng ảnh, ...).
- **AI tổng quát (general / strong AI)**: có thể thực hiện mọi tác vụ trí tuệ như con người.

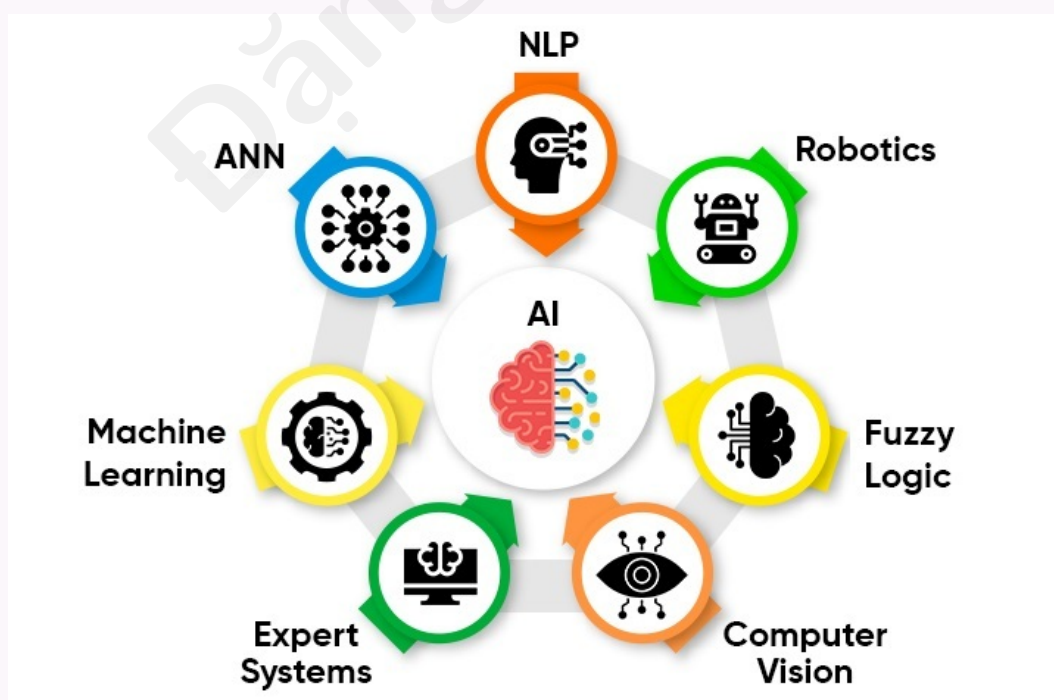
Hiện nay chỉ có AI hẹp đang được dùng; mục tiêu dài hạn là phát triển AI tổng quát áp dụng cho nhiều tác vụ.

17.2 Các nhánh của AI

AI như một “giỏ” các nhánh

AI gồm nhiều nhánh; các nhánh quan trọng gồm: Machine Learning (ML), Robotics, Expert Systems, Fuzzy Logic, Neural Networks, Computer Vision và Natural Language Processing (NLP).

Tổng quan quan hệ AI-ML



17.2.1 Robotics

Robotics

Robot chủ yếu được thiết kế để làm các tác vụ lặp lại, tẻ nhạt. Robotics là nhánh AI nghiên cứu thiết kế, phát triển và điều khiển ứng dụng robot.

17.2.2 Computer Vision

Computer Vision

Computer Vision giúp máy tính, robot và thiết bị số xử lý, hiểu ảnh/video kỹ thuật số và trích xuất thông tin quan trọng. Với AI, Computer Vision xây dựng thuật toán trích xuất, phân tích và hiểu thông tin hữu ích từ ảnh số.

17.2.3 Expert Systems

Expert Systems

Hệ chuyên gia là ứng dụng giải bài toán phức tạp trong một miền cụ thể, với trí tuệ, độ chính xác và chuyên môn gần như chuyên gia con người. Giống chuyên gia, hệ này xuất sắc trong miền mà nó được huấn luyện.

17.2.4 Fuzzy Logic

Fuzzy Logic

Máy tính thường nhận đầu vào số rõ ràng True/False; Fuzzy Logic là phương pháp suy luận giúp máy suy luận như con người trước khi quyết định. Với Fuzzy Logic, máy có thể xét các khả năng trung gian giữa YES và NO, ví dụ “Có thể đúng”, “Có lẽ không”, ...

17.2.5 Neural Networks

Neural Networks

Lấy cảm hứng từ mạng nơ-ron sinh học, Artificial Neural Networks (ANN) là tập các phần tử xử lý (node) kết nối dày, phản hồi động với đầu vào bên ngoài. ANN dùng dữ liệu huấn luyện để cải thiện hiệu quả và độ chính xác (chi tiết ở bài Neural Networks).

17.2.6 Natural Language Processing (NLP)

Natural Language Processing (NLP)

NLP cho phép hệ thống minh giao tiếp với con người bằng ngôn ngữ tự nhiên (ví dụ tiếng Anh). Có thể ra lệnh cho robot bằng tiếng Anh đơn giản; NLP cũng xử lý văn bản và hiểu nghĩa đầy đủ. Ứng dụng phổ biến: chatbot ảo, phân tích cảm xúc (sentiment

analysis).

Ví dụ về AI

Trợ lý ảo, xe tự lái, nhận diện khuôn mặt, xử lý ngôn ngữ tự nhiên, hệ thống ra quyết định.

17.3 AI và ML: tổng quan so sánh

Bốn điểm khác biệt chính

1. ML là **tập con** của AI; ML là kỹ thuật triển khai AI.
2. ML tập trung thuật toán **học từ dữ liệu**; AI tập trung máy thông minh làm tác vụ cần trí tuệ con người.
3. Thuật toán ML cải thiện độ chính xác theo thời gian nhờ dữ liệu; hệ AI có thể thích nghi tình huống/môi trường mới.
4. ML thường **hẹp** hơn (chỉ học từ dữ liệu đã cho); AI hướng tới suy luận và quyết định trong bài toán phức tạp thực tế.

17.4 Bảng: AI và Machine Learning

Khác biệt quan trọng

Tiêu chí	Artificial Intelligence	Machine Learning
Định nghĩa	Khả năng máy/hệ máy tính thực hiện tác vụ thường cần trí tuệ người (hiểu ngôn ngữ, nhận ảnh, ra quyết định).	Một loại AI cho phép hệ học và cải thiện từ kinh nghiệm mà không lập trình tường minh; mô tả cách máy học và dùng kiến thức để cải thiện quyết định.
Khái niệm	Xoay quanh thiết bị thông minh, thông minh.	Xoay quanh việc cho máy học/quyết định và cải thiện kết quả.
Mục tiêu	Mô phỏng trí tuệ con người để giải bài toán phức tạp.	Học từ dữ liệu cho sẵn và cải thiện hiệu năng máy.
Bao gồm	ANN, NLP, Fuzzy Logic, Robotics, Expert Systems, Computer Vision, ML, ...	Supervised, unsupervised, reinforcement learning.
Phát triển	Dẫn tới máy bắt chước hành vi con người.	Giúp phát triển thuật toán tự học.

18 Neural Networks — Mạng nơ-ron và học máy

Hai khái niệm thường đi cùng nhau

Học máy và mạng nơ-ron là hai công nghệ quan trọng trong AI. Chúng thường được dùng cùng nhau nhưng **không** trùng nhau. Bài này làm rõ khái niệm từng bên và so sánh sự khác biệt.

18.1 Học máy là gì?

Machine Learning

Machine Learning là lĩnh vực khoa học máy tính giúp hệ máy tính “hiểu” dữ liệu theo cách gần với con người. Nói đơn giản: ML là một dạng AI trích xuất pattern từ dữ liệu thô bằng thuật toán hoặc phương pháp.

18.2 Mạng nơ-ron là gì?

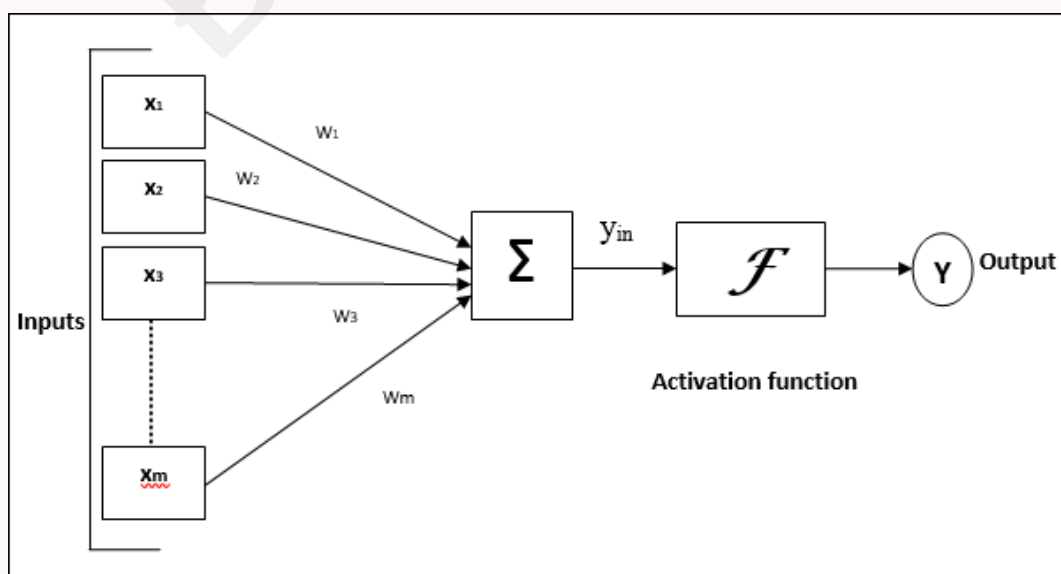
Neural Networks

Mạng nơ-ron là **một loại** thuật toán học máy, lấy cảm hứng từ cấu trúc não người. Chúng mô phỏng cách não hoạt động bằng các lớp node (nơ-ron nhân tạo) liên kết với nhau. Mỗi nơ-ron nhận đầu vào từ lớp trước và tạo đầu ra; quá trình lặp qua các lớp cho đến đầu ra cuối.

Ứng dụng

Mạng nơ-ron dùng cho nhiều tác vụ: nhận dạng ảnh, giọng nói, NLP, dự báo. Đặc biệt phù hợp khi xử lý dữ liệu phức tạp hoặc nhận diện pattern trong dữ liệu.

Mô hình ANN tổng quát



Sơ đồ trên mô tả mô hình mạng nơ-ron nhân tạo tổng quát và quá trình xử lý qua các lớp.

18.3 Machine Learning và Neural Networks

Bốn điểm so sánh chính

1. ML là **nhóm rộng** gồm nhiều thuật toán, trong đó có mạng nơ-ron.
2. ML dùng cho nhiều tác vụ; mạng nơ-ron đặc biệt mạnh với dữ liệu phức tạp và pattern tinh vi mà thuật toán khác có thể bỏ sót.
3. Mạng nơ-ron thường cần **nhiều dữ liệu** và sức mạnh tính toán (ví dụ GPU) hơn nhiều thuật toán ML cổ điển.
4. Mạng nơ-ron có thể cho dự đoán/quyết định rất chính xác nhưng đôi khi **khó giải thích** hơn các thuật toán ML khác (hộp đen).

Giải thích mô hình

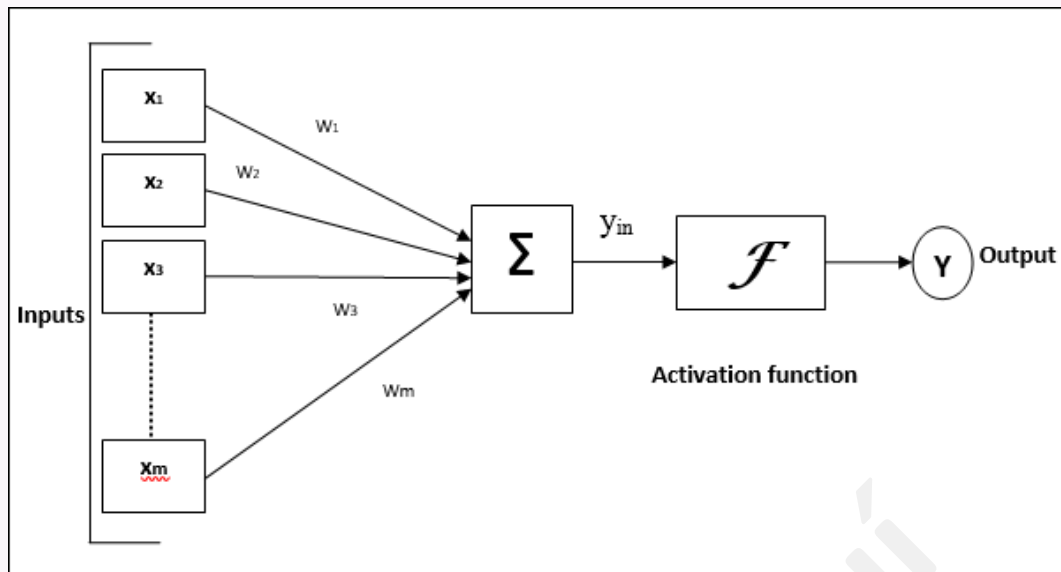
Cách mạng nơ-ron ra quyết định không luôn minh bạch, nên khó truy vết lý do đến kết luận — đây là thách thức quan trọng trong triển khai thực tế (liên quan interpretability ở các bài trước).

19 Deep Learning — Học sâu và học máy

Hai thuật ngữ hay bị dùng lẫn

Trong thế giới AI, “machine learning” và “deep learning” thường được dùng như nhau nhưng không trùng nghĩa. ML là tập con của AI cho phép máy học mà không lập trình tường minh; deep learning là tập con của ML dùng mạng nơ-ron để xử lý dữ liệu phức tạp. Bài này so sánh ML và deep learning và mối quan hệ giữa chúng.

Hình minh hoạ deep learning



19.1 Học máy là gì?

Machine Learning (ML)

Học máy (ML) là lĩnh vực con của AI, tự động cho phép máy học từ kinh nghiệm. Trong ML, phát triển thuật toán là công việc cốt lõi: thuật toán được huấn luyện trên dữ liệu để học pattern ẩn và dự đoán dựa trên những gì đã học; toàn bộ quá trình huấn luyện đôi khi gọi là **model building**.

19.2 Học sâu (Deep Learning) là gì?

Deep Learning

Deep learning là một kiểu học máy dùng mạng nơ-ron xử lý dữ liệu phức tạp: máy tự học pattern và quan hệ trong dữ liệu bằng nhiều lớp node/nơ-ron liên kết. Thuật toán deep learning phát hiện và học từ pattern để dự đoán hoặc ra quyết định.

Phù hợp với dữ liệu phức tạp

Deep learning đặc biệt phù hợp nhận dạng ảnh, giọng nói, NLP, xe tự lái; có thể xử lý lượng dữ liệu rất lớn và học pattern/quan hệ phức tạp.

Ví dụ deep learning

Nhận diện khuôn mặt, nhận dạng giọng nói, xe tự lái.

19.3 Khác biệt giữa Machine Learning và Deep Learning

Bảng so sánh

Tiêu chí	Machine Learning	Deep Learning
Định nghĩa	Tập con AI: máy học không lập trình tường minh; thuật toán học pattern ẩn từ dữ liệu để dự đoán/quyết định trên dữ liệu mới.	Tập con ML: dùng mạng nơ-ron xử lý dữ liệu phức tạp.
Độ phức tạp	Phương pháp đơn giản hơn: decision tree, linear regression, ...	Phương pháp phức tạp trong mạng nơ-ron (CNN, RNN, GAN, ...).
Lượng dữ liệu	Cần nhiều dữ liệu; vẫn hữu ích khi dữ liệu ít.	Cần nhiều dữ liệu hơn ML; độ chính xác tăng khi dữ liệu tăng.
Phương pháp huấn luyện	Bốn hướng: supervised, unsupervised, semi-supervised, reinforcement.	Phương pháp phức tạp: CNN, RNN, GAN, ...
Phụ thuộc phần cứng	Phương pháp đơn giản hơn $\Rightarrow$ ít lưu trữ và tính toán hơn.	Phức tạp hơn, dữ liệu lớn hơn $\Rightarrow$ cần nhiều lưu trữ và sức mạnh tính toán hơn.
Feature engineering	Thường phải làm feature engineering thủ công .	Mô hình deep learning có thể tự thực hiện phần lớn feature engineering.
Cách giải bài toán	Tiếp cận chuẩn; dùng thống kê và toán.	Dùng thống kê, toán kèm kiến trúc mạng nơ-ron.
Thời gian chạy	Thuật toán ML thường nhanh hơn.	Huấn luyện lâu hơn vì nhiều tham số và dữ liệu phức tạp.
Phù hợp nhất	Dữ liệu có cấu trúc (structured).	Dữ liệu phức tạp, không cấu trúc (unstructured).

19.4 So sánh sâu hơn

Bốn điểm

1. ML là nhóm rộng gồm deep learning; deep learning là **một loại** thuật toán ML dùng mạng nơ-ron.
2. Thuật toán ML cải thiện theo dữ liệu; deep learning được thiết kế để xử lý dữ liệu phức tạp và nhận pattern mà ML cổ điển có thể bỏ sót.
3. Deep learning cần nhiều dữ liệu và phần cứng mạnh (GPU) hơn nhiều thuật toán ML khác.
4. Deep learning có thể rất chính xác nhưng **khó giải thích** hơn — khó hiểu vì sao đưa ra kết luận.

19.5 AI không nhất thiết qua ML

Triển khai AI bằng quy tắc

ML và deep learning đều là kỹ thuật giải bài toán AI. Ta **có thể** triển khai AI trong thực tế mà không dùng ML/deep learning — khi đó dùng thuật toán **dựa trên quy tắc** (rule-based).

Quan hệ tập hợp

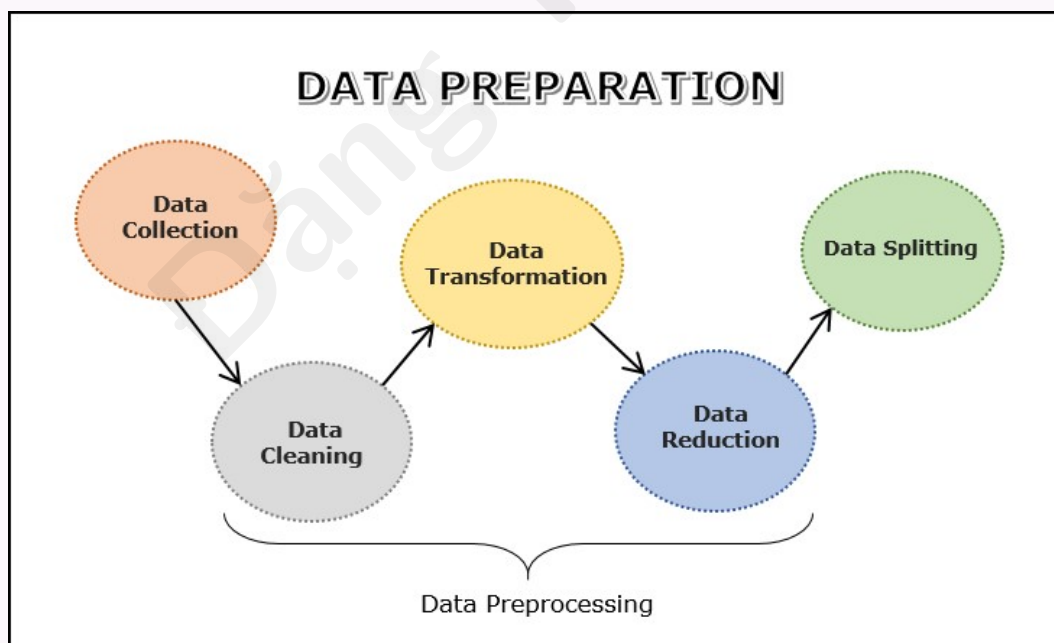
Mọi phương pháp deep learning đều thuộc machine learning, nhưng **không phải** mọi machine learning đều là deep learning.

20 Getting Datasets — Lấy bộ dữ liệu cho học máy

Mô hình chỉ tốt bằng dữ liệu huấn luyện

Mô hình học máy chỉ tốt bằng dữ liệu dùng để huấn luyện. Vì vậy, có được bộ dữ liệu chất lượng tốt và phù hợp là bước then chốt trong quy trình ML. Có nhiều kho mã nguồn mở (ví dụ Kaggle) để tải dataset; bạn cũng có thể mua dữ liệu, scrape website hoặc tự thu thập. Bài này giới thiệu các nguồn dataset và cách lấy chúng.

Hình minh họa dữ liệu cho ML



20.1 Dataset là gì?

Dataset

Dataset là tập dữ liệu được tổ chức có cấu trúc, thường dùng để đơn giản hoá phân tích, lưu trữ, xử lý hoặc huấn luyện mô hình ML. Dataset có thể lưu ở nhiều định dạng: CSV, JSON, file nén, Excel, ...

20.2 Các loại dataset

Phân loại theo kiểu thông tin

- **Tabular**: dữ liệu dạng bảng (hàng/cột).
- **Time series**: dữ liệu theo thời gian (giá cổ phiếu, khí hậu, ...).
- **Image**: ảnh cho computer vision (phân loại ảnh, phát hiện đối tượng, segmentation).
- **Text**: văn bản cho NLP (sentiment analysis, text classification, ...).

20.3 Lấy dataset cho Machine Learning

Tầm quan trọng của bước lấy dữ liệu

Lấy dataset là bước rất quan trọng khi xây giải pháp ML. Dữ liệu là yếu tố bắt buộc để huấn luyện mô hình; chất lượng, số lượng và độ đa dạng của dữ liệu ảnh hưởng mạnh tới hiệu năng mô hình.

Bốn nguồn chính

- Open Source Datasets (dataset công khai)
- Data Scraping (thu thập tự động từ web/nguồn khác)
- Data Purchase (mua dữ liệu)
- Data Collection (thu thập thủ công)

20.4 Dataset công khai / mã nguồn mở phổ biến

Kho dữ liệu công khai

Có nhiều dataset mã nguồn mở dùng cho ML, ví dụ: Kaggle, UCI Machine Learning Repository, Google Dataset Search, AWS Public Datasets. Các dataset này thường dùng cho nghiên cứu và mở cho cộng đồng.

Một số nguồn có dữ liệu có cấu trúc, giá trị cao

Kaggle Datasets, AWS Datasets, Google Dataset Search, UCI Machine Learning Repository, Microsoft Datasets, Scikit-learn Dataset, HuggingFace Datasets Hub, Government Datasets.

20.4.1 Kaggle Datasets**Kaggle Datasets**

Kaggle là cộng đồng trực tuyến phổ biến về data science và ML, lưu trữ hơn 23.000 dataset công khai. Đây là nền tảng được chọn nhiều nhất để lấy dataset vì dễ tìm, tải và đăng dữ liệu. Kaggle cung cấp dataset tiền xử lý chất lượng cao, phù hợp hầu hết mô hình ML theo nhu cầu người dùng. Kaggle còn có notebook kèm thuật toán và các loại mô hình pre-trained.

20.4.2 AWS Datasets**AWS Datasets**

Có thể tìm, tải và chia sẻ dataset công khai trong registry open data trên AWS. Dù truy cập qua AWS, dataset do tổ chức chính phủ, doanh nghiệp và nhà nghiên cứu duy trì, cập nhật.

20.4.3 Google Dataset Search Engine**Google Dataset Search**

Công cụ của Google cho phép tìm dataset từ nhiều nguồn trên web — công cụ tìm kiếm chuyên cho dataset.

20.4.4 UCI Machine Learning Repository**UCI ML Repository**

Kho dataset của Đại học California, Irvine, dành riêng cho ML. Hàng trăm dataset từ nhiều miền: time series, classification, regression, recommendation systems, ...

20.4.5 Microsoft Dataset**Microsoft Dataset**

Microsoft Research Open Data cung cấp kho dữ liệu trên cloud.

20.4.6 Scikit-learn Dataset

Scikit-learn Dataset

Scikit-learn cung cấp dataset mẫu (Iris, ...) để thử nghiệm. Dataset mở, dùng học và thử mô hình ML.

Nạp dataset Iris từ scikit-learn

```
from sklearn.datasets import load_iris
iris = load_iris()
```

Đoạn trên nạp dataset Iris vào script Python.

20.4.7 HuggingFace Datasets Hub

HuggingFace Datasets Hub

Hub cung cấp các dataset công khai lớn: ảnh, âm thanh, văn bản, ...

Cài đặt và nạp dataset với HuggingFace

Cài đặt thư viện datasets:

```
pip3 install datasets
```

Cú pháp đơn giản:

```
from datasets import load_dataset
ds = load_dataset(dataset_name)
```

Ví dụ nạp Iris:

```
from datasets import load_dataset
ds = load_dataset("scikit-learn/iris")
```

20.4.8 Government Datasets

Government Datasets

Mỗi quốc gia thường có nguồn dữ liệu chính phủ công khai, thu từ nhiều bộ ngành, nhằm minh bạch và phục vụ nghiên cứu.

Một số liên kết dataset chính phủ

- Indian Government Public Datasets
- U.S. Government's Open Data
- World Bank Data Catalog
- European Union Open Data

- U.K. Open Data

20.5 Data Scraping

Scrape dữ liệu

Data scraping là trích xuất tự động dữ liệu từ website hoặc nguồn khác. Hữu ích khi không có dataset đóng gói sẵn. Cần đảm bảo scrape **đạo đức và hợp pháp**, và nguồn đáng tin, chính xác.

20.6 Data Purchase

Mua dataset

Đôi khi cần mua dataset cho ML. Nhiều công ty bán dataset theo ngành hoặc use case. Trước khi mua, cần đánh giá chất lượng và mức độ liên quan tới dự án.

20.7 Data Collection

Thu thập thủ công

Thu thập thủ công từ nhiều nguồn — tốn thời gian, cần kế hoạch để dữ liệu chính xác, phù hợp. Có thể qua khảo sát, phỏng vấn hoặc hình thức thu thập khác.

20.8 Chiến lược có dataset chất lượng cao

Sau khi chọn nguồn, cần đảm bảo chất lượng và liên quan

- **Xác định bài toán:** trước khi lấy dữ liệu, làm rõ bài toán ML cần giải — từ đó biết cần loại dữ liệu gì và lấy ở đâu.
- **Xác định quy mô dataset:** phụ thuộc độ phức tạp bài toán; thường dữ liệu nhiều hơn giúp mô hình tốt hơn, nhưng tránh quá lớn với dữ liệu trùng/không liên quan.
- **Đảm bảo liên quan và chính xác:** dữ liệu từ nguồn tin cậy, đã được kiểm tra.
- **Tiền xử lý:** làm sạch, chuẩn hoá, biến đổi để mô hình dùng hiệu quả.

21 Categorical Data — Dữ liệu phân loại trong học máy

21.1 Dữ liệu phân loại là gì?

Categorical data

Dữ liệu phân loại trong ML là dữ liệu gồm các **nhóm** hoặc **nhãn**, thay vì giá trị số. Các nhóm có thể:

- **Nominal**: không có thứ tự tự nhiên (ví dụ màu sắc, giới tính).
- **Ordinal**: có thứ tự tự nhiên (ví dụ trình độ học vấn, nhóm thu nhập).

Biểu diễn và one-hot

Dữ liệu phân loại thường biểu diễn bằng giá trị rời rạc (số nguyên hoặc chuỗi), và thường được mã hoá thành vector one-hot trước khi đưa vào mô hình.

One-hot encoding: với mỗi category tạo vector nhị phân có bit 1 ở vị trí tương ứng category đó và 0 ở các vị trí còn lại.

21.2 Kỹ thuật xử lý dữ liệu phân loại

Tiền xử lý bắt buộc với nhiều thuật toán

Xử lý dữ liệu phân loại là phần quan trọng của tiền xử lý ML vì nhiều thuật toán yêu cầu đầu vào số. Tùy thuật toán và bản chất dữ liệu phân loại, có thể dùng: label encoding, ordinal encoding, binary encoding, ...

Các kỹ thuật trong bài

One-Hot Encoding, Label Encoding, Frequency Encoding, Target Encoding, Binary Encoding.

21.3 One-Hot Encoding

One-hot encoding

Tạo vector nhị phân cho mỗi category; mỗi phần tử biểu diễn có/không có category đó. Ví dụ biến màu với giá trị red, blue, green $\Rightarrow$ ba vector: $[1, 0, 0]$, $[0, 1, 0]$, $[0, 0, 1]$.

Ví dụ One-Hot Encoding với Pandas

```
import pandas as pd

# Creating a sample dataset with a categorical variable
data = {'color': ['red', 'green', 'blue', 'red', 'green']}
df = pd.DataFrame(data)
```

```
# Performing one-hot encoding
one_hot_encoded = pd.get_dummies(df['color'], prefix='color')

# Combining the encoded data with the original data
df = pd.concat([df, one_hot_encoded], axis=1)

# Drop the original categorical variable
df = df.drop('color', axis=1)

# Print the encoded data
print(df)
```

Output:

	color_blue	color_green	color_red
0	0	0	1
1	0	1	0
2	1	0	0
3	0	0	1
4	0	1	0

Tạo ba biến nhị phân color_blue, color_green, color_red (1 nếu có màu tương ứng, 0 nếu không), dùng cho classification/regression.

Hạn chế

One-hot phù hợp biến phân loại nhỏ, hữu hạn; với biến có cardinality lớn có thể tạo quá nhiều cột đặc trưng.

21.4 Label Encoding

Label encoding

Gán một giá trị số duy nhất cho mỗi category; thứ tự số có thể theo thứ tự category. Ví dụ biến “Size” với small, medium, large $\Rightarrow$ có thể gán 0, 1, 2.

Ví dụ Label Encoding với scikit-learn

```
from sklearn.preprocessing import LabelEncoder

# create a sample dataset with a categorical variable
data = ['small', 'medium', 'large', 'small', 'large']

# create a label encoder object
label_encoder = LabelEncoder()

# fit and transform the data using the label encoder
encoded_data = label_encoder.fit_transform(data)

# print the encoded data
```

```
print(encoded_data)
```

Output:

```
[2 1 0 2 0]
```

Mặc định thứ tự mã hoá theo thứ tự alphabet; có thể đổi thứ tự bằng cách truyền danh sách tùy chỉnh vào `LabelEncoder`.

Khi nào dùng label encoding

Hữu ích với biến ordinal (có thứ tự tự nhiên). Với biến nominal cần thận trọng vì số có thể gợi ý thứ tự không tồn tại — khi đó one-hot an toàn hơn.

21.5 Frequency Encoding

Frequency encoding

Thay mỗi category bằng **tần suất** (hoặc tỉ lệ) xuất hiện trong dataset. Ý tưởng: category xuất hiện nhiều có thể quan trọng/hữu ích hơn cho thuật toán.

Ví dụ Frequency Encoding

```
import pandas as pd

# create a sample dataset with a categorical variable
data = {'color': ['red', 'green', 'blue', 'red', 'green']}
df = pd.DataFrame(data)

# calculate the frequency of each category in the categorical variable
freq = df['color'].value_counts(normalize=True)

# replace each category with its frequency
df['color_freq'] = df['color'].map(freq)

# drop the original categorical variable
df = df.drop('color', axis=1)

# print the encoded data
print(df)
```

Output:

	color_freq
0	0.4
1	0.4
2	0.2
3	0.4
4	0.4

Ví dụ: hai lần “red” và ba lần “green” trong tập 5 mẫu $\Rightarrow$ tần suất tương ứng 0.4 và 0.6.

Ghi chú

Frequency encoding là lựa chọn thay one-hot/label khi cardinality cao; hiệu quả phụ thuộc dataset và thuật toán.

21.6 Target Encoding**Target encoding**

Thay mỗi category bằng giá trị tổng hợp (thường là **trung bình**) của biến mục tiêu (target) trong nhóm category đó. Giúp nắm quan hệ giữa biến phân loại và target, có thể cải thiện dự đoán.

Ví dụ Target Encoding (label encoder + mean theo nhóm)

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# create a sample dataset with a categorical variable and a target variable
data = {'color': ['red', 'green', 'blue', 'red', 'green'],
        'target': [1, 0, 1, 0, 1]}
df = pd.DataFrame(data)

# create a label encoder object and fit it to the data
label_encoder = LabelEncoder()
label_encoder.fit(df['color'])

# transform the categorical variable using the label encoder
df['color_encoded'] = label_encoder.transform(df['color'])

# create a mean encoder object and fit it to the transformed data
mean_encoder = df.groupby('color_encoded')['target'].mean().to_dict()

# map the mean encoded values to the categorical variable
df['color_encoded'] = df['color_encoded'].map(mean_encoder)

# print the encoded data
print(df)
```

Output:

	color	target	color_encoded
0	red	1	0.5
1	green	0	0.5
2	blue	1	1.0
3	red	0	0.5
4	green	1	0.5

Lưu ý: data leakage

Target encoding rất mạnh nhưng dễ **rò rỉ nhãn** nếu tính thống kê trên toàn bộ tập (kể cả test). Trong thực hành: chỉ fit encoder trên tập train, hoặc dùng cross-validation/out-of-fold encoding.

21.7 Binary Encoding

Binary encoding

Mỗi category được gán mã nhị phân; mỗi chữ số cho biết category có mặt (1) hay không (0). Mã nhị phân thường dựa trên vị trí category trong danh sách đã sắp xếp.

Ví dụ Binary Encoding với category_encoders

```
import pandas as pd
import category_encoders as ce

# create a sample dataset with a categorical variable
data = {'color': ['red', 'green', 'blue', 'red', 'green']}
df = pd.DataFrame(data)

# create a binary encoder object and fit it to the data
binary_encoder = ce.BinaryEncoder(cols=['color'])
binary_encoder.fit(df['color'])

# transform the categorical variable using the binary encoder
encoded_data = binary_encoder.transform(df['color'])

# merge the encoded variable with the original dataframe
df = pd.concat([df, encoded_data], axis=1)

# print the encoded data
print(df)
```

Output:

	color	color_0	color_1
0	red	0	1
1	green	1	0
2	blue	1	1
3	red	0	1
4	green	1	0

Phạm vi áp dụng

Binary encoding phù hợp biến có số category vừa phải; với số category rất lớn có thể kém hiệu quả.

22 Data Loading — Nạp dữ liệu vào dự án học máy

Bước đầu tiên của mọi dự án ML

Khi bắt đầu dự án ML, thứ cần nhất là **dữ liệu** phải được nạp vào hệ thống.

Data loading

Trong ML, data loading là quá trình nhập/đọc dữ liệu từ nguồn bên ngoài và chuyển sang định dạng thuật toán dùng được. Sau đó dữ liệu được tiền xử lý (loại bất thường, giá trị thiếu, outlier). Khi đã tiền xử lý, dữ liệu được chia train/test để huấn luyện và đánh giá mô hình.

Nguồn và định dạng

Dữ liệu có thể từ CSV, database, web API, cloud storage, ... Định dạng phổ biến nhất cho dự án ML là **CSV** (Comma Separated Values).

22.1 Lưu ý khi nạp dữ liệu CSV

CSV

CSV là định dạng văn bản thuần lưu dữ liệu dạng bảng: mỗi hàng là một bản ghi, mỗi cột là một trường/thuộc tính. Đơn giản, nhẹ, dễ đọc/xử lý bằng Python, R, Java. Trong Python có nhiều cách nạp CSV vào dự án ML; trước khi nạp cần lưu ý các thành phần sau.

22.1.1 File Header (hàng tiêu đề)

Vai trò và lưu ý

Hàng đầu tiên của CSV thường chứa tên cột.

- **Consistency**: số cột và tên cột phải nhất quán trên mọi hàng.
- **Meaningful names**: tên cột có nghĩa, mô tả; tránh `column1`, `column2`, ...
- **Case sensitivity**: tên cột có thể phân biệt hoa/thường tùy thư viện.
- **Special characters**: tránh khoảng trắng, dấu phẩy, ngoặc kép trong tên cột; dùng underscore hoặc camelCase.
- **Missing header**: nếu không có header, cần chỉ định tên cột thủ công hoặc tài liệu kèm theo.
- **Encoding**: encoding của header phải tương thích với công cụ đọc file.

22.1.2 Comments (dòng chú thích)

Dòng comment

Dòng tùy chọn bắt đầu bằng ký tự như # hoặc //, thường bị bỏ qua khi đọc CSV. Không thường là dữ liệu huấn luyện, nhưng nếu có cần xử lý đúng khi nạp.

Lưu ý khi xử lý comment

- Biết **comment marker** đang dùng.
- Comment nên ở **dòng riêng**, không trộn với dữ liệu.
- Dùng marker **nhất quán** trong cả file.
- Hiểu thư viện có bỏ qua comment mặc định hay cần tham số.
- Nếu comment chứa thông tin quan trọng, có thể cần xử lý tách khỏi dữ liệu.

22.1.3 Delimiter (ký tự phân tách)

Ảnh hưởng tới mô hình

Delimiter ảnh hưởng đáng kể độ chính xác và hiệu năng khi nạp dữ liệu.

- Chọn delimiter phù hợp (ví dụ giá trị có dấu phẩy bên trong $\Rightarrow$ có thể dùng tab hoặc semicolon thay vì comma).
- **Consistency** trong cả file.
- **Encoding** tương thích với delimiter (ví dụ delimiter không-ASCII với UTF-8).
- Một số thư viện ML yêu cầu delimiter cụ thể.

22.1.4 Quotes (dấu ngoặc kép)

Lưu ý khi dùng quotes

- Ký tự quote (") phổ biến nhất) nhất quán trong file.
- Giá trị có delimiter hoặc xuống dòng có thể được bọc trong quotes.
- Quote bên trong giá trị phải **escape** (thường nhân đôi quote), ví dụ: "John ""the Hammer""Smith".
- Cách dùng quote có thể khác nhau tùy công cụ tạo CSV.
- Encoding ảnh hưởng cách đọc quotes.

22.2 Các cách nạp file CSV trong Python

Nhiệm vụ then chốt

Nạp đúng dữ liệu là bước quan trọng nhất; CSV có nhiều biến thể và độ khó parse khác nhau.

22.2.1 Dùng module CSV (built-in)

csv.reader

```
import csv
with open('mydata.csv', 'r') as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

Đọc mydata.csv và in từng hàng.

22.2.2 Dùng thư viện Pandas

pandas.read_csv

```
import pandas as pd

data = pd.read_csv('mydata.csv')
```

Nạp CSV vào DataFrame data — tiện cho thao tác và biến đổi dữ liệu.

22.2.3 Dùng thư viện NumPy

numpy.genfromtxt

```
import numpy as np

data = np.genfromtxt('mydata.csv', delimiter=',')
```

Nạp CSV vào mảng NumPy data.

22.2.4 Dùng thư viện SciPy

scipy.loadtxt

```
import numpy as np

from scipy import loadtxt
data = loadtxt('mydata.csv', delimiter=',')
```

22.2.5 Dùng thư viện scikit-learn

load_iris

```
from sklearn.datasets import load_iris

data = load_iris().data
```

Nạp dataset Iris có sẵn trong sklearn vào mảng NumPy data.

23 Data Understanding — Hiểu dữ liệu trước khi huấn luyện

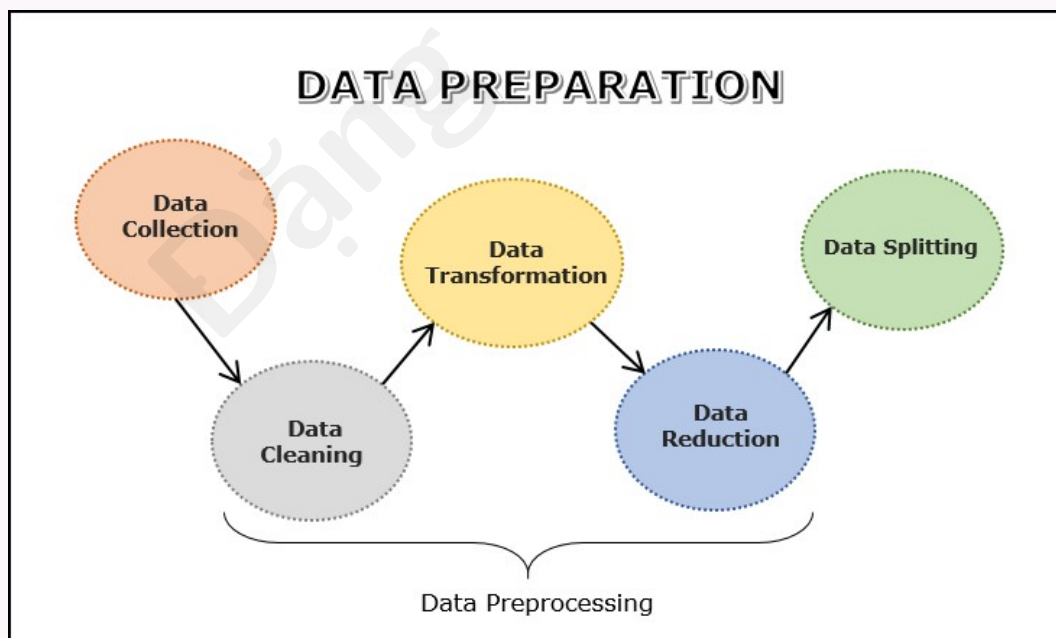
Hai phần thường bị bỏ qua: toán và dữ liệu

Trong dự án ML, người ta thường bỏ qua hai phần quan trọng: **toán học** và **dữ liệu**. Data understanding quan trọng vì ML là hướng **data-driven**: mô hình chỉ tốt bằng dữ liệu cung cấp.

Data understanding

Phân tích và khám phá dữ liệu để nhận diện pattern hoặc xu hướng có trong dữ liệu.

Hình minh họa hiểu dữ liệu



23.1 Các bước trong giai đoạn hiểu dữ liệu

Các bước trong giai đoạn hiểu dữ liệu

- **Data Collection:** thu thập dữ liệu liên quan từ database, website, API, ...
- **Data Cleaning:** loại dữ liệu không liên quan/trùng, xử lý missing; định dạng dễ phân tích.
- **Data Exploration:** khám phá pattern (histogram, scatter plot, correlation, ...).
- **Data Visualization:** biểu diễn trực quan (đồ thị, biểu đồ, bản đồ).
- **Data Preprocessing:** biến đổi dữ liệu cho phù hợp thuật toán ML (scale, đổi định dạng, giảm chiều).

23.2 Vì sao phải hiểu dữ liệu trước khi đưa vào dự án ML?

Bốn lý do chính

- **Phát hiện vấn đề chất lượng:** missing, outlier, sai kiểu dữ liệu, không nhất quán — xử lý sớm cải thiện mô hình.
- **Đánh giá mức độ liên quan:** biết feature nào quan trọng, feature nào có thể bỏ.
- **Chọn kỹ thuật ML phù hợp:** dữ liệu categorical $\Rightarrow$ classification; continuous $\Rightarrow$ regression, ...
- **Cải thiện hiệu năng mô hình:** feature engineering, tiền xử lý, chọn thuật toán đúng $\Rightarrow$ accuracy, precision, recall, F1 tốt hơn.

23.3 Hiểu dữ liệu bằng thống kê

Hai hướng: thống kê và trực quan hoá

Có thể hiểu bằng **thống kê** hoặc trực quan hoá. Dưới đây là các recipe Python; dataset dùng **sklearn**.

23.3.1 Xem dữ liệu thô (Looking at Raw Data)

head() trên dataset Pima Indians Diabetes

```
from sklearn.datasets import fetch_openml

names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
data = fetch_openml("diabetes", version=1, as_frame=True, parser="auto").frame
data.columns = names
print(data.head(10))
```

Output (10 hàng đầu):

preg	plas	pres	skin	test	mass	pedi	age	class
0	6	148	72	35	0	33.6	0.627	50
1	1	85	66	29	0	26.6	0.351	31
...								
9	8	125	96	0	0	0.0	0.232	54
10	4	110	92	0	0	37.6	0.191	30

Cột đầu là chỉ số hàng, hữu ích để tham chiếu một quan sát cụ thể.

23.3.2 Kích thước dữ liệu (Dimensions)

Vì sao cần biết số hàng/cột

- Quá nhiều hàng/cột $\Rightarrow$ huấn luyện lâu.
- Quá ít $\Rightarrow$ không đủ dữ liệu huấn luyện tốt.

shape trên Iris

```
from sklearn.datasets import load_iris
import pandas as pd
data = pd.DataFrame(load_iris().data, columns=load_iris().feature_names)
print(data.shape)
```

Output: (150, 4) — 150 hàng, 4 cột.

23.3.3 Kiểu dữ liệu từng thuộc tính

dtypes trên Iris

```
from sklearn.datasets import load_iris
import pandas as pd
data = pd.DataFrame(load_iris().data, columns=load_iris().feature_names)
print(data.dtypes)
```

Output:

```
sepal_length    float64
sepal_width     float64
petal_length    float64
petal_width     float64
dtype: object
```

Giúp biết khi nào cần chuyển kiểu (ví dụ string $\rightarrow$ float cho biến phân loại/thứ tự).

23.3.4 Tóm tắt thống kê (describe)

8 thống kê từ describe() cho mỗi cột số

Count, Mean, Standard Deviation, Min, Max, 25%, Median (50%), 75%.

describe() trên Pima Indians Diabetes

```
from sklearn.datasets import fetch_openml
from pandas import set_option

names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
data = fetch_openml("diabetes", version=1, as_frame=True, parser="auto").frame
data.columns = names
set_option('display.width', 100)
set_option('precision', 2)
print(data.shape)
print(data.describe())
```

Output: shape (768, 9) và bảng thống kê mô tả đầy đủ các cột (count, mean, std, min, quartiles, max).

23.3.5 Phân phối lớp (Class Distribution)

Quan trọng trong classification

Cần biết cân bằng lớp; lớp lệch mạnh có thể cần xử lý đặc biệt ở giai đoạn chuẩn bị dữ liệu.

groupby().size() trên cột class

```
from sklearn.datasets import fetch_openml

names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
data = fetch_openml("diabetes", version=1, as_frame=True, parser="auto").frame
data.columns = names
count_class = data.groupby('class').size()
print(count_class)
```

Output:

```
Class
0    500
1    268
dtype: int64
```

Lớp 0 gần gấp đôi lớp 1 — dataset mất cân bằng lớp.

23.3.6 Tương quan giữa các thuộc tính

Correlation

Quan hệ giữa hai biến; hệ số tương quan Pearson phổ biến:

- $= 1$: tương quan dương hoàn toàn.
- $= -1$: tương quan âm hoàn toàn.
- $= 0$: không tương quan tuyến tính.

Vì sao xem ma trận tương quan

Linear regression, logistic regression có thể kém nếu có cặp thuộc tính tương quan rất cao (đa cộng tuyến).

corr() Pearson

```
from sklearn.datasets import fetch_openml
from pandas import set_option

names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
data = fetch_openml("diabetes", version=1, as_frame=True, parser="auto").frame
data.columns = names
set_option('display.width', 100)
set_option('precision', 2)
correlations = data.corr(method='pearson')
print(correlations)
```

Ma trận output cho tương quan giữa mọi cặp thuộc tính.

23.3.7 Độ lệch phân phối (Skew)

Skewness

Phân phối giả định Gaussian nhưng bị lệch trái/phải. Nhiều thuật toán ML giả định dữ liệu gần phân phối chuẩn — skew cao có thể cần hiệu chỉnh ở bước chuẩn bị dữ liệu.

skew() trên Pima dataset

```
from sklearn.datasets import fetch_openml

names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
data = fetch_openml("diabetes", version=1, as_frame=True, parser="auto").frame
data.columns = names
print(data.skew())
```

Output:

```
preg    0.90
plas    0.17
pres   -1.84
skin    0.11
test    2.27
mass   -0.43
pedi    1.92
age     1.13
class   0.64
dtype: float64
```

Giá trị gần 0 $\Rightarrow$ ít lệch; dương/âm lớn $\Rightarrow$ lệch rõ.

24 Data Preparation — Chuẩn bị dữ liệu cho học máy

Bước then chốt ảnh hưởng mạnh tới mô hình

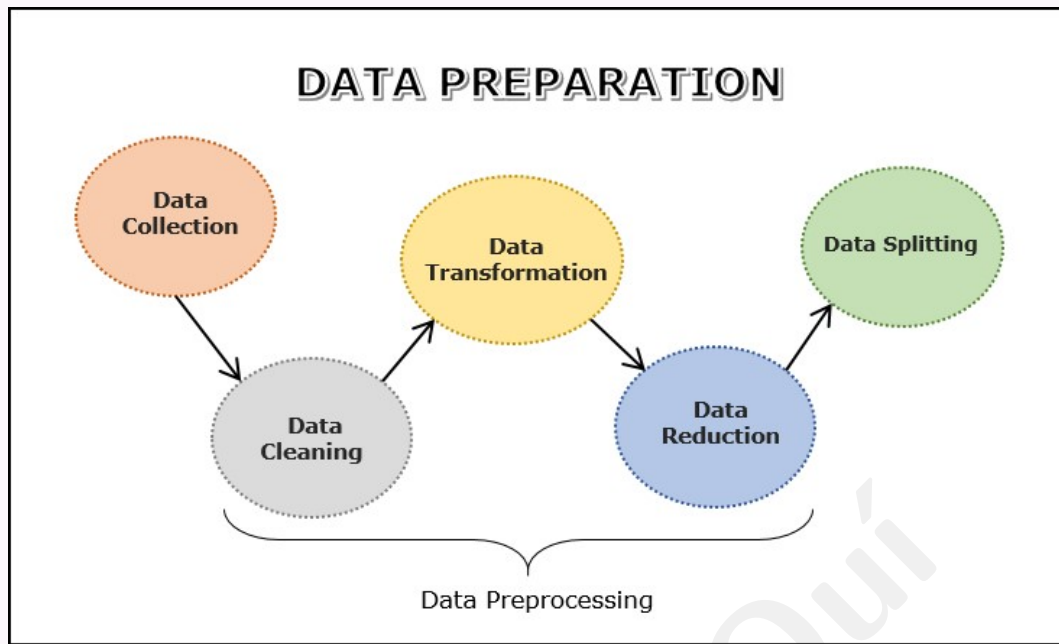
Chuẩn bị dữ liệu là bước quan trọng trong ML, ảnh hưởng lớn tới độ chính xác và hiệu quả mô hình cuối. Cần chú ý chi tiết và hiểu sâu dữ liệu cùng bài toán.

24.1 Data Preparation là gì?

Data preparation

Quá trình xử lý dữ liệu thô: làm sạch, tổ chức và biến đổi để phù hợp thuật toán ML. Đây là quá trình liên tục, ảnh hưởng lớn tới hiệu năng mô hình. Dữ liệu sạch, có cấu trúc $\Rightarrow$ kết quả tốt hơn.

Sơ đồ quy trình chuẩn bị dữ liệu



24.2 Tầm quan trọng

Vì sao chuẩn bị dữ liệu quan trọng

- **Cải thiện độ chính xác:** thuật toán học từ dữ liệu; dữ liệu sạch, có cấu trúc $\Rightarrow$ kết quả chính xác hơn.
- **Hỗ trợ feature engineering:** chọn/tạo feature mới — chuẩn bị dữ liệu tạo nền cho bước này.
- **Chất lượng dữ liệu:** dữ liệu thu thập thường có lỗi, không nhất quán; làm sạch/biến đổi giúp rút insight và pattern.
- **Tăng tốc dự đoán:** dữ liệu đã chuẩn bị dễ phân tích và cho kết quả đáng tin hơn.

24.3 Các bước trong quy trình chuẩn bị dữ liệu

Chuỗi bước mục tiêu

Data Collection $\rightarrow$ Data Cleaning $\rightarrow$ Data Transformation $\rightarrow$ Data Reduction $\rightarrow$ Data Splitting.

Mục tiêu: dữ liệu chính xác, đầy đủ, liên quan cho phân tích và mô hình hoá.

Không luôn tuần tự

Quy trình không nhất thiết theo thứ tự cố định (ví dụ có thể chia dữ liệu trước khi transform); có thể cần thu thập thêm dữ liệu.

24.4 Data Collection và tích hợp

Thu thập dữ liệu

Bước đầu: gom dữ liệu từ database, file văn bản, ảnh, âm thanh, web scraping, ...
Sau khi chọn dữ liệu thô, cần tiền xử lý để có insight — đưa dữ liệu về dạng thuật toán ML dùng được.

Đôi khi thu thập đi kèm bước **data integration**: gộp dữ liệu từ nhiều nguồn (khớp/liên kết bản ghi, merge theo biến chung).

24.5 Data Cleaning

Data cleaning

Nhận diện và sửa lỗi, giá trị thiếu, trùng lặp, outlier, ... Đảm bảo dữ liệu chính xác, liên quan, không lỗi trước khi huấn luyện.

Các bước làm sạch

1. **Xử lý trùng lặp**: nhận diện và xoá bằng `drop_duplicates` trong Pandas.
2. **Sửa lỗi cú pháp/cấu trúc**: chuẩn hoá định dạng, quy ước đặt tên.
3. **Outlier**: phát hiện bằng z-score, IQR, hoặc ML (clustering, SVM, ...).
4. **Missing values**: imputation (mean/median/mode, regression imputation, multiple imputation) hoặc deletion (mất dữ liệu).
5. **Validation**: kiểm tra kiểu dữ liệu, mã, định dạng, khoảng giá trị trước khi lưu DB.

24.6 Data Transformation

Biến đổi dữ liệu

Chuyển dữ liệu từ định dạng gốc sang định dạng phù hợp phân tích/mô hình: cấu trúc hoá, căn chỉnh, trích xuất, lưu trữ.

Kỹ thuật biến đổi phổ biến

Scaling, Normalization (L1, L2), Standardization, Binarization, Encoding, Log Transformation.

24.6.1 Scaling (Min-Max)

Rescaling

Thuộc tính thường có thang đo khác nhau; nhiều thuật toán ML cần rescale về cùng thang (thường 0–1). Dùng `MinMaxScaler` của `scikit-learn`.

MinMaxScaler trên Pima Indians Diabetes

```
from sklearn.datasets import fetch_openml
from numpy import set_printoptions
from sklearn import preprocessing

names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
dataframe = fetch_openml("diabetes", version=1, as_frame=True, parser="auto").frame
dataframe.columns = names
array = dataframe.values

data_scaler = preprocessing.MinMaxScaler(feature_range=(0,1))
data_rescaled = data_scaler.fit_transform(array)

set_printoptions(precision=1)
print ("\nScaled data:\n", data_rescaled[0:10])
```

Output (10 hàng đầu): mọi giá trị nằm trong khoảng [0, 1].

24.6.2 Normalization (L1 và L2)

Normalization

Rescale sao cho mỗi **hàng** có độ dài 1 (hữu ích với sparse data nhiều số 0). Dùng `Normalizer` của `sklearn`.

L1 Normalization: tổng giá trị tuyệt đối mỗi hàng bằng 1 (Least Absolute Deviations).

L1

```
from sklearn.datasets import fetch_openml
from numpy import set_printoptions
from sklearn.preprocessing import Normalizer

names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
dataframe = fetch_openml("diabetes", version=1, as_frame=True, parser="auto").frame
dataframe.columns = names
array = dataframe.values

Data_normalizer = Normalizer(norm='l1').fit(array)
Data_normalized = Data_normalizer.transform(array)
```

```
set_printoptions(precision=2)
print ("\nNormalized data:\n", Data_normalized [0:3])
```

L2 Normalization: tổng bình phương mỗi hàng bằng 1 (least squares).

L2

```
Data_normalizer = Normalizer(norm='l2').fit(array)
Data_normalized = Data_normalizer.transform(array)
set_printoptions(precision=2)
print ("\nNormalized data:\n", Data_normalized [0:3])
```

24.6.3 Standardization

Chuẩn hoá Gaussian

Biến đổi về phân phối Gaussian chuẩn: mean = 0, std = 1. Hữu ích cho linear regression, logistic regression giả định phân phối Gaussian đầu vào. Dùng StandardScaler.

StandardScaler

```
from sklearn.datasets import fetch_openml
from sklearn.preprocessing import StandardScaler
from numpy import set_printoptions

names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
dataframe = fetch_openml("diabetes", version=1, as_frame=True, parser="auto").frame
dataframe.columns = names
array = dataframe.values

data_scaler = StandardScaler().fit(array)
data_rescaled = data_scaler.transform(array)

set_printoptions(precision=2)
print ("\nRescaled data:\n", data_rescaled [0:5])
```

24.6.4 Binarization

Binarize / threshold

Chuyển dữ liệu thành nhị phân theo ngưỡng: trên ngưỡng → 1, dưới → 0. Hữu ích khi có xác suất cần chuyển thành giá trị rời rạc. Dùng Binarizer.

Binarizer với threshold = 0.5

```

from sklearn.datasets import fetch_openml
from sklearn.preprocessing import Binarizer

names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
dataframe = fetch_openml("diabetes", version=1, as_frame=True, parser="auto").frame
dataframe.columns = names
array = dataframe.values

binarizer = Binarizer(threshold=0.5).fit(array)
Data_binarized = binarizer.transform(array)

print ("\nBinary data:\n", Data_binarized [0:5])

```

24.6.5 Encoding (Label Encoding)**Label encoding**

Sklearn thường yêu cầu nhãn dạng số thay vì chữ; chuyển nhãn chữ → số bằng LabelEncoder.

LabelEncoder

```

import numpy as np
from sklearn import preprocessing

input_labels = ['red', 'black', 'red', 'green', 'black', 'yellow', 'white']

encoder = preprocessing.LabelEncoder()
encoder.fit(input_labels)

test_labels = ['green', 'red', 'black']
encoded_values = encoder.transform(test_labels)
print("\nLabels =", test_labels)
print("Encoded values =", list(encoded_values))

encoded_values = [3, 0, 4, 1]
decoded_list = encoder.inverse_transform(encoded_values)
print("\nEncoded values =", encoded_values)
print("\nDecoded labels =", list(decoded_list))

```

Output:

```

Labels = ['green', 'red', 'black']
Encoded values = [1, 2, 0]
Encoded values = [3, 0, 4, 1]
Decoded labels = ['white', 'black', 'yellow', 'green']

```

One-hot và target encoding chi tiết hơn ở bài **Categorical Data**.

24.6.6 Log Transformation

Log transform

Thường dùng với dữ liệu lệch (skewed): áp logarit tự nhiên lên giá trị để thay đổi thang đo.

24.7 Data Reduction

Giảm dữ liệu

Chọn tập con feature hoặc quan sát liên quan nhất để giảm kích thước dataset, giảm nhiễu, có thể cải thiện mô hình. Hữu ích khi dataset rất lớn hoặc nhiều phần không liên quan.

Kỹ thuật

- **Dimensionality reduction** (giảm chiều không mất thông tin quan trọng)
- **Discretization** (rời rạc hoá biến liên tục như thời gian, nhiệt độ).

24.8 Data Splitting

Chia dữ liệu

Bước cuối chuẩn bị dữ liệu cho ML:

- **Training**: tập huấn luyện, học pattern.
- **Validation**: đánh giá trong lúc huấn luyện.
- **Testing**: đánh giá mô hình đã huấn luyện.

24.9 Ví dụ Python: breast cancer dataset

load, split, StandardScaler

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# load the dataset
data = load_breast_cancer()

# separate the features and target
X = data.data
```



```
y = data.target

# split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# normalize the data using StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Nạp dataset, tách feature/target, chia train/test (80/20), chuẩn hoá bằng StandardScaler (fit trên train, transform trên test). Quan trọng cho SVM, neural networks khi các feature cần cùng thang đo.

24.10 Data Preparation và Feature Engineering

Hai mặt của tiền xử lý

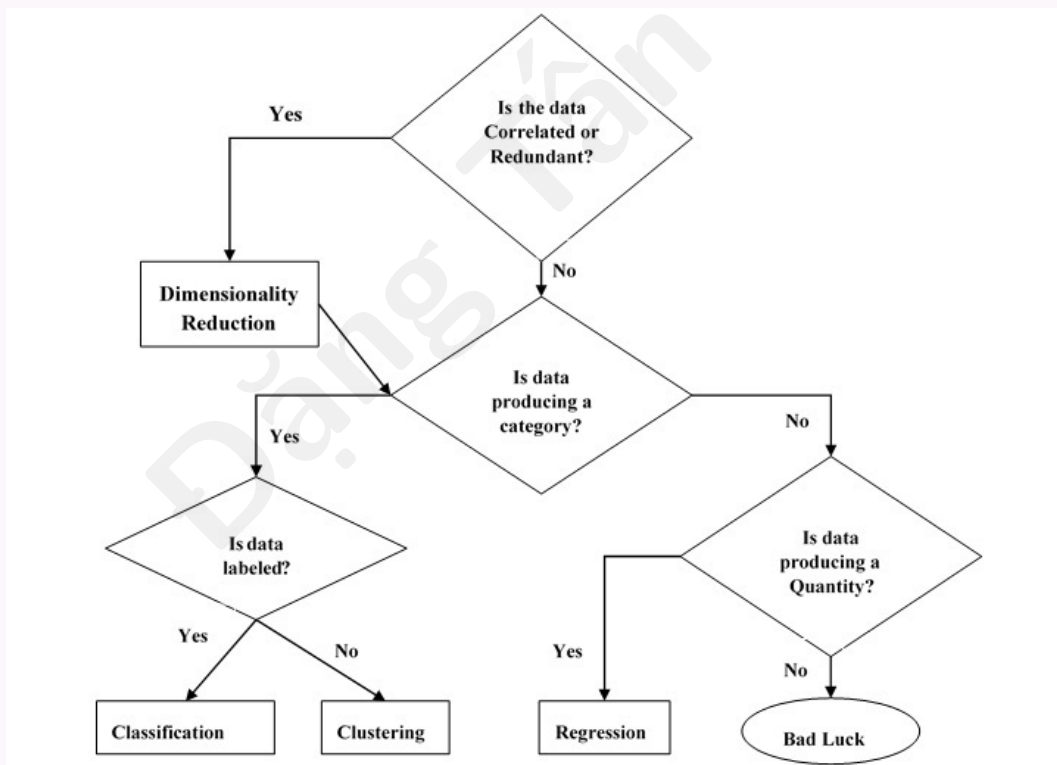
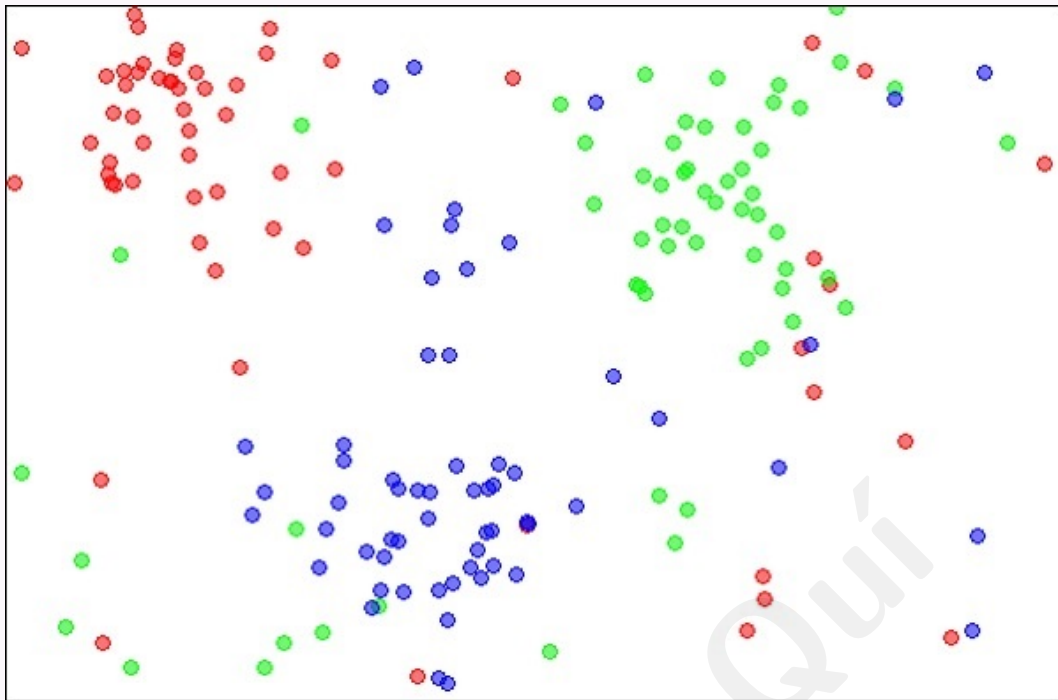
Feature engineering tạo feature mới từ dữ liệu hiện có (kết hợp, biến đổi, kiến thức miền). Data preparation và feature engineering đi song song trong pipeline tiền xử lý tổng thể.

25 Models — Các loại mô hình và phương pháp học máy

Nhiều cách xây mô hình từ dữ liệu

Có nhiều thuật toán, kỹ thuật và phương pháp ML để xây mô hình giải bài toán thực tế bằng dữ liệu.

Tổng quan các kiểu học máy



Bốn loại phương pháp ML (theo mức giám sát)

1. Supervised Learning (học có giám sát)
2. Unsupervised Learning (học không giám sát)
3. Semi-supervised Learning (học bán giám sát)

4. Reinforcement Learning (học tăng cường)

25.1 Supervised Learning (tổng quan)

Học có giám sát

Thuật toán nhận tập huấn luyện gồm mẫu đầu vào và **nhãn/đáp án** (output) tương ứng. Mục tiêu: học ánh xạ từ đầu vào x sang đầu ra Y sau nhiều lần huấn luyện:

$$Y = f(x).$$

Cần xấp xỉ f tốt để dự đoán Y cho dữ liệu x mới.

“Có giám sát”

Quá trình học như có “giáo viên” chỉ đáp án đúng. Ví dụ thuật toán: Decision Tree, Random Forest, KNN, Logistic Regression, ...

Hai nhóm tác vụ supervised

- **Classification** (phân loại)
- **Regression** (hồi quy)

25.1.1 Classification

Mục tiêu

Dự đoán nhãn phân loại (categorical) cho đầu vào; đầu ra là giá trị rời rạc, không có thứ tự bắt buộc, mỗi mẫu thuộc một lớp.

Một số mô hình phân loại phổ biến

Logistic Regression, Decision Trees, Random Forest, K-nearest Neighbor, Support Vector Machine, Naive Bayes, Linear Discriminant Analysis, Neural Networks.

25.1.2 Regression

Mục tiêu

Dự đoán giá trị số **liên tục**; mô hình học quan hệ giữa biến độc lập (feature) và biến phụ thuộc (outcome).

Một số mô hình hồi quy phổ biến

Linear Regression, Ridge Regression, Decision Trees, Random Forest, K-nearest Neighbor, Neural Network Regression.

25.2 Unsupervised Learning (tổng quan)**Học không giám sát**

Ngược với supervised: **không** có nhãn/“giáo viên” hướng dẫn. Hữu ích khi không có dữ liệu đã gán nhãn sẵn nhưng cần trích pattern từ đầu vào x .

Ví dụ trực giác

Chỉ có x , không có Y tương ứng; thuật toán tự khám phá cấu trúc thú vị trong dữ liệu. Ví dụ thuật toán: K-means clustering, K-nearest neighbors (trong ngữ cảnh unsupervised), ...

Ba nhóm tác vụ unsupervised

Clustering, Association, Dimensionality Reduction (và Anomaly Detection).

25.2.1 Clustering**Clustering**

Tìm độ tương đồng/quan hệ giữa mẫu và gom thành nhóm theo feature. Ví dụ thực tế: gom khách hàng theo hành vi mua hàng.

Mô hình clustering

K-Means, Hierarchical Clustering, Mean-shift, DBSCAN, HDBSCAN, BIRCH, Affinity Propagation, Agglomerative Clustering.

25.2.2 Association**Association / Market basket analysis**

Phân tích tập dữ liệu lớn để tìm pattern thể hiện quan hệ giữa các mục (ví dụ sản phẩm hay mua cùng nhau).

Mô hình association

Apriori, Eclat, FP-growth.

25.2.3 Dimensionality Reduction

Giảm chiều

Chọn tập feature đại diện/chính để giảm số chiều mỗi mẫu — tránh **curse of dimensionality** khi có hàng triệu feature.

Mô hình giảm chiều

PCA, Autoencoders, SVD.

25.2.4 Anomaly Detection

Phát hiện bất thường

Tìm sự kiện/quan sát hiếm, không thường xảy ra; phân biệt điểm bất thường và bình thường. Một số thuật toán clustering, KNN có thể hỗ trợ phát hiện anomaly.

25.3 Semi-supervised Learning (tổng quan)

Học bán giám sát

Không hoàn toàn supervised cũng không hoàn toàn unsupervised — nằm giữa hai loại. Thường dùng **ít** dữ liệu có nhãn và **nhiều** dữ liệu không nhãn.

Hai hướng triển khai

1. Huấn luyện mô hình supervised trên ít dữ liệu có nhãn, áp dụng lên dữ liệu không nhãn để gán nhãn thêm, huấn luyện lại và lặp.
2. Dùng unsupervised để cluster mẫu tương tự, gán nhãn cho cụm, rồi huấn luyện mô hình kết hợp thông tin này.

25.4 Reinforcement Learning (tổng quan)

Học tăng cường

Khác các phương pháp trên và ít dùng hơn trong giới thiệu cơ bản. Có **agent** được huấn luyện theo thời gian để tương tác môi trường: quan sát trạng thái, chọn hành động, nhận thưởng/phạt, cập nhật chiến lược.

Sáu bước

1. Chuẩn bị agent với tập chiến lược ban đầu.
2. Quan sát môi trường và trạng thái hiện tại.

3. Chọn policy tối ưu theo trạng thái và thực hiện hành động.
4. Nhận reward hoặc penalty.
5. Cập nhật chiến lược nếu cần.
6. Lặp bước 2–5 đến khi agent học được policy tối ưu.

Thuật toán RL phổ biến

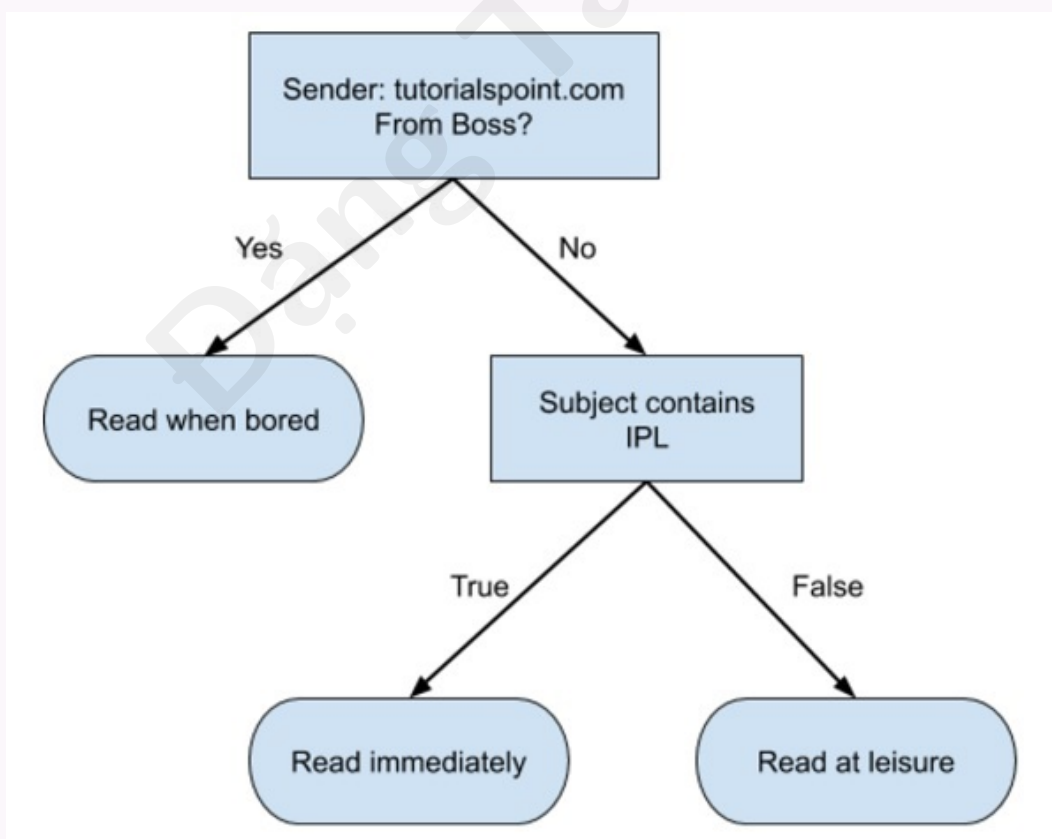
Q-learning, Markov Decision Process (MDP), SARSA, DQN, DDPG.

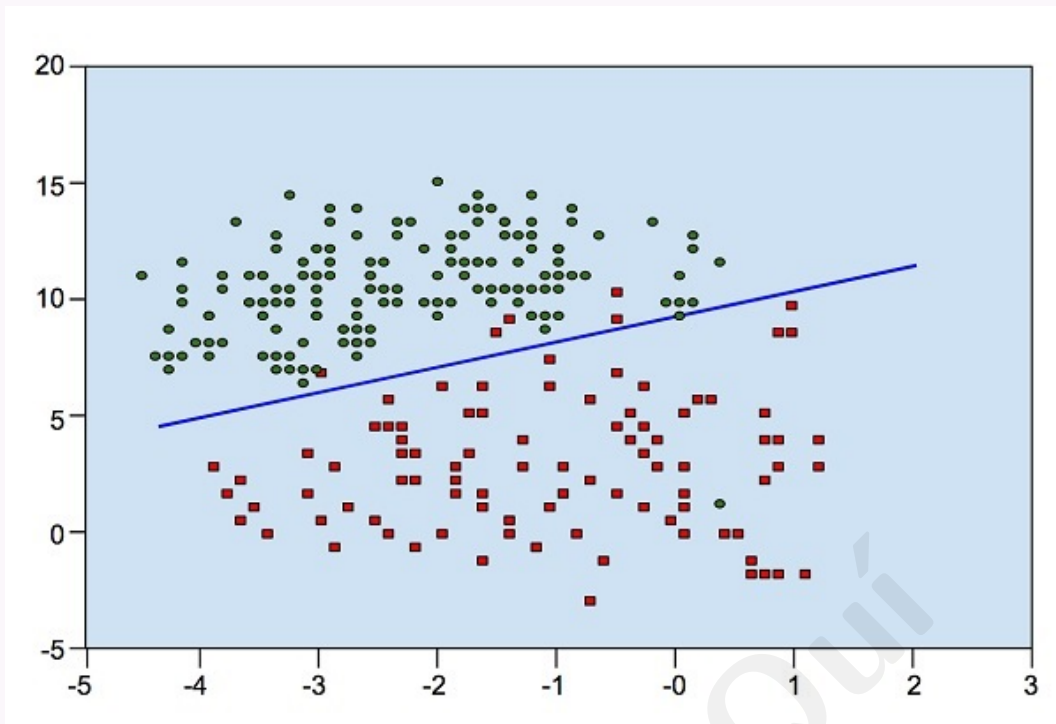
26 Supervised vs Unsupervised Learning — So sánh hai hướng học

Hai hướng phổ biến nhất

Supervised và unsupervised learning là hai cách tiếp cận phổ biến trong ML. Cách phân biệt đơn giản nhất: **loại tập huấn luyện** và **cách huấn luyện mô hình** — nhưng còn nhiều khác biệt khác, trình bày dưới đây.

Minh hoạ so sánh





26.1 Bảng khác biệt chính

Supervised vs Unsupervised

Tiêu chí	Supervised Learning	Unsupervised Learning
Định nghĩa	Thuật toán huấn luyện trên dữ liệu mà mỗi đầu vào có đầu ra tương ứng.	Thuật toán tìm pattern trong dữ liệu không có nhãn định sẵn.
Mục tiêu	Dự đoán hoặc phân loại dựa trên feature đầu vào.	Khám phá pattern, cấu trúc và quan hệ ẩn.
Dữ liệu đầu vào	Có nhãn: đầu vào kèm nhãn đầu ra.	Không nhãn: dữ liệu thô, chưa gán nhãn.
Giám sát con người	Cần giám sát để huấn luyện (gán nhãn).	Không cần giám sát khi huấn luyện.
Tác vụ	Regression, Classification.	Clustering, Association, Dimensionality Reduction.
Độ phức tạp tính toán	Thường đơn giản hơn.	Thường phức tạp hơn.
Thuật toán	Linear regression, KNN, Decision Trees, Naive Bayes, SVM.	K-Means, DBSCAN, Autoencoders.
Độ chính xác	Thường chính xác cao hơn khi có nhãn đúng.	Thường kém chính xác hơn.
Ứng dụng	Phân loại ảnh, sentiment analysis, hệ gợi ý.	Phân khúc khách hàng, anomaly detection, recommendation engines, NLP.

26.2 Chọn Supervised hay Unsupervised?

Các yếu tố cần xem xét

- **Dataset:** có nhãn hay không? có đủ thời gian, nguồn lực, chuyên môn để gán nhãn?
- **Mục tiêu:** phân loại, khám phá pattern mới, hay mô hình dự báo?
- **Thuật toán:** số feature, số chiều, khối lượng dữ liệu có phù hợp không?

26.3 Semi-supervised Learning (lối thoát trung gian)

Khi phân vân giữa hai hướng

Semi-supervised là “điểm giữa an toàn”: kết hợp supervised và unsupervised — **phần nhỏ** có nhãn, **phần lớn** không nhãn. Phù hợp khi có rất nhiều dữ liệu nhưng khó xác định feature liên quan thủ công. Chi tiết ở bài **Semi-Supervised Learning**.

27 Supervised Learning — Học máy có giám sát

27.1 Supervised Machine Learning là gì?

Học có giám sát

Supervised learning huấn luyện mô hình bằng dataset **có nhãn**: mỗi mẫu gồm feature (đầu vào) và target (đầu ra) tương ứng. Mục tiêu: học quan hệ giữa đầu vào và đầu ra sau nhiều lần huấn luyện.

27.2 Cách Supervised Learning hoạt động

Quy trình huấn luyện

- Mô hình huấn luyện trên tập cặp đầu vào–đầu ra.
- Thuật toán phân tích dataset và học quan hệ giữa feature và nhãn/target đúng.
- Trong huấn luyện, mô hình ước lượng tham số bằng cách **tối thiểu hoá hàm mất mát** (loss): đo sai khác giữa dự đoán và giá trị thực.
- Cập nhật tham số lặp đến khi loss/error đủ nhỏ.
- Sau huấn luyện, tham số tối ưu; mô hình dự đoán được cho dữ liệu mới chưa thấy.

27.3 Hai loại bài toán Supervised

27.3.1 Classification (phân loại)

Mục tiêu

Dự đoán nhãn phân loại (đúng/sai, nam/nữ, có/không, ...) — giá trị rời rạc, không có thứ tự bất buộc.

Thuật toán phổ biến

decision trees, random forests, SVM, logistic regression, ...

27.3.2 Regression (hồi quy)

Mục tiêu

Dự đoán giá trị số **liên tục**; học quan hệ giữa biến độc lập và biến phụ thuộc.

Thuật toán phổ biến

linear regression, polynomial regression, Lasso regression, ...

27.4 Thuật toán Supervised Learning

27.4.1 Linear Regression

Hồi quy tuyến tính

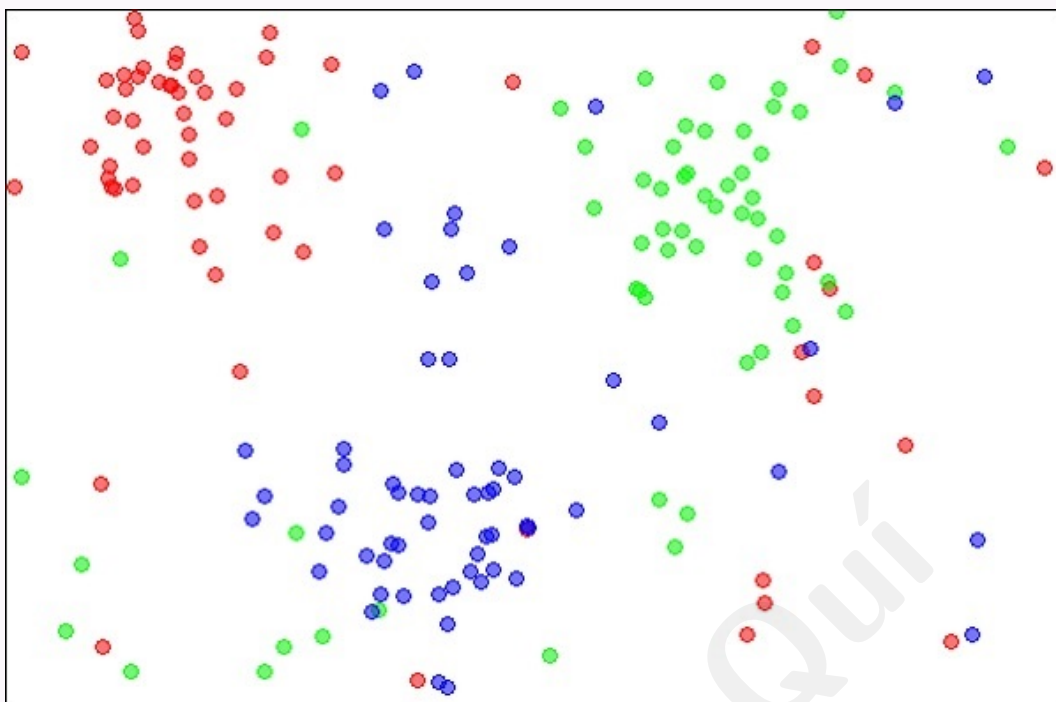
Tìm quan hệ **tuyến tính** giữa feature và giá trị đầu ra để dự báo sự kiện tương lai. Ứng dụng: phân tích cổ phiếu, dự báo thời tiết, ...

27.4.2 K-Nearest Neighbors (kNN)

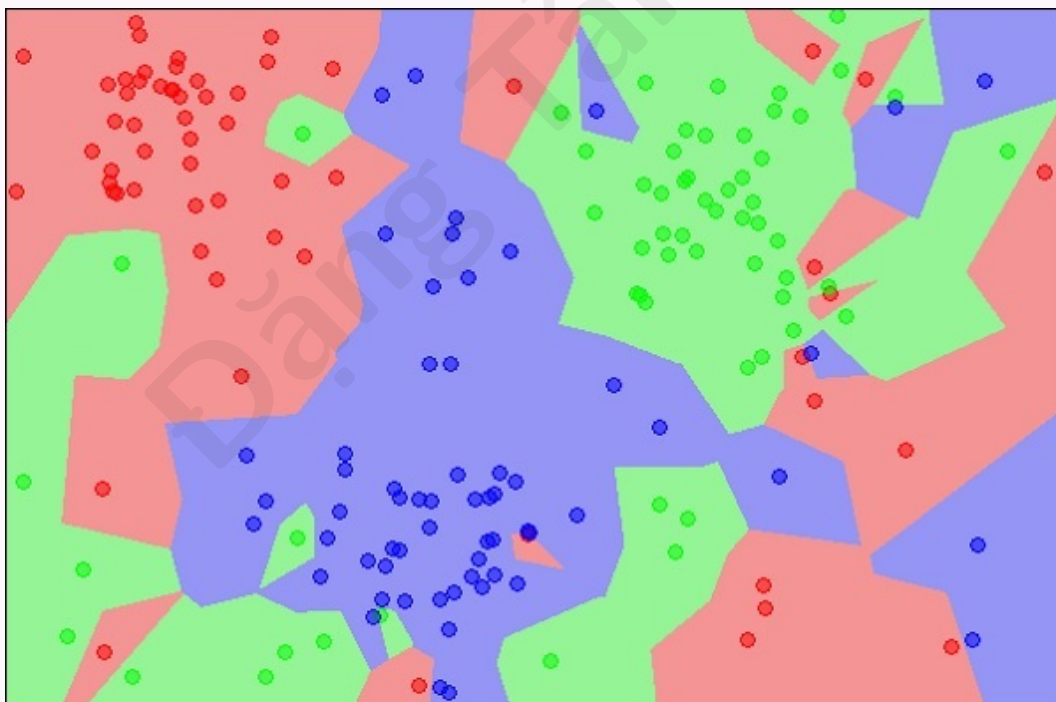
kNN

Kỹ thuật thống kê cho classification và regression: phân loại/dự đoán mẫu mới bằng khoảng cách tới các điểm trong tập huấn luyện.

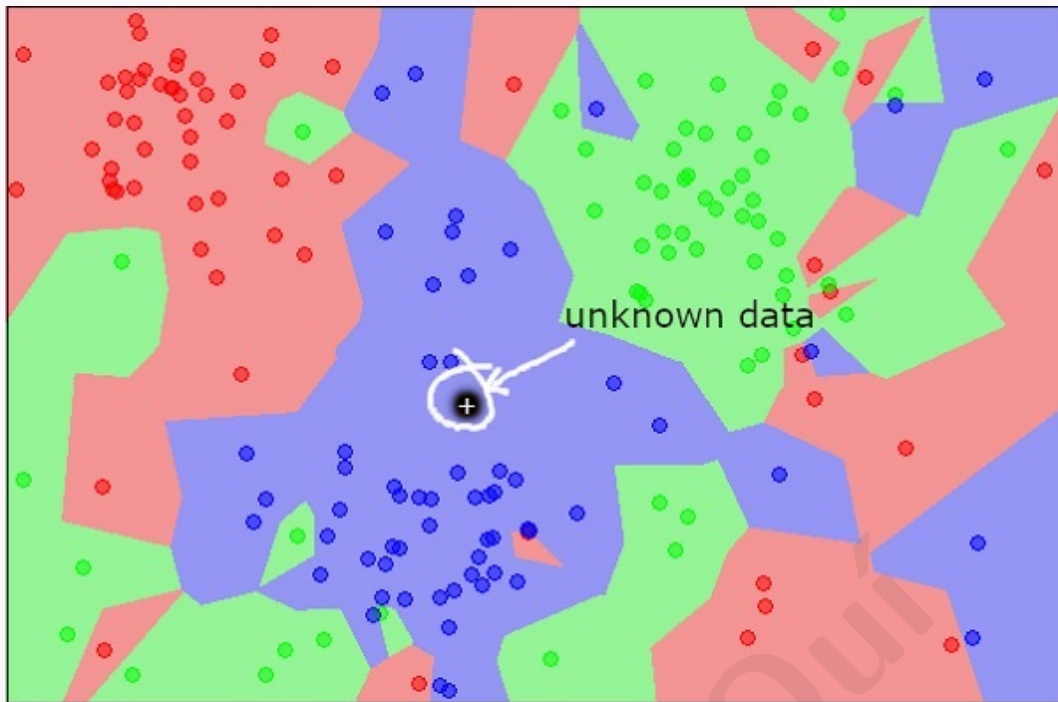
Phân loại đối tượng lạ bằng kNN



Ba loại đối tượng (đỏ, xanh dương, xanh lá). Sau kNN, biên từng lớp như sau:



Điểm mới (chưa biết lớp):



Trực quan, điểm lạ thuộc lớp xanh dương.

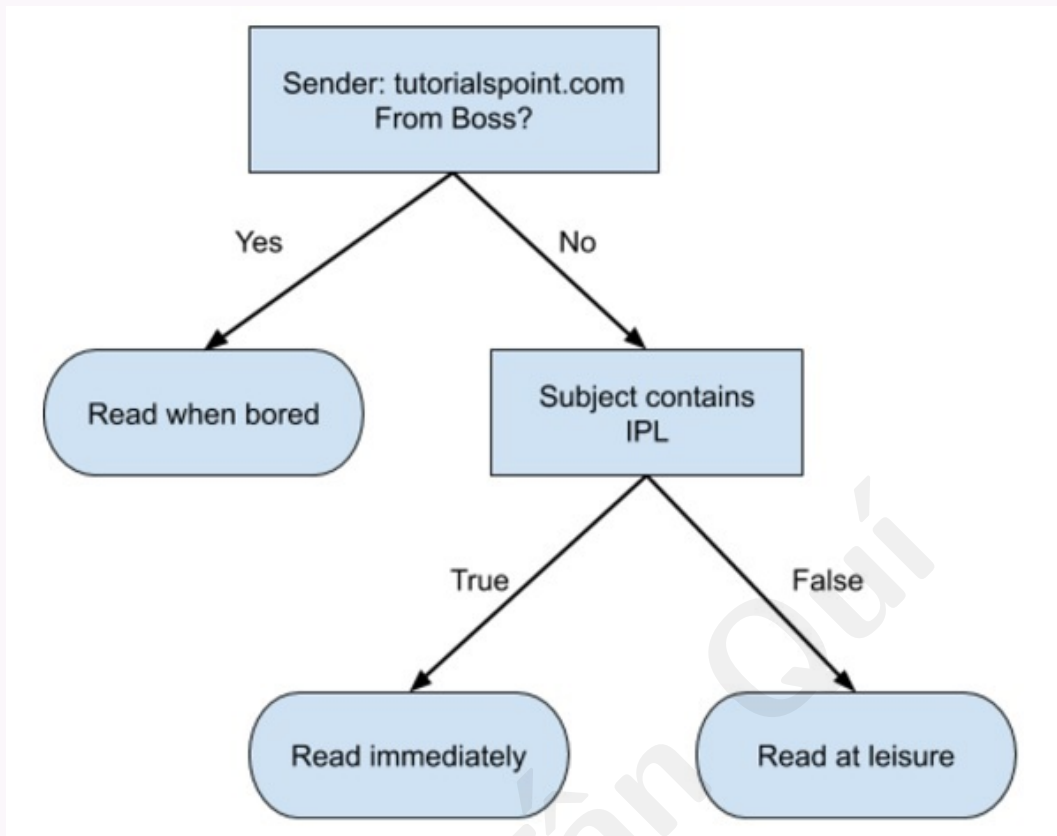
Toán học: đo khoảng cách tới mọi điểm khác — hầu hết láng giềng gần nhất là xanh dương; khoảng cách trung bình tới đỏ/xanh lá lớn hơn tới xanh dương $\Rightarrow$ phân lớp xanh dương. kNN cũng dùng cho regression; có sẵn trong hầu hết thư viện ML.

27.4.3 Decision Trees

Cây quyết định

Cấu trúc dạng cây để ra quyết định và phân tích hậu quả. Chia dữ liệu thành tập con theo feature; node trong là quyết định, lá là dự đoán cuối.

Ví dụ cây quyết định đơn giản (email)



Sơ đồ minh họa quy tắc đọc email (trivial trong ví dụ nhỏ). Trong thực tế cây có thể lớn và phức tạp; cần nắm kỹ thuật tạo và duyệt cây.

27.4.4 Naive Bayes

Naive Bayes

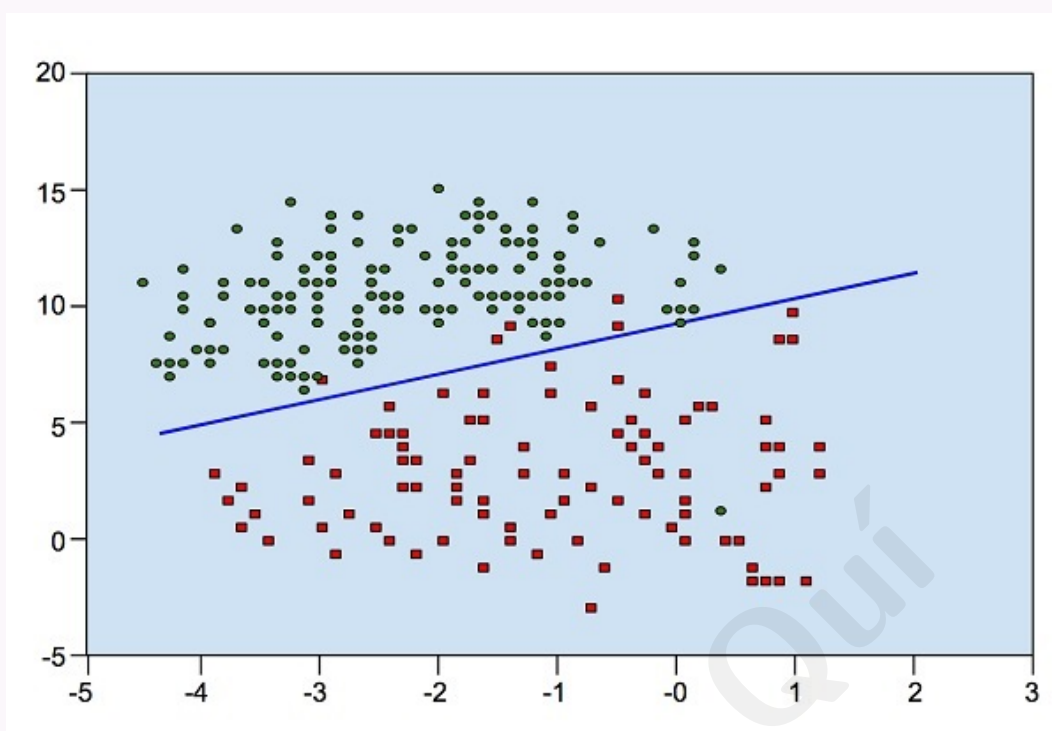
Dùng xây classifier. Ví dụ phân loại trái cây: dùng màu, kích thước, hình dạng; tính xác suất từng feature thỏa ràng buộc (ví dụ “tròn, ~10 cm $\Rightarrow$ táo”). Kết hợp xác suất các feature $\Rightarrow$ xác suất mẫu là táo. Thường cần ít dữ liệu huấn luyện.

27.4.5 Logistic Regression

Hồi quy logistic

Thuật toán thống kê ước lượng **xác suất** xảy ra sự kiện.

Phân tách lớp bằng đường biên

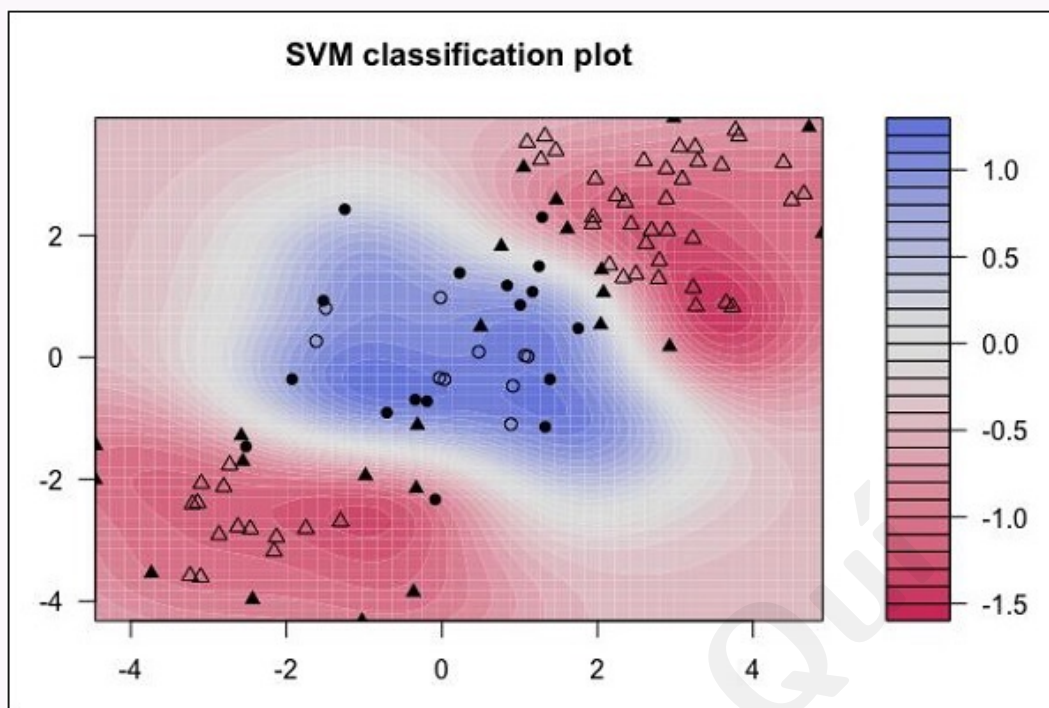


Phân bố điểm đỏ và xanh; có thể vẽ đường biên — điểm mới thuộc phía nào của đường quyết định thuộc lớp đó.

27.4.6 Support Vector Machines (SVM)

SVM

Dùng cho classification và regression. Classification: tạo **siêu phẳng** tách lớp. Regression: khớp đường hồi quy với sai số nhỏ.

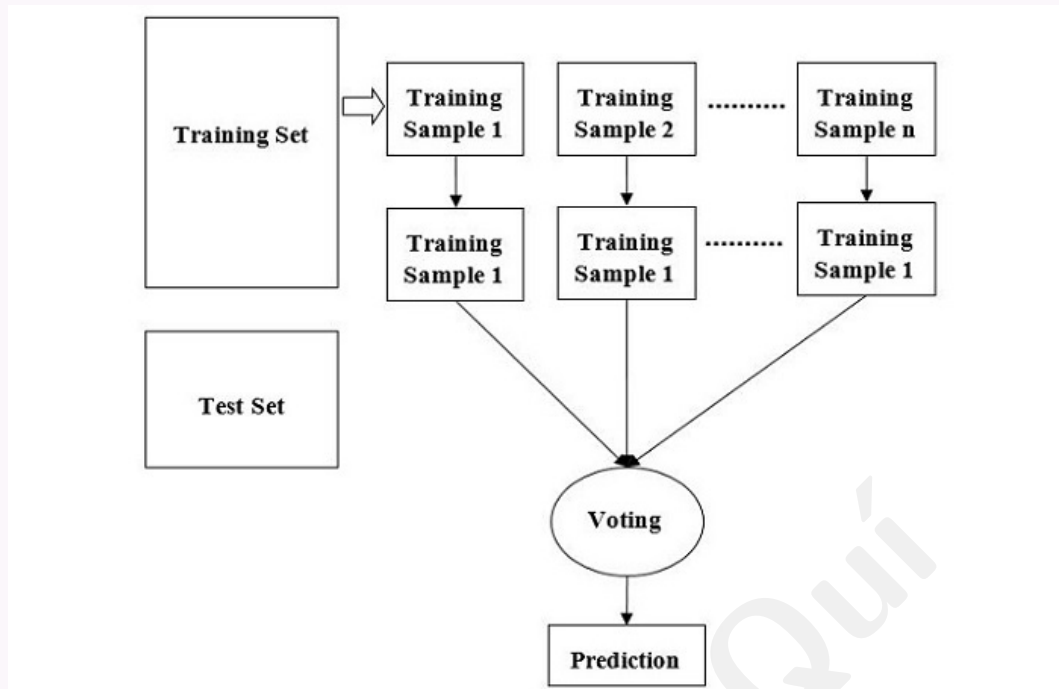
Dữ liệu không tách tuyến tính

Ba lớp không tách được bằng đường thẳng; biên phi tuyến — SVM hữu ích xác định ranh giới.

27.4.7 Random Forest**Random Forest**

Thuật toán supervised linh hoạt cho classification và regression. Kết hợp nhiều decision tree để cải thiện độ chính xác dự đoán.

Cách Random Forest hoạt động



27.4.8 Gradient Boosting

Gradient Boosting

Kết hợp các weak learner (decision tree) thành mô hình mạnh; mô hình mới sửa lỗi mô hình trước. Mục tiêu: tối thiểu hoá loss. Dùng hiệu quả cho classification và regression.

27.5 Ưu điểm Supervised Learning

Ưu điểm

- Mục tiêu rõ ràng $\Rightarrow$ cải thiện độ chính xác dự đoán.
- Dataset có nhãn giúp dự đoán và phân loại hiệu quả.
- Linh hoạt: spam detection, giá cổ phiếu, ...

27.6 Nhược điểm

Nhược điểm

- Cần **lượng lớn** dữ liệu có nhãn — tốn kém, tốn thời gian thu thập và gán nhãn.
- Khó dự đoán chính xác nếu dữ liệu test khác phân phối train.
- Gán nhãn chính xác phức tạp, cần chuyên môn.

27.7 Ứng dụng

Lĩnh vực ứng dụng

Image recognition: huấn luyện trên ảnh có nhãn; nhận diện khuôn mặt, phát hiện đối tượng.

Predictive analytics: học từ dữ liệu lịch sử có nhãn để nhận diện xu hướng, hỗ trợ quyết định kinh doanh dựa trên dữ liệu.

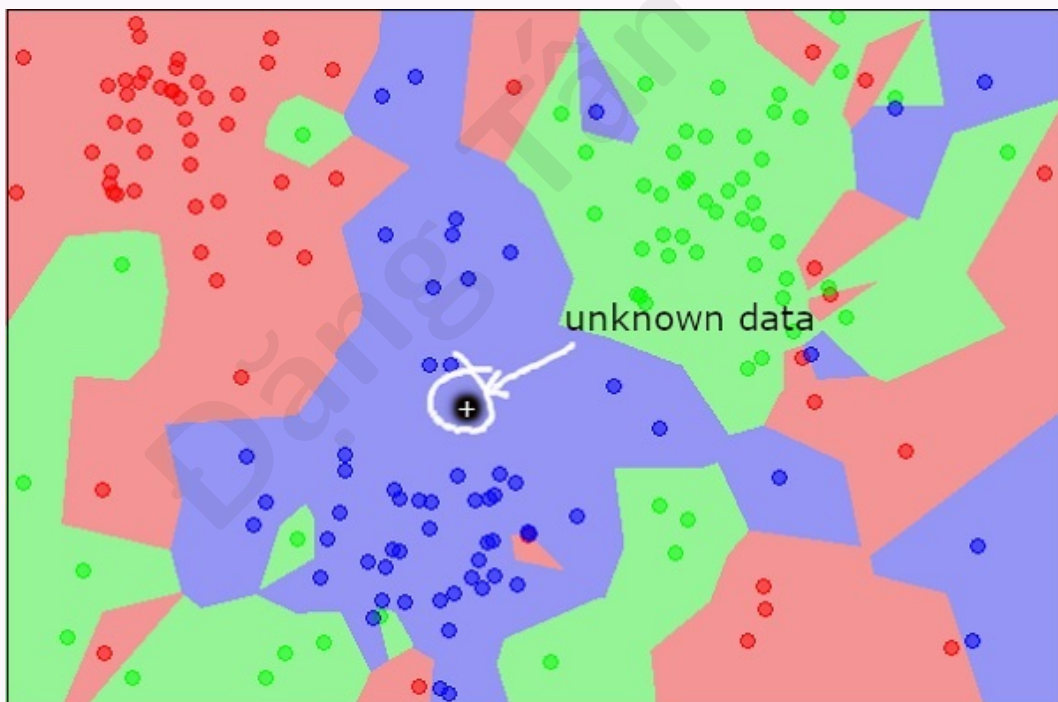
28 Unsupervised Learning — Học máy không giám sát

28.1 Unsupervised Machine Learning là gì?

Học không giám sát

Unsupervised learning học pattern và cấu trúc trong dữ liệu **không** cần giám sát con người. Thuật toán phân tích dữ liệu để khám phá pattern ẩn trong tập **không nhãn**.

Minh họa unsupervised learning



Khác supervised

Không có quyền “thoải mái” như supervised với dữ liệu đã gán nhãn sẵn; cần trích pattern hữu ích từ đầu vào.

Tóm tắt một câu

- Là cách tiếp cận/kỹ thuật ML
- Dùng thuật toán ML
- Tìm pattern/cấu trúc ẩn
- Trong dữ liệu không có giám sát con người

Một số hướng và ví dụ

Association, clustering, dimensionality reduction. Ví dụ thuật toán: K-means clustering, K-nearest neighbors, ...

Ví dụ trực giác: dữ liệu cử tri

Thay vì huấn luyện máy dự đoán Y cho mỗi X như supervised, ta hỏi: “Với tập dữ liệu khổng lồ X , bạn cho biết gì về X ?” Hoặc: “Có thể chia X thành năm nhóm tốt nhất nào?” Hoặc: “Ba feature nào hay xuất hiện cùng nhau nhất trong X ?” — Đó là tinh thần của unsupervised learning.

28.2 Cách Unsupervised Learning hoạt động**Quy trình huấn luyện**

- Thuật toán tự học (self-learning) huấn luyện trên dữ liệu **không nhãn**.
- Tùy tác vụ (clustering, association, ...) chọn thuật toán phù hợp.
- Thuật toán tự suy ra quy tắc dựa trên độ tương đồng, pattern, khác biệt — không có nhãn hay pre-training.
- Kết quả: **mô hình unsupervised**, sẵn sàng cho clustering, association hoặc giảm chiều.

Phù hợp bài toán phức tạp

Mô hình unsupervised phù hợp tổ chức dataset lớn thành các cụm.

28.3 Ba nhóm phương pháp**28.3.1 Clustering****Clustering**

Gom tập đối tượng/điểm dữ liệu thành cụm theo độ tương đồng. Điểm trong cùng cụm phải **giống nhau hơn** điểm ở cụm khác. Đôi khi gọi là **unsupervised classification** vì kết quả giống phân loại nhưng không có lớp định sẵn.

Một số thuật toán

K-Means: gán điểm vào một trong K cụm theo khoảng cách tới tâm; cập nhật centroid lặp đến khi ổn định.

Mean Shift: tìm vùng mật độ dữ liệu cao; dịch mean của từng điểm về vùng dày nhất.

Gaussian Mixture Models: mô hình xác suất kết hợp nhiều phân phối Gaussian để xác định điểm thuộc thành phần nào.

28.3.2 Association Rule Mining**Association**

Kỹ thuật dựa trên luật để tìm liên kết giữa tham số trong dataset lớn. Phổ biến trong Market Basket Analysis; hỗ trợ quyết định và recommendation engine. Thuật toán chính:

Apriori — tìm item lặp lại thường xuyên và luật kết hợp trong dữ liệu giao dịch.

28.3.3 Dimensionality Reduction**Giảm chiều**

Chọn tập feature đại diện để giảm số chiều mỗi mẫu — tránh curse of dimensionality khi có hàng triệu feature. Thuật toán phổ biến: Principal Component Analysis (PCA), Missing Value Ratio, SVD, Autoencoders.

28.4 Thuật toán Unsupervised Learning**Danh sách thường dùng**

Clustering: K-Means, Hierarchical, Mean-shift, DBSCAN, HDBSCAN, BIRCH, Affinity Propagation, Agglomerative.

Association: Apriori, Eclat, FP-growth.

Giảm chiều: PCA, Autoencoders, SVD.

28.5 Ưu điểm**Ưu điểm**

- Không cần dữ liệu có nhãn — dễ và rẻ hơn.
- Khám phá pattern/quan hệ ẩn $\Rightarrow$ insight và ra quyết định.
- Phù hợp clustering, anomaly detection, giảm chiều.

28.6 Nhược điểm

Nhược điểm

- **Khó đánh giá:** không có nhãn chuẩn để đo hiệu năng.
- **Kết quả có thể kém chính xác:** đặc biệt khi dữ liệu nhiễu hoặc thuật toán không biết đầu ra đúng.

28.7 Ứng dụng

Thực tế

- **Customer Segmentation:** gom khách theo mua hàng, hoạt động, sở thích.
- **Anomaly Detection:** phát hiện pattern bất thường (gian lận, an ninh mạng).
- **Recommendation Engines:** phân tích hành vi khách hàng, marketing cá nhân hoá.
- **NLP:** ví dụ Google phân loại bài báo tin tức.

28.8 Anomaly Detection

Phát hiện bất thường

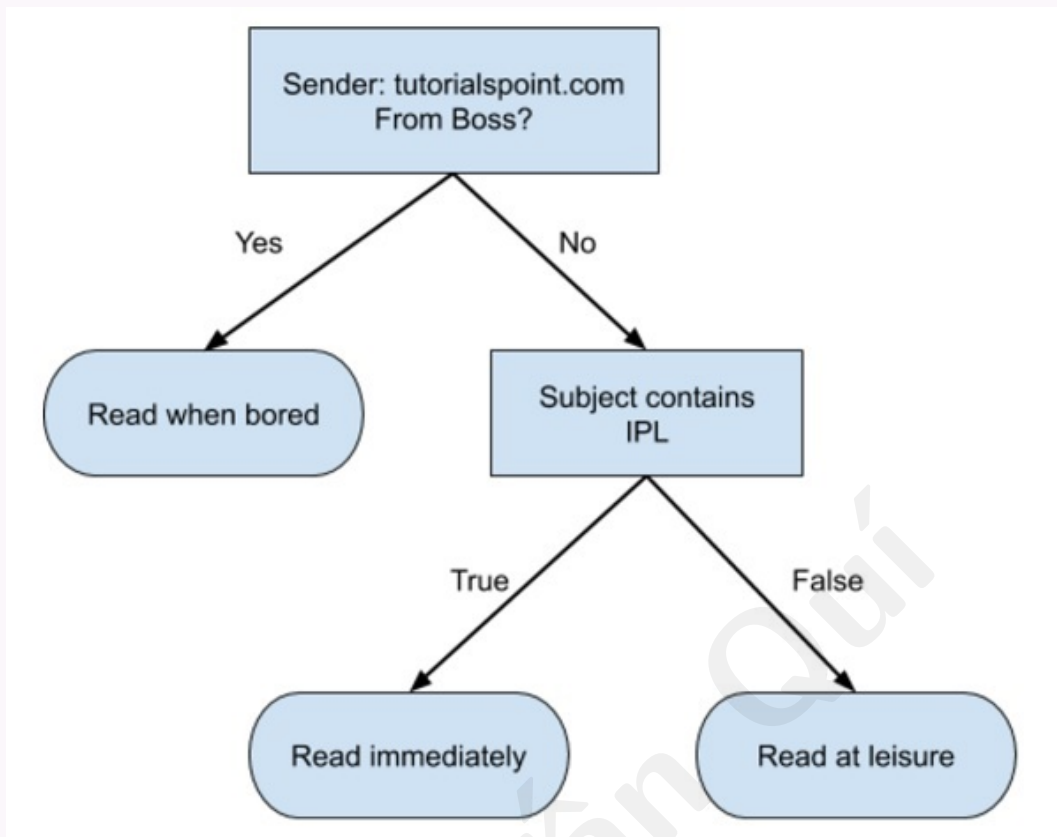
Tìm sự kiện/quan sát hiếm, hiếm khi xảy ra. Dùng kiến thức đã học để phân biệt điểm bất thường và bình thường. Clustering, KNN có thể hỗ trợ dựa trên feature.

29 Semi-Supervised Learning — Học máy bán giám sát

Giữa supervised và unsupervised

Semi-supervised learning không hoàn toàn có giám sát cũng không hoàn toàn không giám sát — nằm giữa hai phương pháp.

Minh hoạ semi-supervised



Bối cảnh sử dụng

Huấn luyện trên dataset gồm cả dữ liệu **có nhãn** và **không nhãn**. Thường dùng khi có rất nhiều dữ liệu không nhãn. Gán nhãn thủ công trong supervised tốn kém; unsupervised thuần có ứng dụng hạn chế $\Rightarrow$ semi-supervised cân bằng hai bên.

29.1 Semi-Supervised Learning là gì?

Định nghĩa

Kỹ thuật ML kết hợp supervised và unsupervised: huấn luyện trên **ít** dữ liệu có nhãn và **nhiều** dữ liệu không nhãn.

Mục tiêu

Phát triển thuật toán chia toàn bộ dữ liệu thành các cụm; điểm gần nhau trong không gian đặc trưng có khả năng cao cùng nhãn đầu ra, rồi gán cụm vào category định sẵn.

Tóm tắt

- Cách tiếp cận ML kết hợp supervised và unsupervised
- Huấn luyện bằng dữ liệu có nhãn và không nhãn

- Phục vụ classification và regression

29.2 So với Supervised Learning

Khác biệt chính: dataset

Supervised: mỗi mẫu có cặp feature–nhãn đích $\Rightarrow$ dự đoán/phân loại chính xác hơn khi nhãn đủ tốt.

Semi-supervised: ít mẫu có nhãn, nhiều mẫu không nhãn; huấn luyện ban đầu trên phần có nhãn, dùng insight đó để học thêm pattern từ phần không nhãn.

29.3 So với Unsupervised Learning

Khác biệt

Unsupervised: chỉ dữ liệu không nhãn, gom cụm theo feature chung.

Semi-supervised: hỗn hợp có nhãn + không nhãn; hiệu quả hơn vì mỗi cụm có thể được gán nhãn định sẵn nhờ phần dữ liệu đã gán nhãn.

29.4 Khi nào chọn Semi-Supervised?

Tình huống điển hình

Khó và đắt có đủ dữ liệu có nhãn, nhưng thu thập dữ liệu không nhãn dễ hơn. Khi đó supervised thuần hoặc unsupervised thuần có thể không đủ tốt $\Rightarrow$ dùng semi-supervised.

29.5 Cách Semi-Supervised Learning hoạt động

Thành phần huấn luyện

Thành phần supervised nhỏ (ít dữ liệu có nhãn) + thành phần unsupervised lớn (nhiều dữ liệu không nhãn).

Hai hướng triển khai

1. Huấn luyện mô hình supervised trên ít dữ liệu có nhãn; áp dụng lên dữ liệu không nhãn để tạo thêm nhãn; huấn luyện lại và lặp.
2. Dùng unsupervised cluster mẫu tương tự, gán nhãn cho cụm, huấn luyện mô hình với thông tin kết hợp.

Dữ liệu không nhãn phải liên quan

Dữ liệu không nhãn phải phù hợp tác vụ mô hình đang học. Trong ký hiệu xác suất: phân phối đầu vào $p(x)$ phải chứa thông tin về $p(y|x)$ — xác suất mẫu x thuộc lớp y .

29.6 Các giả định (assumptions)**29.6.1 Giả định mượt mà (Smoothness Assumption)****Smoothness Assumption**

Hai điểm x_1, x_2 trong vùng mật độ cao (cùng cụm) thì nhãn y_1, y_2 nên gần nhau. Ở vùng mật độ thấp, nhãn đầu ra không nhất thiết gần.

29.6.2 Giả định cụm (Cluster Assumption)**Cluster Assumption**

Điểm trong cùng cụm có khả năng cùng lớp. Dữ liệu không nhãn giúp xác định biên cụm chính xác hơn; dữ liệu có nhãn gắn lớp cho từng cụm.

29.6.3 Giả định mật độ thấp (Low Density Separation)**Low Density Separation**

Biên quyết định nên nằm ở vùng mật độ thấp. Ví dụ nhận dạng chữ số 0 vs 1: điểm ngay trên biên trông như số “dài” lẫn lộn — xác suất gặp mẫu như vậy rất nhỏ.

29.6.4 Giả định manifold (Manifold Assumption)**Manifold Assumption**

Trong không gian đầu vào chiều cao, dữ liệu nằm trên các manifold chiều thấp hơn; điểm cùng nhãn nằm trên cùng manifold.

29.7 Kỹ thuật Semi-Supervised**Một số kỹ thuật phổ biến**

- **Self-training:** chỉnh supervised (classification/regression) để dùng cả nhãn và không nhãn.
- **Co-training:** mở rộng self-training với nhiều “view” dữ liệu (ví dụ trang web: nội dung text + hyperlink); hai classifier riêng trên hai view cải thiện học.
- **Graph-based label propagation:** mô hình hoá dữ liệu thành đồ thị (node =

điểm, cạnh = độ tương đồng); nhãn lan truyền từ điểm có nhãn sang láng giềng; cập nhật lặp để gán nhãn cho node chưa nhãn.

29.8 Thách thức

Thách thức

- **Chất lượng dữ liệu không nhãn:** nhiều hoặc không liên quan $\Rightarrow$ dự đoán sai.
- **Distribution shift:** dữ liệu có nhãn và không nhãn khác phân phối (ví dụ ảnh rõ vs ảnh camera an ninh) $\Rightarrow$ khó tổng quát hoá.

29.9 Ứng dụng

Lĩnh vực

Text classification, image classification, speech analysis, anomaly detection — mục tiêu chung: gán thực thể vào category định sẵn; điểm gần nhau có khả năng cùng nhãn.

Speech Recognition: gán nhãn audio tốn thời gian; kết hợp audio không nhãn với ít bản ghi có transcript.

Web Content Classification: hàng tỷ trang, không thể gán nhãn thủ công; hỗ trợ công cụ tìm kiếm xếp hạng nội dung.

Text Document Classification: học từ ít tài liệu có nhãn + corpus lớn không nhãn $\Rightarrow$ tăng độ chính xác không cần dataset nhãn khổng lồ.

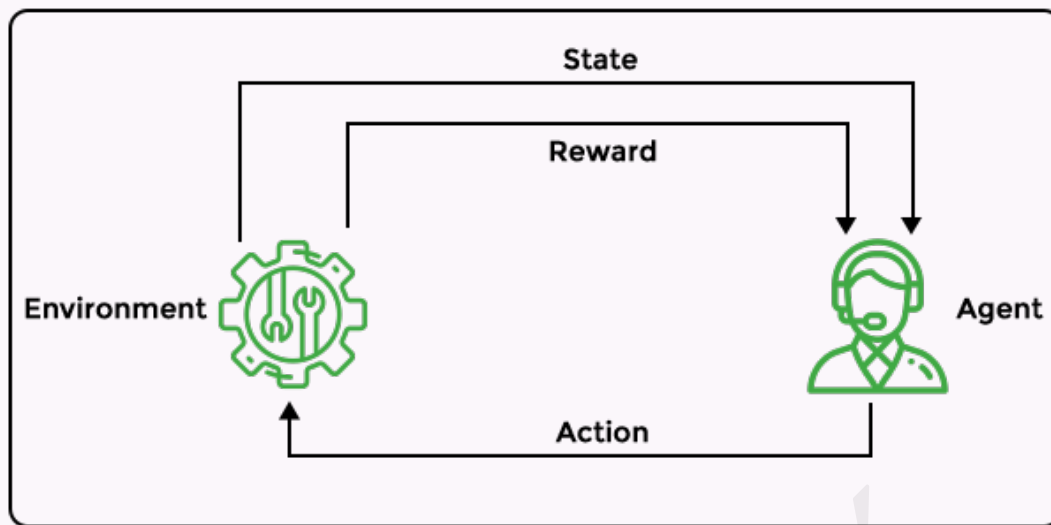
30 Reinforcement Learning — Học tăng cường

30.1 Reinforcement Learning là gì?

Học tăng cường

Agent (thực thể phần mềm) được huấn luyện để hiểu môi trường bằng cách thực hiện hành động và quan sát kết quả. Hành động tốt $\Rightarrow$ phản hồi tích cực; hành động xấu $\Rightarrow$ phản hồi tiêu cực. Lấy cảm hứng từ cách động vật học từ kinh nghiệm: quyết định dựa trên hậu quả hành động.

Mô hình RL điển hình



Agent ở một trạng thái, thực hiện hành động trong môi trường để hoàn thành tác vụ; nhận reward hoặc punishment.

30.2 Cách Reinforcement Learning hoạt động

Vòng lặp agent–môi trường

Huấn luyện agent theo thời gian để tương tác môi trường: theo chiến lược, quan sát trạng thái, chọn hành động, học từ reward/penalty.

Ví von kỳ thử cờ

Exploration: như kỳ thử thử nhiều nước đi và hậu quả — agent thử hành động khác nhau để hiểu tác động.

Exploitation: dùng kinh nghiệm quá khứ chọn nước đi tốt — agent tận dụng kiến thức đã học.

30.3 Bốn thành phần chính (ngoài agent và môi trường)

Policy (chính sách)

Cách agent hành xử tại một thời điểm: ánh xạ từ trạng thái cảm nhận được sang hành động cần thực hiện.

Reward signal

Xác định mục tiêu bài toán RL: điểm số học từ môi trường; định nghĩa sự kiện “tốt” và “xấu” cho agent.

Value function

Đánh giá điều gì tốt về **dài hạn**: tổng reward kỳ vọng tích lũy từ trạng thái hiện tại trở đi.

Model

Dùng cho **planning**: quyết định chuỗi hành động bằng cách mô phỏng tình huống tương lai trước khi trải nghiệm thực tế.

MDP

Markov Decision Processes (MDP) là khung toán cho ra quyết định trong môi trường có trạng thái, hành động, reward, xác suất. RL dùng MDP để tối đa hoá reward và tìm chiến lược quyết định tốt.

30.4 Markov Decision Processes (MDP)

Thành phần MDP

- **States (S)**: mọi tình huống agent có thể gặp.
- **Actions (A)**: lựa chọn hành động từ trạng thái cho trước.
- **Transition probabilities (P)**: xác suất chuyển trạng thái sau một hành động.
- **Rewards (R)**: phản hồi sau khi chuyển trạng thái, thể hiện mức “mong muốn” của kết quả.
- **Policy (π)**: chiến lược chọn hành động ở mỗi trạng thái để đạt reward cao.

30.5 Các bước trong quy trình RL

Sáu bước

1. Chuẩn bị agent với tập chiến lược ban đầu.
2. Quan sát môi trường và trạng thái hiện tại.
3. Chọn policy tối ưu theo trạng thái và thực hiện hành động.
4. Nhận reward hoặc penalty.
5. Cập nhật chiến lược nếu cần.
6. Lặp bước 2–5 đến khi agent học policy tối ưu.

30.6 Hai loại Reinforcement

Positive và negative reinforcement

- **Positive reinforcement:** hành động mong muốn $\Rightarrow$ reward $\Rightarrow$ tăng khả năng lặp lại hành động đó.
- **Negative reinforcement:** hành động tránh kết quả xấu $\Rightarrow$ bỏ kích thích tiêu cực (ví dụ robot tránh vật cản thành công $\Rightarrow$ ít gặp “mối đe dọa” hơn).

30.7 Loại thuật toán RL

Ví dụ thuật toán

Q-learning, policy gradient, Monte Carlo, ...

Model-free RL: học trực tiếp từ tương tác môi trường, không xây mô hình động học môi trường; thử nhiều hành động để học policy tối ưu reward — phù hợp môi trường lớn, phức tạp, thay đổi.

Model-based RL: xây mô hình động học môi trường để ra quyết định — phù hợp môi trường tĩnh, xác định rõ, khó thử nghiệm thực tế.

30.8 Ưu điểm

Ưu điểm

- Không cần hướng dẫn/ quy tắc định sẵn và can thiệp con người nhiều.
- Thích nghi nhiều môi trường (tĩnh và động).
- Giải bài toán ra quyết định, dự báo, tối ưu.
- Cải thiện dần khi tích lũy kinh nghiệm.

30.9 Nhược điểm

Nhược điểm

- Phụ thuộc chất lượng **hàm reward** — thiết kế kém thì mô hình không cải thiện.
- Thiết kế và tinh chỉnh RL phức tạp, cần chuyên môn.

30.10 Ứng dụng

30.10.1 Robotics

Robotics

Ra quyết định trong môi trường không chắc chắn; dùng cho bắt chước hành vi người, thao tác, điều hướng, di chuyển; robot thích nghi môi trường mới qua thử-sai.

30.10.2 Natural Language Processing

Natural Language Processing

Cải thiện chatbot trong hội thoại phức tạp; huấn luyện tóm tắt văn bản và tác vụ NLP khác.

30.11 Reinforcement Learning vs Supervised Learning

So sánh ngắn

Tiêu chí	Supervised	Reinforcement
Dữ liệu huấn luyện	Cặp đầu vào–đầu ra có nhãn.	Agent tương tác môi trường; học từ reward/penalty.
Mục tiêu	Dự đoán/phân loại từ feature.	Tối đa hoá reward tích lũy qua chuỗi hành động.
Tác vụ điển hình	Đầu ra có cấu trúc rõ (nhãn, giá trị số).	Ra quyết định phức tạp, chiến lược tối ưu theo thời gian.